MRSpinDynamics

Python User Manual

Python magnetic-resonance spin-dynamics workspace

Simulation workflows for magnetic-resonance spin dynamics across three useful scales: validated spin-1/2 Bloch kernels for NMR sequence and imaging work, small dense Hamiltonian models for explicitly quantum effects, and targeted extensions for pulsed NQR, ESR/EPR, exchange, relaxation, motion, and instrument-level field models.

June 2026

Contents

1 Physical Scope and Assumptions

PythonSpinDynamics is not a single universal spin simulator. It is a set of closely related models, each chosen because it gives a clean description of a common magnetic-resonance experiment. The central question throughout the manual is therefore: what is represented explicitly, and what is represented by an effective parameter?

The MATLAB-compatible workflow layer answers that question in the classical Bloch language. It models a bath of uncoupled spin-1/2 nuclei evolving in a static, spatially non-uniform, or time-varying main field $B_0(\mathbf{r}, t)$ and applied RF fields. The elementary object is an isochromat: a subensemble that shares an offset, RF sensitivity, relaxation constants, and probe response. Other namespaces add small quantum Hamiltonians, quadrupolar sites, ESR spectra, exchange, and microscopic relaxation where the physics requires those variables to be explicit.

Modeled directly in the core Bloch workflows. Independent spin-1/2 magnetization vectors; offset distributions from $B_0(\mathbf{r}, t)$; RF rotations; free precession; phenomenological T_1 and T_2 relaxation; probe transmit and receive filtering; optional deterministic radiation damping; optional motion through field maps.

Scalar-coupling extension. The `spin_dynamics.coupling` namespace adds small dense spin-1/2 systems with scalar J couplings, analytic low-field J-editing models, ideal TANGO-B filter curves, and initial SLIC simulations. These models are intentionally scoped and are documented in Chapter 5.

Pulsed NQR extension. The `spin_dynamics.nqr` namespace adds dense single-site quadrupolar spin tools for selective pulsed NQR, including spin-1 and spin-3/2 transition metadata, single-crystal and powder orientation averages, SLSE echo trains, two-frequency population transfer, EFG inhomogeneity, finite-window SLSE spectra, and weak-B0 perturbations. These models are documented in Chapter 7.

RFI and reference-channel rejection. The `spin_dynamics.interference` namespace models radio-frequency interference as a vector magnetic field, samples it with primary and reference coils, and removes it with masked, adaptive, windowed, or echo-aware joint estimators. The RFI tools are sequence agnostic; the NQR layer supplies SLSE and SORC acquisition masks on the same sample clock.

ESR/EPR extension. The `spin_dynamics.esr` namespace adds single-electron spin-1/2 ESR tools, including scalar or anisotropic g tensors, single-crystal and powder spectra, CW absorption/derivative lineshapes, static g-strain and field-strain distributions, rectangular pulsed ESR FID and Hahn-echo simulations with Liouville-space T_1/T_2 relaxation, and a first dense isotropic electron-nuclear hyperfine model. These tools are documented in Chapter 6.

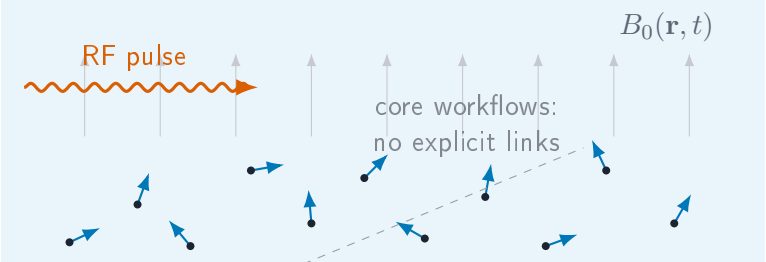
Chemical / site exchange extension. The `spin_dynamics.exchange` namespace adds Bloch-McConnell site exchange: a bath of magnetically distinct sites that swap magnetization at finite first-order kinetic rates while each site relaxes with its own T_1/T_2 and precesses at its own offset. It provides kinetic generators with detailed-balance checks, transverse lineshape coalescence, longitudinal mixing propagators, and encode-mix-detect T_2-T_2 relaxation exchange (REXSY) data that inverts to an exchange map. These tools are documented in Chapter 9.

Internal / susceptibility gradients. The `spin_dynamics.susceptibility` namespace generates the static internal field produced by magnetic-susceptibility contrast in porous media. It provides analytic two-dimensional cylindrical-grain off-resonance maps that feed the moving-isochromat pipeline and pore-space internal-gradient distributions for diffusion-in-internal-gradient studies. These tools are documented in Chapter 9.

Still outside the package scope. Dipolar coupling networks, arbitrary nonselective multi-quantum pulse sequences, shaped sample microstructure beyond the supplied field and density maps, microscopic spin noise, and large sparse coupled-spin simulations. Spin-spin effects enter the Bloch workflows only indirectly when represented by effective relaxation times such as T_2 or T_2^*. The Redfield/dipolar relaxation model treats fluctuating dipolar couplings as stochastic bath sources; it is not a coherent many-spin dipolar network. Site exchange is modeled phenomenologically by the `spin_dynamics.exchange` layer rather than from a microscopic exchange Hamiltonian.

This scope is deliberate. The package is strongest when the model boundary is chosen honestly: Bloch isochromats for independent subensembles, dense Hamiltonians for small quantum problems, and stochastic Redfield terms when the microscopic couplings are important but the bath itself is not being propagated.

1.1 Spin Bath Cartoon



Independent isochromats sample the local field and RF sensitivity.

1.2 Consequences for Users

- Use offset grids, field maps, or moving particles to represent spatially varying B_0, transmit B_1, and receive B_1 effects.
- Use relaxation constants to represent ensemble dephasing and energy exchange that are outside the explicit isochromat model.
- Use `spin_dynamics.coupling` when a small spin-1/2 scalar J-coupled model is the phenomenon of interest.
- Use `spin_dynamics.esr` for single-electron ESR spectra, pulsed ESR FID/echo simulations, static disorder studies, and simple isotropic electron-nuclear hyperfine doublets.
- Use `spin_dynamics.nqr` when selective pulsed NQR of a small quadrupolar site is the phenomenon of interest.
- Use `spin_dynamics.exchange` for Bloch-McConnell site exchange.

- Do not use the package for large coherent coupled-spin networks or arbitrary nonselective multi-quantum coherence pathways.

2 Overview

PythonSpinDynamics is a Python port of the MATLAB spin-dynamics simulation package. The MATLAB tree under `../MATLABSpinDynamics/Version_3/code` remains the reference for numerical behavior, while this workspace provides a typed, testable Python API for simulations, validation fixtures, examples, and new analysis workflows.

Read the manual in layers. Chapters 1–4 define the model boundaries and shared conventions. Chapters 5, 6, and 7 introduce the explicitly quantum extensions. Chapter 8 presents the unified experiment facade – the recommended declarative entry point that ties the sample, hardware, sequence, and acquisition together, plans a run, and saves it. Chapter 9 then documents the underlying experiment-level workflows the facade delegates to: CPMG, FID, imaging, diffusion, motion, field sources, phase cycling, exchange, analysis, and optimization. The final chapters collect runnable examples, validation status, API tables, and known gaps.

The supported surface includes ideal, tuned, untuned, and matched-probe CPMG workflows; finite echo trains; FID; phase-encoded and frequency-encoded imaging; diffusion and moving-isochromat simulations; WURST and UDD timing helpers; radiation damping; receiver noise; inverse-Laplace analysis; compact optimization scaffolds; small spin-1/2 scalar-coupling models; pulsed NQR; ESR/EPR; Bloch-McConnell exchange; susceptibility fields; and opt-in Liouville-space relaxation models.

2.1 Package Layout

Path	Purpose
`src/spin_dynamics/`	Installable Python package.
`examples/`	CLI examples and plotting scripts.
`tests/`	Smoke tests, validation tests, and unit tests.
`validation/fixtures/`	MATLAB/Octave fixture arrays.
`validation/octave/`	Fixture generation scripts.
`docs/python_api/`	Lightweight Markdown API notes.
`docs/validation_results.md`	Current validation status and tolerance notes.
`docs/radiation_damping.md`	Focused radiation-damping model notes.

3 Installation and Test Runs

3.1 Editable Install

PythonSpinDynamics requires Python 3.10 or newer. The core install depends on NumPy 1.24 or newer and does not require MATLAB at runtime. MATLAB is only needed to regenerate the complete MATLAB reference fixture set; Octave can regenerate the smaller dependency-light fixtures.

From `PythonSpinDynamics`, install the package into an active environment:

```
python -m pip install -e .
```

Optional extras are provided for development, plotting, benchmarking, and SciPy-backed optimization:

```
python -m pip install -e ".[dev,opt,plot,bench]"
```

The core package depends only on NumPy. The `opt` extra enables SciPy-backed continuous optimizers, non-negative inverse-Laplace solves, and MATLAB `.mat` result import/export helpers. The `plot` extra enables the Matplotlib examples.

3.2 Tests

Use the smoke tier during normal editing:

```
python -m unittest tests.smoke_tests
```

Run the full validation suite before committing numerical or workflow changes:

```
python -m unittest discover -s tests
```

4 Model Conventions

This chapter collects conventions that appear across several workflows. The details are intentionally close to the equations, because most user mistakes in this package come from mixing coordinate conventions, timing conventions, or relaxation levels of description.

4.1 Coherence Ordering

Low-level kernels preserve MATLAB's coherence ordering:

$$\mathbf{M} = \begin{bmatrix} M_0 & M_- & M_+ \end{bmatrix}^{\mathsf{T}}.$$

This ordering differs from many Bloch-vector presentations, so new kernels and validation tests should state the expected ordering explicitly.

4.2 Normalized Time

The core MATLAB routines usually normalize time to the nominal RF amplitude $\omega_1 = 1$. In these units, a 90-degree hard pulse has duration

$$t_{90} = \frac{\pi}{2},$$

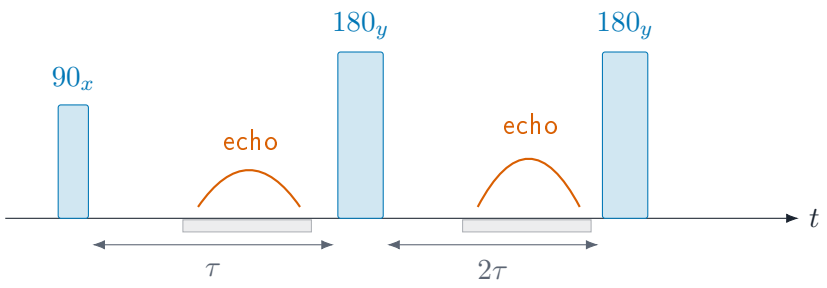
and a 180-degree hard pulse has duration

$$t_{180} = \pi.$$

Workflow helpers that expose physical seconds convert into these normalized units before calling the lower-level kernels.

4.3 Pulse Timing Cartoon

A minimal spin-echo or CPMG building block is a sequence of RF pulses, free precession windows, and acquisition windows. The package represents these as ordered intervals with pulse matrices and free-precession delays.



4.4 Effective Rotations

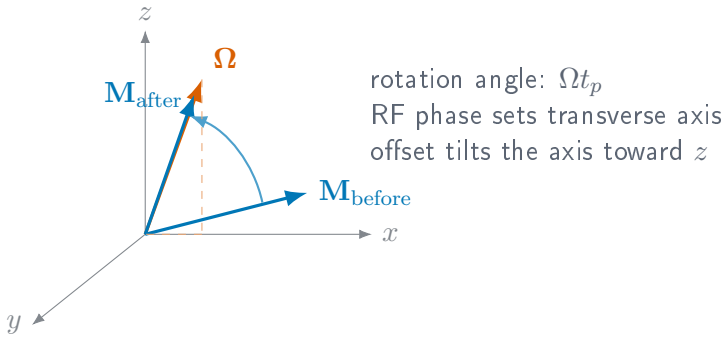
For an isochromat with normalized offset $\Delta\omega$, RF amplitude ω_1, phase ϕ, and pulse duration t_p, the rotating-frame effective field is

$$\boldsymbol{\Omega} = \begin{bmatrix} \omega_1 \cos\phi \\ \omega_1 \sin\phi \\ \Delta\omega \end{bmatrix}, \qquad \Omega = \sqrt{\omega_1^2 + \Delta\omega^2}.$$

The magnetization update is a rotation by angle Ωt_p about $\boldsymbol{\Omega}/\Omega$. The Python rotation helpers expose this calculation through `spin_dynamics.core.rotations` and return array shapes chosen to match the MATLAB reference fixtures.

4.5 Rotation Cartoon

In the rotating frame, a rectangular RF segment rotates the magnetization about an effective axis set by RF phase, RF amplitude, and off-resonance offset.



4.6 Initial Magnetization and Relaxation

Relaxation appears in PythonSpinDynamics at three different levels. The Bloch workflows use T_1 and T_2 factors attached to free-precession intervals. Prepolarization helpers use the same longitudinal recovery law to prepare a non-thermal starting magnetization. The shared `spin_dynamics.relaxation` module adds Liouville-space relaxation models, including opt-in Redfield/dipolar dissipators for spin-1/2 NMR and quadrupolar NQR, plus stochastic collision models for gas-wall relaxation. Keeping these levels separate is important: a fitted T_2, a BPP rate estimate, a Redfield bath model, and a wall-collision model can describe related decays, but they enter the sequence simulator at different points.

4.6.1 Bloch Relaxation in Isochromat Workflows

In the MATLAB-compatible Bloch kernels, relaxation is diagonal in the local magnetization basis. Free-precession intervals use the usual exponential factors:

$$M_{xy}(t + \Delta t) = M_{xy}(t) \exp\left(-\frac{\Delta t}{T_2}\right) \exp(-\mathrm{i}\Delta\omega\, \Delta t),$$

$$M_z(t+\Delta t) = M_{\mathrm{th}} + (M_z(t) - M_{\mathrm{th}}) \exp\left(-\frac{\Delta t}{T_1}\right).$$

Finite acquisition workflows assemble these intervals from pulse-sequence parameters and then call the `arb1O`-style kernels.

4.6.2 Preparing Longitudinal Magnetization

Many compact and low-field experiments do not begin at the thermal equilibrium magnetization of the detection field. The `spin_dynamics.prepolarization` module computes prepared longitudinal magnetization values in the same normalized units used by the sequence kernels. For a sample relaxing in a polarizing field B_{pol}, then detected at B_{det}, the full polarizer equilibrium in detection-field units is

$$M_{\mathrm{pol,eq}} = M_{\mathrm{det,eq}} \frac{B_{\mathrm{pol}}}{B_{\mathrm{det}}}.$$

Finite prepolarization time is then just the usual longitudinal buildup:

$$M_0(t_{\mathrm{pol}}) = M_{\mathrm{pol,eq}} + (M_{\mathrm{init}} - M_{\mathrm{pol,eq}}) \exp\left(-t_{\mathrm{pol}}/T_1\right).$$

For flow or sample transport, `prepolarized_flow_state` first converts a path length and local speed into a residence time. The returned `PrepolarizedMagnetization` object exposes `m0` and `mth` arrays that can be passed into the lower-level kernels or used to initialize moving isochromats.

4.6.3 BPP Scalar Rate Estimates

For prepolarization and compact low-field estimates, the same module also provides a BPP-style scalar relaxation model. The rotational spectral density is

$$J(\omega) = \frac{2\tau_c}{1+\omega^2\tau_c^2},$$

with an Arrhenius temperature dependence

$$\tau_c(T) = \tau_{\mathrm{ref}} \exp\left[\frac{E_a}{R}\left(\frac{1}{T} - \frac{1}{T_{\mathrm{ref}}}\right)\right].$$

The default rate ratios are the common dipolar-style combinations

$$R_1 = C\left[J(\omega_0) + 4J(2\omega_0)\right],$$

$$R_2 = C\left[\frac{3}{2}J(0) + \frac{5}{2}J(\omega_0) + J(2\omega_0)\right],$$

where C absorbs the spin-pair constants and convention-specific prefactors. Custom coefficient triples $(J(0), J(\omega_0), J(2\omega_0))$ can be supplied for other dipolar conventions or phenomenological fits.

```
import numpy as np
from spin_dynamics.prepolarization import prepolarized_state
from spin_dynamics.relaxation import BPPRelaxationModel

model = BPPRelaxationModel(
    angular_frequency_rad_per_s=2*np.pi*20e6,
    tau_ref_seconds=8e-9,
```

```
    coupling_scale_per_second2=5e8,
    activation_energy_j_per_mol=16e3,
)
rates = model.rates([280.0, 300.0, 320.0])
prepared = prepolarized_state(
    polarizing_field_tesla=0.1,
    detection_field_tesla=50e-6,
    prepolarization_time_seconds=2.0,
    t1_seconds=rates.t1_seconds,
)
```

4.6.4 Prepolarized Spin-Lock ($T_{1\rho}$) Dispersion

A spin-lock experiment measures relaxation in the presence of a resonant RF field: a $(\pi/2)_{-y}$ pulse tips the prepared magnetization onto the spin-lock axis, an on-resonance field of nutation frequency $\omega_1 = \gamma B_1$ holds it for a time T_{SL}, and the surviving locked magnetization is read out. The on-resonance dipolar rate is

$$\frac{1}{T_{1\rho}} = C\left[\tfrac{3}{2}J(2\omega_1) + \tfrac{5}{2}J(\omega_0) + J(2\omega_0)\right],$$

so, unlike T_2, $T_{1\rho}$ probes the spectral density at the operator-set spin-lock frequency $2\omega_1$ (typically 0.1–$3\,\mathrm{kHz}$). As $\omega_1 \to 0$ the expression reduces exactly to the transverse rate R_2 above, and it disperses downward once $\omega_1 \gtrsim 1/\tau_c$. Because the rotational correlation time grows with molecular size through the Stokes–Einstein–Debye relation $\tau_c = 4\pi\eta a^3 f/(3k_BT)$, sweeping the hydrodynamic radius a sweeps the dispersion: fast, small tumblers stay flat while slow, large ones show a strong $T_{1\rho}$ dispersion in the kHz band.

The helper `t1rho_relaxation_rate` returns $R_{1\rho}$, and `simulate_prepolarized_t1rho` composes prepolarization, the spin lock, and readout into a size × lock grid. The example `plot_t1rho_molecular_size_dispersion.py` maps a set of molecule radii to their dispersion curves and prepolarized contrast.

```
import numpy as np
from spin_dynamics.relaxation import (
    simulate_prepolarized_t1rho,
    stokes_einstein_debye_correlation_time,
)

radii_m = np.array([0.5, 2.0, 6.0, 20.0]) * 1e-9
tau_c = stokes_einstein_debye_correlation_time(
    radii_m, viscosity_pa_s=0.1, temperature_kelvin=310.0
)
experiment = simulate_prepolarized_t1rho(
    spin_lock_angular_rad_per_s=2*np.pi*np.geomspace(20.0, 20e3, 160),
    larmor_angular_rad_per_s=2*np.pi*20e6,
    correlation_time_seconds=tau_c,
    spin_lock_time_seconds=0.03,
    coupling_scale_per_second2=2e6,
    polarizing_field_tesla=0.3,
    detection_field_tesla=0.02,
    prepolarization_time_seconds=3.0,
    t1_seconds=np.array([8.7, 0.2, 0.008, 5.0]),
)
# experiment.r1rho_per_second and experiment.locked_signal are (n_sizes, n_locks)
```

4.6.5 Liouville-Space Relaxation Models

The `spin_dynamics.relaxation` module provides shared Liouville-space utilities for spin-1/2 NMR and quadrupolar NQR simulations. The historical scalar model remains available as the phenomenological relaxation class and is also re-exported as `NQRRelaxationModel` for compatibility. It damps populations and coherences in the Hamiltonian eigenbasis using supplied T_1 and T_2 values. This is still a phenomenological model, but it now lives in the shared relaxation layer rather than in NQR-specific code.

For microscopic relaxation, `RedfieldDipolarRelaxationModel` builds a secular Markovian dissipator from fluctuating dipolar bath sources. A source is specified by a target–bath vector, gyromagnetic ratios, bath spin, and optional explicit coupling; if the coupling is omitted it is computed from $\mu_0 h \gamma_a \gamma_b / (4\pi r^3)$. The motional model determines how the dipolar covariance is averaged before the Redfield spectral-density step: `RigidSolidMotionalAveraging` keeps the fixed-frame anisotropy and is the natural choice for NQR solids, while `IsotropicLiquidMotionalAveraging` rotationally averages the covariance for liquid-state NMR.

```
from spin_dynamics.relaxation import (
    DipolarRelaxationSource,
    IsotropicLiquidMotionalAveraging,
    RedfieldDipolarRelaxationModel,
)

source = DipolarRelaxationSource(
    vector_angstrom=(1.52, 0.0, 0.0),
    gamma_target_hz_per_t=42.57747892e6,
    gamma_bath_hz_per_t=42.57747892e6,
    bath_spin=0.5,
)
relaxation = RedfieldDipolarRelaxationModel.from_dipolar_sources(
    0.5,
    (source,),
    motion=IsotropicLiquidMotionalAveraging(correlation_time_seconds=2.5e-12),
)
```

Gas-wall relaxation is another microscopic Liouville model, but it is not a dipolar Redfield bath. The wall-collision model treats wall encounters as a Poisson sequence of quantum jumps. Kinetic theory supplies the encounter rate

$$k_{\mathrm{wall}} = p_{\mathrm{acc}} \frac{\bar{v}}{4} \frac{S}{V}, \qquad \bar{v} = \sqrt{\frac{8 k_B T}{\pi m}},$$

where p_{acc} is the probability that a kinetic wall encounter actually samples the relaxing surface. Each encounter applies an isotropic depolarization channel with probability p_{dep}, giving the continuous Liouville generator $k_{\mathrm{wall}}(\Phi - I)$. In the weak-loss limit this has the familiar surface-relaxivity form

$$R_{\mathrm{wall}} = p_{\mathrm{dep}} k_{\mathrm{wall}} = \rho_{\mathrm{eq}} \frac{S}{V}, \qquad \rho_{\mathrm{eq}} = \frac{p_{\mathrm{acc}} p_{\mathrm{dep}} \bar{v}}{4},$$

but the model exposes the microscopic ingredients instead of asking the user to choose T_1 directly.

```
from spin_dynamics.relaxation import (
    WallCollisionRelaxationModel,
    sphere_surface_to_volume_per_m,
)

surface_to_volume = float(sphere_surface_to_volume_per_m(10e-3))
relaxation = WallCollisionRelaxationModel.from_geometry(
```

```
    0.5,
    surface_to_volume_per_m=surface_to_volume,
    temperature_kelvin=295.0,
    mass_amu=128.9047808611,
    accommodation_probability=1.0,
    depolarization_probability=1e-8,
)
print(relaxation.t1_seconds, relaxation.collision_rate_per_second)
```

4.6.6 Choosing a Relaxation Description

The Redfield model is non-default and opt-in. Existing pulse-sequence simulators keep their historical behavior when `relaxation=None`. The Redfield plotting examples show the two main dipolar regimes: isotropic-liquid proton CPMG and rigid-solid $^{14}\mathrm{N}$ SLSE. The wall-relaxation example covers a different microscopic regime, polarized gas whose dominant relaxation source is repeated surface contact. Use fitted T_1/T_2 values when the experiment only needs the observed decay law, use BPP rates when a compact temperature- or correlation-time trend is the goal, and use Redfield/dipolar relaxation when the geometry and motional regime of the bath are part of the physics being tested. Use the wall-collision model when container size, gas temperature, and per-collision spin loss are the physics being tested.

4.7 Echo Conversion

The echo utilities direct-sum a received spectrum $S(\Delta\omega)$ over the offset grid:

$$e(t) = \sum_k S(\Delta\omega_k)\exp(\mathrm{i}\Delta\omega_k t)\,\Delta\omega.$$

The implementation uses the same direct-sum convention as the MATLAB `calc_time_domain_echo` reference path. FID traces use the corresponding FID conversion helper.

4.8 Radiation Damping

The deterministic radiation-damping back-action model follows the local Measurements text convention:

$$T_{\mathrm{rd}} = \frac{2}{\gamma\mu_0\eta M_0 Q},$$

where γ is the gyromagnetic ratio, η is the magnetic-energy fill factor, M_0 is the equilibrium magnetization density, and Q is the loaded probe quality factor.

The analytic on-resonance hard-pulse FID envelope used by the test suite is

$$|M_{xy}(t)| = \frac{M_0}{\cosh\left(t/T_{\mathrm{rd}} - \log\tan(\theta/2)\right)}.$$

The finite-sequence implementation can add feedback during free-precession and acquisition windows, and can optionally apply an operator-split approximation during RF pulse matrices.

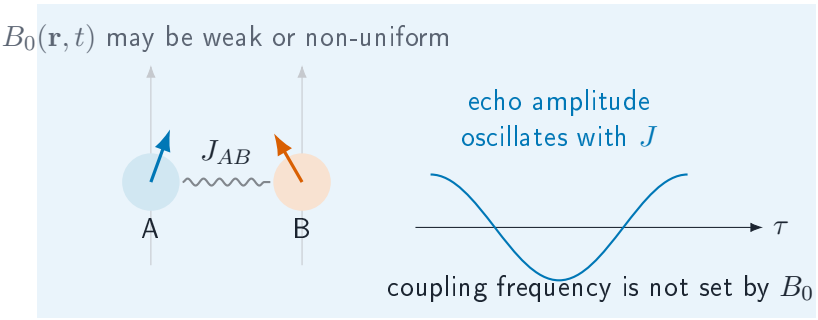
5 Scalar J-Coupling Models

Scalar J coupling is field independent, so it can remain visible in weak or grossly inhomogeneous magnetic fields where chemical-shift resolution is compressed. The `spin_dynamics.coupling` namespace adds a scoped set of spin-1/2 tools for these cases. It is separate from the MATLAB-compatible Bloch workflow layer: Bloch workflows propagate independent isochromats, while the coupling layer propagates small dense Hilbert-space systems or evaluates closed-form J-editing response curves.

Use this layer for: heteronuclear low-field J-editing curves, known-J spectrum fitting, ideal TANGO-B filter selectivity, exploratory two-spin or few-spin scalar-coupled Hamiltonian simulations, and multinuclear zero/ultra-low-field J-coupled spectra with mixed spin quantum numbers, quadrupolar nuclei, and per-spin relaxation (Section 5.6).

Current limits: dense matrices only (few-spin Hilbert spaces); no chemical exchange in this layer; no large sparse coupled-spin networks; no full pulse-sequence compiler for arbitrary multi-quantum pathways. The multinuclear extension models quadrupolar relaxation with a phenomenological per-spin "extended T1/T2" superoperator rather than a full quadrupolar Redfield tensor.

5.1 J-Coupled Spin Cartoon



5.2 Dense Spin-1/2 Hamiltonians

A coupled system is built from offsets ν_i in hertz and symmetric scalar couplings J_{ij} in hertz:

```
from spin_dynamics.coupling import (
    coupled_spin_system,
    isotropic_j_hamiltonian,
    zeeman_hamiltonian,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
    labels=["A", "B"],
)
hamiltonian = zeeman_hamiltonian(system) + isotropic_j_hamiltonian(system)
```

Hamiltonian builders return angular-frequency matrices in radians per second and use spin-1/2 operators with eigenvalues $\pm 1/2$. The rotating-frame offset Hamiltonian is

$$H_Z = 2\pi \sum_i \nu_i I_{iz}.$$

For weakly coupled systems, the secular scalar Hamiltonian is

$$H_J^{\mathrm{sec}} = 2\pi \sum_{i<j} J_{ij} I_{iz} I_{jz},$$

while the isotropic scalar Hamiltonian is

$$H_J^{\mathrm{iso}} = 2\pi \sum_{i<j} J_{ij} \left(I_{ix} I_{jx} + I_{iy} I_{jy} + I_{iz} I_{jz} \right).$$

The dense propagator evaluates

$$\rho(t) = U(t)\rho(0)U^{\dagger}(t), \qquad U(t) = \exp\left(-\mathrm{i}Ht\right),$$

and is intended for small systems where dense matrices are acceptable.

5.3 Inhomogeneous B0/B1 Isochromats

The bridge between the coupled-spin layer and the Bloch workflow layer is a coupled isochromat ensemble. Each isochromat contains the same scalar-coupled spin network, but samples a different local field and receive weight. For isochromat k, the local spin offsets are

$$\nu_{i,k} = \nu_i^{(0)} + s_i \, \Delta\nu_k,$$

where $\nu_i^{(0)}$ is the base spin offset, $\Delta\nu_k$ is the local B0 offset in hertz, and s_i is a per-spin offset scale. The default $s_i = 1$ adds the same field offset to every spin. Heteronuclear or carrier-specific models can supply different scales.

During an RF or spin-lock interval, the nominal nutation frequency is scaled by the local transmit field:

$$\omega_{1,k} = b_{1,k}^{\mathrm{tx}} \omega_1.$$

The ensemble signal is accumulated as

$$S = \sum_k w_k b_{1,k}^{\mathrm{rx}} \, \mathrm{Tr}(\rho_k D),$$

where w_k is the isochromat weight, $b_{1,k}^{\mathrm{rx}}$ is the local receive scale, and D is the detection operator.

```
from spin_dynamics.coupling import (
    coupled_isochromat_ensemble,
    free_precession_step,
    rf_step,
    simulate_coupled_isochromat_sequence,
)

ensemble = coupled_isochromat_ensemble(
```

```
    system,
    b0_offsets_hz=[-2.0, 0.0, 2.0],
    weights=[0.25, 0.5, 0.25],
    b1_tx_scale=[0.9, 1.0, 1.1],
    b1_rx_scale=[1.0, 1.0, 0.8],
)

result = simulate_coupled_isochromat_sequence(
    ensemble,
    [
        rf_step(duration=0.25, nutation_hz=1.0, phase=1.5708),
        free_precession_step(duration=0.01),
    ],
    initial_axis="x",
    detect_axis="x",
)
```

For time-varying fields, each sequence step may override the ensemble B0 offsets or B1 transmit scale. This keeps the first implementation close to the existing isochromat approach: the coupled-spin Hilbert space is dense and small, while field inhomogeneity is represented by an outer ensemble sum.

5.4 Low-Field J-Editing

Analytic J-editing helpers model response curves as sums of field-independent cosine modulations:

$$s(\tau) = b + \sum_m a_m \cos^{p_m}(2\pi N J_m \tau)\,,$$

where τ is the encoding time, N is the number of encoding cycles, J_m is a known or hypothesized coupling in hertz, a_m is its amplitude, and p_m is an optional multiplicity exponent. Proton-detected models use $p_m = 1$. Carbon-detected CH_n groups use $p_m = n$.

```
from spin_dynamics.coupling import (
    carbon_detected_j_modulation,
    fit_known_j_spectrum,
    j_modulation_curve,
)

signal = j_modulation_curve(
    encoding_times,
    couplings_hz=[125.0, 160.0],
    amplitudes=[0.85, 0.15],
)

carbon_signal = carbon_detected_j_modulation(
    encoding_times,
    couplings_hz=[125.0, 160.0],
    abundances=[0.85, 0.15],
    proton_counts=[2, 1],
)

fit = fit_known_j_spectrum(
    encoding_times,
    signal,
```

```
    couplings_hz=[125.0, 160.0],
    include_background=False,
)
```

`fit_known_j_spectrum` solves a linear least-squares problem once the candidate J_m values are supplied. It returns amplitudes, optional background, fitted signal, residual, rank, and residual norm.

5.5 TANGO-B Filtering

The ideal TANGO-B helper evaluates the coupled-spin transverse filter envelope

$$A(J; t_d) = \sin^2(\pi J t_d)\,.$$

When a target coupling $J_\star$ and positive odd order n are supplied, the delay is selected as

$$t_d = \frac{n}{2J_\star}.$$

```
from spin_dynamics.coupling import tango_b_filter

response = tango_b_filter(
    couplings_hz,
    target_coupling_hz=160.0,
    order=3,
)
```

This model is idealized: it reports the scalar-coupling selectivity envelope, not RF imperfections, relaxation, diffusion, or probe filtering.

5.6 Zero/Ultra-Low-Field Multinuclear J-Coupled Spectra

At Earth's field ($\sim 50\,\mu$T) chemical shifts scale away and the spectrum is a *J-coupled spectrum* (JCS): every nucleus sits within a few kHz, so the frequency separations $|\nu_i - \nu_j|$ can be comparable to the couplings and spins are often *strongly* coupled. The multinuclear layer therefore works in the lab frame with absolute Larmor frequencies $\nu_i = \gamma_i B_0$ and the full isotropic coupling $2\pi J_{ij}\,\mathbf{I}_i\cdot\mathbf{I}_j$, over a tensor-product Hilbert space that mixes spin quantum numbers (e.g. spin-1/2 ^{1}H/^{19}F with spin-1 ^{14}N).

A quadrupolar nucleus such as ^{14}N relaxes quickly, and its rate $R(^{14}\mathrm{N}) = R_1 = R_2$ broadens the splittings of any spin J-coupled to it: as R grows the multiplet moves through resolved → coalesced → self-decoupled singlet (Altenhof et al., *J. Magn. Reson.* 355 (2023) 107540). The rate is set by molecular tumbling through the single-correlation-time quadrupolar model, which in the extreme-narrowing limit relevant at Earth's field is

$$R_1 = R_2 = \frac{3\pi^2}{10}\,\frac{2I+3}{I^2(2I-1)}\left(1+\tfrac{\eta^2}{3}\right) C_Q^2\,\tau_c,$$

with $C_Q = e^2qQ/h$ the quadrupole coupling constant. Relaxation is applied with a per-spin phenomenological superoperator that is exact for spin-1/2 (ladder channels $\sqrt{R_1/2}\,I^{\pm}$ and dephasing $\sqrt{2R_2 - R_1}\,I_z$).

```
import numpy as np
from spin_dynamics.coupling import (
    multinuclear_system,
```

```
    multinuclear_quadrupolar_rates,
    simulate_zulf_spectrum,
)

# 1H-19F-14N with a strong 2J(F,N); 14N is the last (quadrupolar) site.
couplings = np.zeros((3, 3))
couplings[0, 1] = couplings[1, 0] = 8.0    # J(H,F)
couplings[1, 2] = couplings[2, 1] = 37.0   # J(F,N)
system = multinuclear_system(["1H", "19F", "14N"], couplings, b0_tesla=50e-6)

# Derive R(14N) from the quadrupole coupling and a rotational correlation time.
r1, r2 = multinuclear_quadrupolar_rates(
    system,
    correlation_time_seconds=3e-12,
    quadrupole_coupling_hz=4.0e6,
    asymmetry=0.4,
    spin_half_rate_hz=0.3,
)
spectrum = simulate_zulf_spectrum(
    system, r1_per_second=r1, r2_per_second=r2,
    dwell_seconds=2e-4, n_points=32768,
    detect_indices=system.indices_for_isotope("19F"),
    apodization_hz=0.8,
)
# spectrum.frequencies_hz / spectrum.spectrum are the 19F J-coupled spectrum.
```

The examples `plot_zulf_quadrupolar_jcoupling.py` (sweeping $R(^{14}\mathrm{N})$ directly, with heteronuclear and homonuclear panels) and `plot_zulf_quadrupolar_relaxation.py` (deriving $R(^{14}\mathrm{N})$ from C_Q and τ_c) reproduce the coalescence and self-decoupling behaviour.

5.7 SLIC Response

Spin-lock induced crossing (SLIC) is modeled by adding an RF spin-lock Hamiltonian to the static coupled-spin Hamiltonian:

$$H_{\mathrm{SLIC}} = H_Z + H_J^{\mathrm{iso}} + 2\pi\omega_1 \sum_i I_{ix}.$$

The helper scans spin-lock nutation frequencies ω_1 in hertz and returns the remaining transverse magnetization and its corresponding dip:

```
from spin_dynamics.coupling import (
    coupled_spin_system,
    simulate_slic_spectrum,
    two_spin_slic_transfer_time,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
)
duration = two_spin_slic_transfer_time(offset_difference_hz=0.7)
result = simulate_slic_spectrum(
    system,
    nutation_frequencies_hz,
```

```
    spin_lock_time=duration,
)
```

For a nearly equivalent two-spin system, the SLIC dip is expected near the coupling frequency. The helper is exploratory and dense; broad homonuclear networks will need sparse or specialized methods.

6 Electron Spin Resonance Models

The `spin_dynamics.esr` namespace adds an ESR/EPR extension for single-electron spin-1/2 systems. The first layer is intentionally compact: it uses dense matrices, explicit g-tensor axes, and small product Hilbert spaces for hyperfine examples. Hamiltonians are expressed in radians per second, while public resonance frequencies, linewidths, and hyperfine constants are expressed in hertz.

Use this layer for: single-electron ESR resonance calculations, anisotropic g-tensor single-crystal and powder spectra, CW absorption or first-derivative display, static g-strain and field-strain broadening, rectangular pulsed ESR FID and Hahn-echo simulations, Liouville-space T_1/T_2 relaxation, first electron-nuclear isotropic hyperfine doublets, four-pulse DEER/PELDOR distance measurement with $P(r)$ recovery (Section 6.6), and ESEEM, HYSCORE, and ENDOR of an $I = 1/2$, 1, or $3/2$ nucleus including the nuclear quadrupole interaction (Section 6.7).

Current limits: electron spin $S = 1/2$ only in the main ESR system; dense matrices only. The DEER model uses the secular point-dipole coupling in the weak-coupling observer/pump regime, and the ESEEM/HYSCORE/ENDOR model treats a single $I \leq 3/2$ nucleus with secular plus pseudosecular hyperfine and a field-collinear quadrupole tensor. No anisotropic A tensors, tilted quadrupole tensors, multiple coupled nuclei, exchange, higher-spin zero-field splitting, saturation, or microwave resonator model yet.

6.1 Zeeman Hamiltonian and g Tensors

The single-electron Zeeman Hamiltonian is

$$H_Z = 2\pi \frac{\mu_B}{h} \mathbf{B}_0^{\mathsf{T}} \mathbf{g} \mathbf{S},$$

where μ_B/h is exposed as `BOHR_MAGNETON_HZ_PER_T`. The g tensor may be supplied as a scalar, a three-component principal-axis vector, or a full 3×3 matrix. For a static-field direction $\hat{\mathbf{n}}$, the effective g value is

$$g_{\text{eff}}(\hat{\mathbf{n}}) = \left| \mathbf{g}^{\mathsf{T}} \hat{\mathbf{n}} \right|,$$

and the spin-1/2 transition frequency is

$$\nu_{\text{ESR}} = \frac{\mu_B}{h} B_0 \, g_{\text{eff}}.$$

```
from spin_dynamics.esr import (
    ESRSpinSystem,
    effective_g_value,
    resonance_field_tesla,
    resonance_frequency_hz,
)

system = ESRSpinSystem(g_tensor=[2.00, 2.08, 2.24])
g_z = effective_g_value(system, [0.0, 0.0, 1.0])
nu = resonance_frequency_hz(system, [0.0, 0.0, 0.34])
b_res = resonance_field_tesla(system, microwave_frequency_hz=9.5e9)
```

6.2 CW Spectra, Lineshapes, and Disorder

Frequency-swept and field-swept helpers broaden orientation-resolved ESR lines with Gaussian or Lorentzian profiles. The same helpers can return absorption or first-derivative CW-style spectra:

```
from spin_dynamics.esr import simulate_field_sweep

result = simulate_field_sweep(
    system,
    microwave_frequency_hz=9.5e9,
    orientations="powder",
    broadening_tesla=0.2e-3,
    lineshape="lorentzian",
    detection_mode="derivative",
)
```

Static disorder is represented by weighted samples. The convenience `static_disorder_grid` builds Gaussian quadrature-like samples for diagonal g-strain and applied-field offsets:

```
from spin_dynamics.esr import (
    simulate_field_sweep_distribution,
    static_disorder_grid,
)

samples = static_disorder_grid(
    system,
    g_std=[0.0, 0.0, 0.005],
    field_std_tesla=0.1e-3,
    g_points=5,
    field_points=5,
)

strained = simulate_field_sweep_distribution(
    samples,
    microwave_frequency_hz=9.5e9,
    orientations="powder",
    broadening_tesla=0.05e-3,
)
```

6.3 Pulsed ESR

The pulsed ESR helpers use a single rotating frame and a rotating-wave approximation. The `nutation_hz` parameter is the on-resonance spin-1/2 Rabi rate for the selected microwave B_1 direction, so a rectangular pulse of flip angle θ has duration

$$t_p = \frac{\theta}{2\pi\nu_1}.$$

In particular, a 90-degree pulse has duration $1/(4\nu_1)$.

```
import numpy as np

from spin_dynamics.esr import (
    flip_angle_duration,
    simulate_fid,
```

```
    simulate_hahn_echo,
)

carrier = resonance_frequency_hz(system, [0.0, 0.0, 0.34])
nu1 = 5e6
t90 = flip_angle_duration(np.pi / 2.0, nu1)
t180 = flip_angle_duration(np.pi, nu1)

fid = simulate_fid(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    pulse_duration_seconds=t90,
    times_seconds=np.linspace(0.0, 5e-6, 256),
    rf_frequency_hz=carrier,
)

echo = simulate_hahn_echo(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    excitation_duration_seconds=t90,
    refocus_duration_seconds=t180,
    tau_seconds=2e-6,
    times_seconds=np.linspace(0.0, 4e-6, 256),
    rf_frequency_hz=carrier,
)
```

6.4 Pulsed Relaxation

For physical T_1/T_2 relaxation during pulses, free evolution, and acquisition, pass an `ESRRelaxationModel`. The model acts in Liouville space on the trace-zero high-temperature density-matrix deviation: T_1 damps population differences while preserving trace, and T_2 damps coherences.

```
from spin_dynamics.esr import ESRRelaxationModel

relaxed = simulate_fid(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    pulse_duration_seconds=t90,
    times_seconds=np.linspace(0.0, 5e-6, 256),
    rf_frequency_hz=carrier,
    relaxation=ESRRelaxationModel(t1_seconds=50e-6, t2_seconds=3e-6),
)
```

The older `t2_seconds` argument in the pulsed helpers is a simple post-propagation envelope retained for quick examples. When using `ESRRelaxationModel`, leave `t2_seconds=inf` to avoid double-counting coherence decay.

6.5 Isotropic Hyperfine Coupling

The first hyperfine layer models one electron spin-1/2 coupled isotropically to one or more nuclei:

$$H = H_{eZ} + H_{nZ} + 2\pi \sum_k A_k \, \mathbf{S} \cdot \mathbf{I}_k,$$

with A_k in hertz. The helper scans the applied field, diagonalizes the full dense electron-nuclear Hamiltonian at each point, and sums ESR-active transitions according to the microwave B_1 matrix element.

```
from spin_dynamics.esr import (
    NuclearSite,
    electron_nuclear_system,
    simulate_hyperfine_field_sweep,
)

hf_system = electron_nuclear_system(
    [20e6],
    nuclei=[NuclearSite("H1", isotope="1H", gamma_hz_per_t=42.577e6)],
    g_tensor=2.0023,
)

hf_sweep = simulate_hyperfine_field_sweep(
    hf_system,
    microwave_frequency_hz=9.5e9,
    broadening_hz=0.5e6,
)
```

For one spin-1/2 nucleus in the high-field limit, the ESR line splits by approximately A in frequency, or by

$$\Delta B \approx \frac{A}{(\mu_B/h)g}$$

in a field-swept spectrum.

6.6 Pulsed Dipolar ESR: DEER/PELDOR

Double electron-electron resonance (DEER, also PELDOR) measures the dipolar coupling between two electron spins, and hence the nanometre distance between them. The `spin_dynamics.esr.dipolar` and `spin_dynamics.esr.deer` modules provide the point-dipole coupling, the four-pulse DEER forward model, and regularized recovery of a distance distribution $P(r)$.

The secular dipolar coupling between two spectrally distinct electron spins is

$$\nu_{dd}(r,\theta) = \nu_\perp(r)\,(1 - 3\cos^2\theta), \qquad \nu_\perp(r) = \frac{\mu_0}{4\pi}\,\frac{g_a g_b \mu_B^2}{h\, r^3},$$

which for two free-electron g values gives the canonical $\nu_\perp \approx 52.04\ \mathrm{MHz}$ at $r = 1\ \mathrm{nm}$ (`dipolar_constant_hz_nm3`). A pump pulse on the partner spin inverts the local dipolar field, modulating the observer echo. For a single pair the DEER form factor is

$$F(t) = 1 - \lambda\big(1 - \cos(2\pi\nu_{dd}(r,\theta)\,t)\big),$$

where the modulation depth $\lambda = \sin^2(\beta/2)$ is set by the pump flip angle β. Averaging over an isotropic orientation distribution gives the powder kernel, whose dipolar (Pake) spectrum has its singularity at $\nu_\perp$ and an edge at $2\nu_\perp$.

```
import numpy as np

from spin_dynamics.esr import (
    deer_dipolar_spectrum,
    extract_distance_distribution,
    gaussian_distance_distribution,
    simulate_deer,
)

times = np.linspace(0.0, 2.5e-6, 100)
distances = np.linspace(1.5, 4.0, 60)
truth = gaussian_distance_distribution(distances, 2.5, 0.2)

form_factor = simulate_deer(times, distances, truth, lambda_depth=0.35)
freqs, pake = deer_dipolar_spectrum(times, form_factor)
recovered = extract_distance_distribution(
    times, form_factor, distances, lambda_depth=0.35, snr=300.0
)
```

Distance recovery solves the regularized non-negative least-squares problem $K\,p \approx F$ using the SNR-informed Tikhonov selector from `spin_dynamics.analysis.regularization` with the DEER powder kernel as the design matrix (SciPy required for the non-negative solve).

The analytic kernel is validated against an independent two-electron density-matrix simulation of the full four-pulse sequence (`deer_pair_trace_quantum`): with ideal spin-selective observer and pump pulses it reproduces $F(t)$ to floating-point precision for every orientation and pump flip angle, and the observer echo refocuses the observer offset.

6.7 ESEEM, HYSCORE, and ENDOR

While DEER measures electron-electron distances, ESEEM (electron spin echo envelope modulation), HYSCORE (hyperfine sublevel correlation), and ENDOR (electron-nuclear double resonance) measure the weak hyperfine coupling between an electron and a nearby nucleus. The modules `spin_dynamics.esr.eseem`, `spin_dynamics.esr.hyscore`, and `spin_dynamics.esr.endor` model an $S = 1/2$ electron coupled to a nucleus ($I = 1/2$, 1, or $3/2$) in the secular approximation,

$$H = -\omega_I I_z + S_z(A\,I_z + B\,I_x) + H_Q,$$

parameterized by `HyperfineCoupling(larmor_hz, secular_hz, pseudosecular_hz, nuclear_spin, quadrupole_hz, eta)`. For $I \geq 1$ the nuclear quadrupole term H_Q (quadrupole frequency ν_Q and asymmetry η; principal axis taken collinear with the static field) reuses the NQR quadrupole Hamiltonian. ESEEM requires the nuclear quantization axis to differ between the two electron manifolds, which the pseudosecular B and the quadrupole both provide. For $I = 1/2$ the two nuclear (ENDOR) frequencies and modulation depth have the closed forms

$$\nu_{\alpha,\beta} = \sqrt{(\omega_I \pm A/2)^2 + (B/2)^2}, \qquad k = \left(\frac{\omega_I B}{\nu_\alpha \nu_\beta}\right)^2;$$

for any spin, `manifold_frequencies` returns the nuclear transition frequencies in each electron manifold by diagonalization. The classic ^{14}N ($I = 1$) *exact-cancellation* regime, where the hyperfine field cancels the nuclear Zeeman in one manifold and leaves the three pure-quadrupole (NQR) lines $\nu_\pm = \nu_Q(1 \pm \eta/3)$, $\nu_0 = \frac{2}{3}\nu_Q\eta$, is reproduced by this model.

ESEEM. `two_pulse_eseem` and `three_pulse_eseem` return the analytic two- and three-pulse envelopes, and `eseem_spectrum` their frequency spectrum (peaks at ν_α, ν_β, and – for two-pulse – their sums and differences). Each has a density-matrix counterpart (`*_quantum`) that propagates the spin Hamiltonian through ideal pulses and reproduces the analytic trace to floating-point precision.

```
import numpy as np

from spin_dynamics.esr import HyperfineCoupling, three_pulse_eseem, eseem_spectrum

coupling = HyperfineCoupling(larmor_hz=14.5e6, secular_hz=3e6, pseudosecular_hz=2e6)
times = np.linspace(0.0, 8e-6, 400)
trace = three_pulse_eseem(times, coupling, tau_seconds=200e-9)
freqs, spectrum = eseem_spectrum(times, trace)
```

Coherence selection and phase cycling. The stimulated-echo and HYSCORE pathways are isolated in simulation by electron coherence-order filtering. This is the idealized limit of an experimental phase cycle: shifting a pulse phase by φ multiplies a pathway of coherence-order change Δp by $e^{-i\Delta p\varphi}$, so cycling φ and weighting the record by $e^{+i\Delta p_{\mathrm{desired}}\varphi}$ selects that pathway. `three_pulse_eseem_phase_cycled` performs the explicit cycle (four steps per pulse suffice because the electron coherence order spans only $\{-1, 0, +1\}$) and reproduces the coherence-filtered result to numerical precision.

HYSCORE. `hyscore_signal` simulates the 2D four-pulse experiment and `hyscore_spectrum` returns its 2D spectrum, whose cross-peaks appear at (ν_α, ν_β) and (ν_β, ν_α) (`cross_peak_positions`). Choose the dwell time so the nuclear frequencies stay below the Nyquist limit, or the cross-peaks alias.

ENDOR. `endor_frequencies` returns the line positions, and `davies_endor_spectrum` / `mims_endor_spectrum` return the analytic Davies and Mims responses. Mims ENDOR carries the τ-dependent blind-spot factor $\sin^2(\pi\nu\tau)$, which suppresses a line whenever $\nu\tau$ is an integer.

The model covers a single nucleus of spin $I = 1/2$, 1, or $3/2$ in the secular regime, with the quadrupole principal axis collinear with the static field. Anisotropic hyperfine A tensors with powder averaging, an arbitrary quadrupole-tensor orientation, and multiple coupled nuclei are the natural next extensions.

7 Pulsed NQR Models

Nuclear quadrupole resonance is driven by the interaction between a nuclear quadrupole moment and the local electric-field-gradient (EFG) tensor. The `spin_dynamics.nqr` namespace adds a scoped quadrupolar extension for solid-state pulsed NQR experiments. It is intentionally separate from the spin-1/2 Bloch and scalar-coupling layers.

Use this layer for: reduced two-level (selective) pulsed NQR of spin-1 sites; full $(2I+1)$-level density-matrix FID and echo dynamics required for spin-3/2; automatic reduced-vs-full model selection; spin-1 and spin-3/2 transition metadata; single-crystal and powder averages over local EFG orientations; SLSE echo trains; two-frequency population transfer; static EFG inhomogeneity; finite-window SLSE spectra; weak-B0 line splitting; optional Liouville-space phenomenological or Redfield/dipolar relaxation; and instrument-level polarization-enhanced NQR transport with CIF-based ^{1}H-quadrupolar dipolar-coupling estimates.

Current limits: dense single-site matrices only; two pulse models (the reduced spin-1 selective pulse and the full $(2I+1)$-level density-matrix pulse). Any integer or half-integer spin above 1/2 is supported: the level and transition structure is exact for all of them (spin-5/2, 7/2, 9/2 – ^{27}Al, ^{51}V, ^{93}Nb, ...), and FID/echo/SLSE run on the full model. The full-model *pulse* uses a single-carrier rotating-wave approximation, so it is faithful on the *fundamental* (lowest) NQR line; driving a higher/satellite line of a spin $> 3/2$ nucleus emits a warning (correct satellite pulsing needs exact time-domain propagation, a planned follow-up). The *reduced* SLSE and population-transfer workflows remain spin-1; a two-frequency 2D population-transfer workflow on the full model is still future work. Microscopic relaxation is a stochastic Redfield/dipolar treatment, not a coherent multi-site quadrupolar cluster; no multi-site coherent coupling between quadrupolar nuclei.

7.1 Quadrupolar Site Model

A quadrupolar site is described in the EFG principal-axis system. The zero-field Hamiltonian used by the Python helper is

$$H_Q = 2\pi\nu_{\text{scale}}\left[3I_z^2 - I(I+1)\mathbf{1} + \eta\left(I_x^2 - I_y^2\right)\right],$$

where I is the spin quantum number, ν_Q is the quadrupole-frequency parameter in hertz, and $0 \le \eta \le 1$ is the EFG asymmetry parameter. Hamiltonians are represented in radians per second. For spin-1, the package uses $\nu_{\text{scale}} = \nu_Q/3$, matching the nitrogen-14 convention where the zero-field lines are $\nu_x = \nu_Q(1+\eta/3)$, $\nu_y = \nu_Q(1-\eta/3)$, and $\nu_z = 2\eta\nu_Q/3$. For spin-3/2, $\nu_{\text{scale}} = \nu_Q/6$, so the zero-field chlorine-style NQR line is

$$\nu_{\text{NQR}} = \nu_Q\sqrt{1+\eta^2/3}.$$

Zero-frequency Kramers-doublet transitions are omitted from the transition inventory.

Weak Zeeman perturbations can be added as

$$H_Z = -2\pi\gamma\,\mathbf{B}_0 \cdot \mathbf{I},$$

where $\mathbf{B}_0$ is expressed in the EFG principal-axis system for the current crystallite orientation. The default is $B_0 = 0$, which is the usual NQR case.

```
from spin_dynamics.nqr import QuadrupolarSite, diagonalize_site

site = QuadrupolarSite(
    spin=1,
    isotope="14N",
    quadrupole_frequency_hz=900e3,
    eta=0.3,
)

eigensystem = diagonalize_site(site)
for transition in eigensystem.transitions:
    print(transition.label, transition.frequency_hz)
```

For spin-1 at zero field, transitions are labeled by their dominant RF polarization in the EFG principal-axis system: `x`, `y`, and `z`. For the example above the line positions are approximately 990, 810, and 180 kHz.

Spin-3/2 transition metadata can be inspected with the same diagonalization helper:

```
chlorine = QuadrupolarSite(
    spin=1.5,
    isotope="35Cl",
    quadrupole_frequency_hz=30e6,
    eta=0.1,
    gamma_hz_per_t=4.171e6,
)

for transition in diagonalize_site(chlorine).transitions:
    print(transition.frequency_hz)
```

7.2 Two Modeling Regimes

Before propagating a pulse, the package chooses between two models. The choice is governed by how many states the RF pulse can connect, not by the number of spectral lines. The detailed reasoning is given in the technical note `References/Pulsed_NQR_Spin_Dynamics_Narrative_Rewrite.pdf`; the tracked `.tex` source is in the same directory.

- **Reduced two-level model** (`SelectivePulse`, `simulate_slse`): one pulse addresses one transition as an embedded fictitious spin-1/2 rotation. This is honest only when the selected transition is isolated. For a target transition t, with $\Delta_{\mathrm{iso}} = \min_{j \neq t} |\omega_t - \omega_j|$ the spacing to the nearest RF-active competitor, the validity condition is

  $$\Delta_{\mathrm{iso}} \gg \max(\Omega_1,\ \Delta\omega_{\mathrm{pulse}},\ \Gamma)\,,$$

  where Ω_1 is the effective nutation rate, $\Delta\omega_{\mathrm{pulse}} \sim 1/t_p$ is the pulse bandwidth, and Γ is the linewidth. A nonzero η is necessary but *not* sufficient: at $\eta = 0$ the spin-1 x and y lines coincide and the reduction fails. The reduced path is restricted to spin-1 for this reason.

- **Full density-matrix model** (`spin_dynamics.nqr.full_dynamics`): the complete $(2I+1)$-level density matrix is propagated. This is the correct general model and is required for spin-3/2, whose single zero-field line connects two Kramers doublets—four states hidden behind one frequency.

7.3 Selective Pulse Approximation

Most practical spin-1 pulsed NQR RF pulses are narrowband relative to the separation between quadrupolar transitions. The reduced model uses this selective limit: one pulse addresses one transition and acts as an embedded two-level rotation inside the full $(2I+1)$-level Hilbert space.

For a transition $|a\rangle \leftrightarrow |b\rangle$, the transition dipole vector is

$$\mathbf{d}_{ab} = (\langle a|I_x|b\rangle, \langle a|I_y|b\rangle, \langle a|I_z|b\rangle)\,.$$

For a local RF direction $\hat{\mathbf{b}}_1$, the complex drive projection is proportional to $\hat{\mathbf{b}}_1 \cdot \mathbf{d}_{ab}$. Preserving this signed complex projection is essential for powder averages because transmit and receive phases must pair consistently across crystallites.

```
from spin_dynamics.nqr import SelectivePulse, apply_selective_pulse

pulse = SelectivePulse(
    "x",
    duration_seconds=50e-6,
    nutation_hz=10e3,
)
```

The pulse helper also supports an optional RF frequency. If omitted, the RF is placed on the selected transition. If supplied, the difference between the RF frequency and transition frequency is treated as a two-level detuning.

The selective-pulse propagation routines support spin-1 sites only. Spin-3/2 nuclei such as $^{35}\mathrm{Cl}$ and $^{37}\mathrm{Cl}$ have degenerate doublets at zero field, so their pulsed response is handled by the full density-matrix model below rather than by an embedded two-level transition; calling the reduced selective-pulse routines on a non-spin-1 site raises a clear error.

7.4 Full Density-Matrix Model

The `spin_dynamics.nqr.full_dynamics` module propagates the complete $(2I+1)$-level density matrix in a rotating frame at the pulse carrier. It is the required model for spin-3/2 and runs for any spin from 1 to 9/2. The static Hamiltonian $H_Q + H_Z$ is diagonalized for the actual EFG and field orientation, the RF and detection operators $\hat{\mathbf{e}} \cdot \mathbf{I}$ are transformed into that eigenbasis, and the density matrix is propagated through each constant-Hamiltonian interval by $U_k = \exp(-iH_k\,\Delta t_k)$, $\rho_{k+1} = U_k \rho_k U_k^\dagger$. The free-evolution acquisition windows diagonalize the constant free generator once and reconstruct every sample, so a spin-9/2 FID with thousands of points stays a few milliseconds despite the 100×100 Liouville space.

A half-integer spin I has $I - 1/2$ zero-field lines; for an axial EFG they fall at $\nu_Q, 2\nu_Q, 3\nu_Q, \ldots$ (the $\frac{1}{2} \leftrightarrow \frac{3}{2}$, $\frac{3}{2} \leftrightarrow \frac{5}{2}$, … transitions), where ν_Q is `quadrupole_frequency_hz`, the fundamental line; a non-zero η shifts them and switches on a weak combination line. `quadrupolar_site` builds a named nucleus ($^{27}\mathrm{Al}$, $^{51}\mathrm{V}$, …) from its C_Q or ν_Q, filling the spin and gyromagnetic ratio from the isotope registry. `examples/plot_nqr_higher_spin_transitions.py` shows the level ladder, the transition ratios, and a pulsed FID/echo on the $^{27}\mathrm{Al}$ fundamental line.

Here `nutation_hz` is the bare field nutation $\gamma B_1/(2\pi)$, so the realized Rabi rate on a transition $a-b$ is $2\,\nu_1\,|\langle a|\hat{\mathbf{e}}_1 \cdot \mathbf{I}|b\rangle|$; the same pulse is a different flip angle on different transitions, which is the physical effect the full model captures. A single-carrier rotating-wave approximation is used: it is faithful on the

fundamental (lowest) line, which is the default carrier and the strongest line. Driving a higher/satellite line of a spin $> 3/2$ nucleus is flagged with a warning (the winding-number frame would otherwise select the wrong transition); correct satellite pulsing needs exact time-domain propagation, a planned follow-up.

```
import numpy as np
from spin_dynamics.nqr import QuadrupolarSite, simulate_full_fid

site = QuadrupolarSite(spin=1.5, isotope="35Cl",
                       quadrupole_frequency_hz=30e6, eta=0.1)

fid = simulate_full_fid(
    site,
    nutation_hz=20e3,            # bare field nutation gamma*B1/(2*pi)
    pulse_duration_seconds=10e-6,
    times_seconds=np.linspace(0.0, 200e-6, 1024),
)
print(fid.signal)               # complex baseband signal
```

A two-pulse (Hahn-style) echo is available through `simulate_full_echo`, and the spin-lock spin-echo (SLSE) detection train—the chlorine-style spin-3/2 measurement—through `simulate_full_slse`:

```
from spin_dynamics.nqr import QuadrupolarSite, simulate_full_slse

site = QuadrupolarSite(spin=1.5, isotope="35Cl",
                       quadrupole_frequency_hz=1.0e6, eta=0.0)

slse = simulate_full_slse(
    site, nutation_hz=10e3,
    excitation_duration_seconds=25e-6, refocus_duration_seconds=50e-6,
    echo_spacing_seconds=400e-6, num_echoes=12,
    orientations="powder",   # powder average; b0_tesla > 0 adds weak Zeeman
    t2e_seconds=3e-3,
)
```

`simulate_full_slse` returns a `FullNQRSLSEResult` with the orientation-weighted echo train, the per-orientation trains, and the weights. Pass `b0_tesla` > 0 for a weak Zeeman perturbation (the field direction follows each orientation sample; use `b0_powder_average_grid` for a Zeeman powder), and an optional `relaxation=NQRRelaxationModel(...)` adds Liouville-space T_1/T_2 decay. The lower-level primitives `pulse_hamiltonian`, `static_hamiltonian_rotating`, and `detection_operator` are exposed for custom sequences. The full model has been validated against an exact lab-frame propagator (the rotating-wave error scales as the Ω_1^2 Bloch-Siegert shift), and its spin-1 powder nutation curve reproduces the classic $119°$ maximum while the spin-3/2 curve peaks at a slightly smaller flip angle. The `plot_nqr_spin32_slse.py` example runs the $^{35}\mathrm{Cl}$ powder SLSE train and shows how a weak Zeeman field reshapes the decay.

7.5 Model Selection

`select_nqr_model` reads the reduced-vs-full choice from the static Hamiltonian and the RF matrix elements for the coil polarization, not from the spin or the line count. It builds the pulse-addressed connected set (states the pulse can drive from the target pair, plus RF-driven near-degenerate partners) and the isolation ratio $\Delta_{\mathrm{iso}}/\max(\Omega_1, 1/t_p, \Gamma)$.

```
from spin_dynamics.nqr import QuadrupolarSite, select_nqr_model
```

```
site = QuadrupolarSite(spin=1.5, quadrupole_frequency_hz=30e6, eta=0.1)
choice = select_nqr_model(
    site, "x",
    nutation_hz=5e3,
    pulse_duration_seconds=50e-6,
    b1_direction_pas=(1, 1, 1),
)
print(choice.recommended_model)   # "full" -- spin-3/2 Kramers doublets
print(choice.describe())          # diagnostic report
```

The reduced model is recommended only when the addressed set is a single, non-degenerate, RF-active pair *and* the isolation ratio clears `isolation_threshold` (default 5). The returned `NQRModelSelection` reports the target states, nearest competing RF-active transition, isolation distance and ratio, pulse bandwidth, the addressed-state set, a degeneracy flag, and human-readable reasons. It handles the regime-defining cases: spin-1 $\eta = 0$ and unresolved small-η splittings go full via the isolation ratio; spin-3/2 Kramers doublets go full because the addressed set spans four states; a strongly Zeeman-resolved spin-3/2 line can return reduced as a verified special case; and a polarization that makes neighboring lines RF-dark keeps a crowded spectrum reducible. The check is currently advisory—the reduced workflows do not yet invoke it automatically.

7.6 Orientations and Powder Averages

NQR is a solid-state experiment, so the signal depends on the orientation of the RF field relative to the local EFG principal axes. A single crystal is one fixed orientation. A powder is represented by a normalized spherical quadrature grid:

```
from spin_dynamics.nqr import powder_average_grid, single_crystal_orientation

single = single_crystal_orientation(alpha=0.0, beta=1.57079632679)
powder = powder_average_grid(n_theta=16, n_phi=32)
```

For weak-B0 calculations, each orientation may also specify the direction of B_0 in the EFG frame. If no separate B_0 direction is supplied, the current helper uses the local RF direction as the default field direction for that sample.

Weak-field spectra use a correlated powder grid for the static field and RF field directions. The helper `b0_b1_powder_average_grid` samples the orientation of B_0 in the EFG frame and the rotation angle of B_1 around B_0, preserving a chosen lab-frame angle between the two fields. For a conventional transverse coil one uses $B_1 \perp B_0$.

7.7 Weak Static B0 Spectra

Weak static magnetic fields are modeled by exact diagonalization of

$$H = H_Q + H_Z, \qquad H_Z = -2\pi\gamma\,\mathbf{B}_0 \cdot \mathbf{I}.$$

The helper is intended for the NQR regime

$$\epsilon_Z = \frac{|\gamma B_0|}{\nu_{\mathrm{ref}}} \ll 1,$$

and reports the largest perturbation ratio in the result. It can warn or raise if the requested field is no longer weak compared with the selected NQR line.

```
from spin_dynamics.nqr import simulate_weak_b0_spectrum

weak = simulate_weak_b0_spectrum(
    chlorine,
    b0_tesla=1e-3,
    orientations="powder",
    broadening_hz=200.0,
    weak_ratio_threshold=0.05,
)

print(weak.max_perturbation_ratio)
```

For each crystallite, the code diagonalizes $H_Q + H_Z$, keeps transitions near the selected zero-field NQR line, and weights them by the RF projection

$$I_{ab} \propto w_k \left| \hat{\mathbf{b}}_{1,k} \cdot \mathbf{d}_{ab,k} \right|^2 ,$$

where w_k is the orientation weight. This RF projection is important for spin-3/2 in weak fields: the eigenvalue splitting can contain several nearby transitions, but transitions dark to the coil should not contribute to the plotted spectrum. In the axial limit $\eta = 0$, $B_0 \parallel z_{\mathrm{PAS}}$, and $B_1 \perp B_0$, a spin-3/2 site gives the expected pair $\nu_Q \pm \gamma B_0$.

7.8 Polarization-Enhanced NQR Transport

Polarization-enhanced NQR is modeled at the instrument level following the Glickstein–Mandal automated pre-polarization workflow. A solid sample first sits in a permanent magnet long enough to build proton polarization. It is then translated through the falling fringe field into the NQR probe. Whenever the proton Larmor frequency

$$\nu_H(x) = \gamma_H B_0(x)$$

crosses a nitrogen NQR line, dipolar coupling can transfer polarization from the proton reservoir into the quadrupolar population difference. For a sample moving with speed v through a local gradient dB_0/dx, the available crossing time is approximately

$$t_{\mathrm{cp}} \approx \frac{\Delta\nu_H}{\gamma_H \, v \, |dB_0/dx|}.$$

The transfer is adiabatic when this time is long compared with the inverse ^{1}H-^{14}N coupling rate ν_{NH}^{-1}, or equivalently when

$$\frac{\gamma_H v |dB_0/dx|}{\Delta\nu_H \, \nu_{NH}} \ll 1.$$

The helper `simulate_adiabatic_polarization_transfer` evaluates this criterion along a user-specified transport path, using either a finite Halbach magnet model or any callable B0 sampler. It reports the level-crossing positions, local gradients, adiabatic ratios, transfer efficiencies, ideal spin-1 enhancement factors, and practical enhancement after finite proton build-up, transfer efficiency, finite sample size, and T_1 loss during transport.

```
from spin_dynamics.nqr import (
    CylindricalSampleGeometry, HalbachPrepolarizationMagnet,
    LinearTransportMotion, PolarizationEnhancedNQRSample,
    simulate_adiabatic_polarization_transfer,
```

```
)

sample = PolarizationEnhancedNQRSample(
    name="melamine-like",
    line_labels=("nu+", "nu-", "nu0"),
    line_frequencies_hz=(2.766e6, 2.034e6, 0.732e6),
    protons_per_molecule=6,
    nitrogens_per_molecule=6,
    proton_t1_seconds=48.6,
    proton_linewidth_hz=80e3,
    proton_nitrogen_coupling_hz=1.3e3,
)
magnet = HalbachPrepolarizationMagnet(
    rod_shape="square", rod_width=25.4e-3, length=101.6e-3,
)
geometry = CylindricalSampleGeometry(length=20e-3, diameter=8e-3)
motion = LinearTransportMotion(0.0, 0.10, velocity=0.1667, axis="z")

pe = simulate_adiabatic_polarization_transfer(
    magnet, sample, geometry, motion,
    prepolarization_time_seconds=100.0,
)
print(pe.practical_enhancement)
```

The model intentionally does not replace a microscopic coupled-spin density-matrix calculation of each avoided crossing. Instead, it provides a transparent engineering model for choosing magnet geometry, transport speed, sample size, pre-polarization time, and the uncertain coupling scale ν_{NH}.

7.8.1 Estimating ^{1}H-N Coupling from CIF Structures

The coupling scale can be estimated from a crystal structure when nearby proton positions are known. The helper `estimate_proton_dipolar_couplings_from_cif` reads a small-molecule CIF, applies the listed symmetry operations and periodic images, finds protons within a search radius of the quadrupolar atom label, and evaluates the point-dipole prefactor

$$d_{HQ}(r) = \frac{\mu_0}{4\pi}\frac{h\,|\gamma_H\gamma_Q|}{r^3}.$$

It returns individual proton distances and couplings, plus an RMS effective coupling useful as ν_{NH} in the adiabatic-transfer criterion. If a field direction is supplied, it also reports the secular angular factor $(1-3\cos^2\theta)d_{HQ}$.

```
from spin_dynamics.nqr import estimate_proton_dipolar_couplings_from_cif

estimate = estimate_proton_dipolar_couplings_from_cif(
    "../QuadrupolarDFT/structures/Melamine/237082.cif",
    "N101",
    proton_radius_angstrom=3.0,
)
print(estimate.effective_rms_hz)
for item in estimate.proton_couplings:
    print(item.proton_label, item.distance_angstrom, item.coupling_hz)
```

For the bundled melamine CIF, the ring nitrogen `N101` sees four nearby protons within $3\,\text{Å}$ and gives an RMS coupling of about 1.3 kHz. Protonated amine nitrogens have direct N–H distances and therefore much larger couplings. The example `plot_nqr_polarization_enhancement.py` uses the melamine CIF estimate by default, but accepts `-nh-coupling-hz` to override it.

7.8.2 Using the Local NQR Data and Structure Workspaces

The example `plot_nqr_database_prepolarization.py` demonstrates how the separate workspaces can be used together. It reads measured transition frequencies, site metadata, T_1 values, and provenance from the SQLite export in `../NQRDatabase/data/exports/nqr.sqlite`; it then estimates the effective ^{1}H-^{14}N coupling from a CIF stored under `../QuadrupolarDFT/structures`; finally, it passes the resulting sample parameters to `simulate_adiabatic_polarization_transfer`. The default case uses the glycine database record and the bundled glycine CIF:

```
python examples/plot_nqr_database_prepolarization.py \
    --compound Glycine \
    --coupling-target N1 \
    --output results/nqr_database_prepolarization.png
```

This is a useful pattern for screening real compounds. The database supplies experimentally reported NQR lines and relaxation metadata; the structure workspace supplies atom positions for nearby-proton coupling estimates and can also hold first-principles EFG or resonant-frequency calculations when measured lines are absent or need comparison. The simulation layer then converts those inputs into transition-by-transition ideal and practical prepolarization signal gains for a chosen magnet, sample geometry, and transport speed.

7.9 SLSE Detection

The spin-lock spin-echo (SLSE) sequence is the classic pulsed NQR detection sequence. The helper models it as repeated selective pulses on a spin-1 detection transition and reports echo amplitudes at the requested echo spacing. Echo decay can be represented by a scalar T_{2e} envelope or by a phenomenological Liouville-space relaxation model with T_1 and T_2. For quantitative powder echo trains, especially when comparing coherent powder signal amplitudes, prefer the full density-matrix `simulate_full_slse` path described above.

Microscopic Redfield/dipolar relaxation can be supplied through the same `relaxation=` slot on the full-density-matrix path. The NaNO2 example reads the ^{14}N EFG parameters from the QuadrupolarDFT summary, builds nearby ^{23}Na and ^{14}N stochastic bath sources from the CIF, and propagates a coherent SLSE train. The powder row uses equal SLSE pulse lengths with the spin-1 powder optimum near $119°$; the refocusing pulse phase is the full-model default $\pi/2$ shift. It plots the coherent measured powder voltage, not magnitude- or RMS-averaged local crystallite signals:

```
python examples/plot_redfield_nano2_slse.py --output nano2_redfield_slse.png
```

```
from spin_dynamics.nqr import simulate_slse, slse_sequence

sequence = slse_sequence(
    "x",
    pulse_duration_seconds=25e-6,
    nutation_hz=10e3,
    echo_spacing_seconds=1e-3,
    num_echoes=16,
```

```
)

result = simulate_slse(
    site,
    sequence,
    orientations="powder",
    t2e_seconds=20e-3,
)
```

The returned `SLSEResult` includes echo times, powder-averaged echo amplitudes, per-orientation echo amplitudes, orientation weights, transition metadata, and the eigensystem used for the first orientation. When a relaxation model is supplied, the result also reports local cycle eigenvalues and cycle-derived effective decay times.

```
from spin_dynamics.nqr import NQRRelaxationModel

relaxed = simulate_slse(
    site,
    sequence,
    orientations="powder",
    relaxation=NQRRelaxationModel(t1_seconds=1.0, t2_seconds=20e-3),
)
```

Offset and pulse-period sweeps are available as compact diagnostics for the SLSE modulation:

```
from spin_dynamics.nqr import simulate_slse_offset_sweep

sweep = simulate_slse_offset_sweep(
    site,
    "x",
    offsets_hz=[-2e3, 0.0, 2e3],
    pulse_duration_seconds=25e-6,
    nutation_hz=10e3,
    echo_spacing_seconds=500e-6,
    relaxation=NQRRelaxationModel(t2_seconds=20e-3),
)
```

7.9.1 Circular and Multi-Axis Excitation

The examples above excite the powder with a single linearly-polarized coil. The full density-matrix engine keeps a general RF direction and phase per coil and is linear in the field, so several coils superpose as co-rotating RWA couplings. The helper `multi_axis_pulse_hamiltonian` sums a list of `CoilDrive` components (each a bare nutation, phase, and PAS direction); with one component it reproduces `pulse_hamiltonian` exactly. *Circular* polarization is the special case of two orthogonal coils driven $\pi/2$ out of phase (`circular_pulse_hamilto` producing a rotating RF field, with a matched quadrature receiver `quadrature_detection_operator`.

Because a multi-coil drive couples to two or three orthogonal lab axes at once, its powder response depends on the *full* crystallite orientation, not just one coil axis. `powder_frame_grid` provides the required SO(3) grid of orthonormal lab-to-PAS frames (Gauss–Legendre $\cos\beta \times$ uniform $\alpha \times$ uniform in-plane χ).

```
python examples/plot_nqr_circular_polarization_nutation.py --output circular.png
```

In zero-field NQR the transition has no intrinsic precession handedness, so a linear coil couples equally to both rotation senses whereas a circular coil couples to one. Across a powder this raises the *refocused*

(SLSE) signal: neither circular excitation alone (single-coil detection) nor quadrature detection alone helps, but together, at matched per-coil B_1, they lift the powder echo train by $\sim 1.6\times$ (each scheme calibrated to its own optimal flip, so the trivial $\sim 2\times$ field efficiency of a rotating field is normalized out). Detecting with the *wrong* rotation sense nulls the powder signal — a direct check that the handedness is physical. Circular uses two coils, drawing $\sim 2\times$ peak RF power and adding a $\sqrt{2}$ two-channel receiver-noise penalty, so the $\sim 1.6\times$ signal gain corresponds to a $\sim 1.1\times$ net SNR gain — the slight increase reported in the powder NQR literature.

7.10 SORC Detection

The strong off-resonance comb (SORC) sequence is represented as $(\tau - \phi - \tau)^N$, with the observation sampled between repeated off-resonant pulses. The default `simulate_sorc` path follows the finite-N pathway recurrence used by Konnai, Odano, and Asaji (2008). As the number of pulses grows, this recurrence converges to the published steady-state powder expression and retains the expected $\Delta\omega\tau$ comb periodicity. Use `model="coherent"` only when the desired calculation is the closed unitary transient train rather than the dephased SORC pathway model.

```
from spin_dynamics.nqr import QuadrupolarSite, simulate_sorc, sorc_sequence

site = QuadrupolarSite(
    spin=1,
    isotope="14N",
    quadrupole_frequency_hz=4.2e6,
    eta=0.3,
)
sequence = sorc_sequence(
    "x",
    pulse_duration_seconds=20e-6,
    nutation_hz=16.5e3,
    half_spacing_seconds=0.8e-3,
    num_pulses=96,
    rf_frequency_hz=4.60425e6 - 2.05e3,
)

result = simulate_sorc(site, sequence, orientations="powder")
print(result.signal_amplitudes[-1])
```

For direct paper-style comparisons, `sorc_powder_pathway_signal` evaluates the finite-N recurrence in Konnai's one-dimensional powder angle, which avoids aliasing from coarse spherical orientation grids in long pulse-width sweeps. `sorc_powder_theory_signal` evaluates the corresponding infinite-N steady-state response, while `fid_powder_theory_signal` gives the spin-1 powder FID pulse-width response. The example `examples/plot_nqr_sorc_konnai2008.py` overlays finite-N SORC markers with the infinite-N theory curves for the offset, spacing, and pulse-width sweeps in the paper.

7.10.1 Sensitivity per Unit Time: SLSE vs. Steady-State SORC

The two schemes are often quoted with a $\sqrt{T_1/T_{2e}}$ advantage for SORC, but that law is an artifact of a T_1-independent steady-state model. Done honestly – both sequences fully optimized in one common full density-matrix framework – the schemes are *comparable*. The figure of merit is SNR per unit $\sqrt{\text{time}}$, because averaging N acquisitions improves SNR as $\sqrt{N}$ with $N = T/t_{\text{scan}}$.

`examples/plot_nqr_slse_sorc_sensitivity.py` grounds the comparison in a specific material, $^{14}\mathrm{N}$ in $NaNO_2$, whose three relaxation times are not inserted by hand: T_1 and the spin-locked T_{2e} emerge from a microscopic dipolar Redfield model built from the actual neighbour spins (`RedfieldDipolarRelaxationModel.from_dipola`) and the fast reversible T_2^* is the Van Vleck rigid-lattice second moment of the same neighbour list ($\sim 1\,\mathrm{ms}$ here – weak, because $^{14}\mathrm{N}$ has a small gyromagnetic ratio, so it is set mostly by the $^{23}\mathrm{Na}$ neighbours). The correlation time τ_c is the temperature knob; sweeping it walks the sample over $T_1/T_{2e} \sim 1$ to 100 while T_2^* stays fixed. Both sequences are propagated from the same equilibrium ρ_{eq} and read with the same detection operator (one common normalization, no per-scheme scaling); the SORC steady state is the affine fixed point of its cycle with an equilibrium-target source $(I-M)\rho_{\mathrm{eq}}$, so it correctly carries the T_1 that replenishes the polarization. Sensitivity is the matched-filter energy $\sqrt{\int |s(t)|^2\,dt}$ of each acquisition window, and flip angle, RF offset, and pulse spacing are optimized for each scheme at each τ_c.

Result 1: comparable sensitivity, with a mild crossover. On a single crystal the two schemes stay within $\sim 15\%$ of each other across the entire T_1/T_{2e} range. SORC is marginally ahead when the sample is warm ($T_1/T_{2e} \lesssim 5$) and SLSE marginally ahead when it is cold, the ratio crossing unity near $T_1/T_{2e} \approx 5$ (Table 7.1). Neither grows a running advantage. The often-quoted $\sqrt{T_1/T_{2e}}$ law would predict SORC winning by more than an order of magnitude at the cold end; it does not, because that law comes from a T_1-*independent* steady-state model (the reduced Konnai pathway). Once the SORC steady state is forced to carry T_1 through its equilibrium-target source, its per-time signal falls off with relaxation at essentially the same rate as SLSE's. The mirror-image error – "SLSE wins increasingly" – comes from under-optimizing the SORC flip angle: its true optimum is a *small* tip (~ 10–$20°$), a balanced-SSFP-like high-retention comb, and coarse large-flip grids miss it.

T_1/T_{2e}	T_1 (ms)	T_{2e} (ms)	SNR/$\sqrt{\text{time}}$ (a.u.)		$\frac{\text{SORC}}{\text{SLSE}}$
			SLSE	SORC	
0.8	134	165	0.230	0.248	1.08
1.2	156	134	0.197	0.221	1.12
2.6	255	100	0.136	0.152	1.12
8.0	482	60	0.076	0.070	0.91
29	949	33	0.040	0.036	0.89
112	1891	17	0.020	0.018	0.88

Table 7.1: Optimized single-crystal sensitivity of SLSE and steady-state SORC for $^{14}\mathrm{N}$ in $NaNO_2$, swept over the dipolar correlation time ($T_2^* \approx 1.07\,\mathrm{ms}$ throughout). Both schemes are fully optimized over flip, offset, and spacing in one common normalization.

Result 2: SORC is more robust to powder averaging. The one robust *sensitivity* asymmetry appears when the single crystal is replaced by a powder. At $T_1/T_{2e} \approx 29$ the SLSE sensitivity drops by $\sim 30\%$ from crystal to powder while SORC barely moves, so SORC leads by $\sim 1.3\times$ on the powder (and $\sim 1.1\times$ with fixed operating points; the effect is grid-converged from a 4×6 to a 9×18 orientation mesh). The mechanism is the per-orientation spread of the signal: the SLSE $90/180$ echo has a strongly orientation-dependent refocusing efficiency – across the powder its per-crystallite echo energy varies by more than an order of magnitude, with some crystallites sitting near a nutation null – whereas SORC's small-tip steady state is nearly uniform across orientations (its per-crystallite energy varies by only a factor of a few). A powder therefore averages away much of the SLSE signal but little of the SORC signal. This is a genuine, potentially useful result for powder samples, not a grid artifact.

Result 3: SORC draws less average RF power. Because both schemes drive the same coil at the same nutation frequency, the peak (in-pulse) RF power is identical and the *average* RF power is peak power times duty cycle; the average-power ratio is thus a pure duty-cycle ratio. At the optima SORC draws only ~ 0.2–$0.4\times$ the average power of SLSE across the whole sweep. This is the opposite of the intuition that a *continuously* pulsing comb must dissipate more: SORC's small $\sim 15^\circ$ tip makes each pulse very short (here $\sim 1.7\,\mu\mathrm{s}$ every $260\,\mu\mathrm{s}$, a 0.6% duty), whereas the SLSE matched filter prefers *wide* pulses ($\sim 70^\circ$) at the *shortest* allowed echo spacing, giving a much higher in-train duty ($\sim 4\%$). Even after amortizing SLSE's duty over its T_1-recovery idle time, SORC stays the lower-power method. One caveat keeps this an operating-point statement rather than a fundamental bound: SLSE's sensitivity is nearly flat in echo spacing near the optimum, so it could be run at a longer spacing – and hence lower power – at little cost in SNR; the example simply reports the matched-filter optimum, which happens to be power-hungry.

Putting it together. For this material the two classic pulsed-NQR detection families are close enough in raw sensitivity that the decision is driven by the *secondary* properties the example quantifies: SORC is somewhat more forgiving on powders and runs at lower average RF power (attractive for portable, battery, or heating-sensitive work), but only as a continuously running steady state; SLSE is a simple pulse-and-recover experiment that reaches the same crystal sensitivity and is marginally better in the cold, long-T_1 regime. The broader lesson is methodological: a fair sensitivity comparison of two pulse sequences requires a common signal normalization, grounded (not hand-inserted) relaxation times, and a full optimization of *both* sequences – dropping any one of these reproduces one of the folklore "advantage laws."

7.11 EFG Inhomogeneity and SLSE Spectra

Static EFG disorder is modeled as an isochromat ensemble of independent quadrupolar sites. Each isochromat has its own ν_Q, η, transition frequency, and normalized weight. This covers impurity-induced broadening, static temperature gradients, and other inhomogeneous mechanisms analogous to NMR isochromat broadening.

```
from spin_dynamics.nqr import (
    SelectivePulse,
    gaussian_efg_distribution,
    simulate_fid_efg_distribution,
)
import numpy as np

distribution = gaussian_efg_distribution(
    site,
    quadrupole_std_hz=2e3,
    samples=41,
)

fid = simulate_fid_efg_distribution(
    distribution,
    "x",
    times_seconds=np.linspace(0.0, 20e-3, 512),
    excitation=SelectivePulse("x", duration_seconds=2.5e-6, nutation_hz=100e3),
)
```

The distribution simulators check whether a coarse EFG grid can artificially rephase within the simulated duration. Increase the number of isochromats when the warning appears, or pass `rephase_action="ignore"`

only after a convergence check.

In an SLSE experiment, the spectrum is not the FFT of the echo train. It is the FFT of the averaged echo acquired over a receiver window T_{acq} centered on a selected echo, with T_{acq} shorter than the pulse spacing. The finite acquisition window itself broadens the spectrum:

```
from spin_dynamics.nqr import simulate_slse_acquisition_spectrum

slse_spectrum = simulate_slse_acquisition_spectrum(
    distribution,
    sequence,
    acquisition_duration_seconds=200e-6,
    acquisition_points=256,
    echo_index=-1,
    noise={"target_snr": 20.0, "seed": 123},
    deconvolution_strength=1e-2,
)
```

Noise is added to the complex time-domain acquisition before the FFT. The optional deconvolution performs a regularized inversion of the same finite-window FFT operator. It is useful for exploring acquisition-window broadening, but it can amplify noise and should be checked across plausible SNR values.

7.12 RFI Rejection and Echo-Aware SLSE Fitting

Low-field pulsed NQR is often limited by coherent radio-frequency interference (RFI), not only by receiver noise. This is especially true for $^{14}\mathrm{N}$ compounds whose transition frequencies can lie in or near the AM broadcast band. The new `spin_dynamics.interference` layer treats this as a reference-channel estimation problem on the same shot clock as the NQR sequence. It is deliberately split into two parts:

- sequence-independent RFI sources, pickup coils, cancellers, and diagnostics in `spin_dynamics.interference`;
- NQR-specific mask adapters, such as `slse_acquisition_mask`, in `spin_dynamics.nqr`.

The mask is essential. During transmit and ringdown the primary receiver is not a faithful linear measurement of the field, while the reference channels may continue to observe the environment. During quiet receive intervals the primary sample can be modeled as

$$y_n = s_n(\theta) + r_n + \epsilon_n,$$

where $s_n(\theta)$ is the NQR response, r_n is the RFI coupled into the primary channel, and ϵ_n is receiver noise. The reference array $X \in \mathbb{C}^{K\times N}$ contains K auxiliary pickup channels,

$$x_{k,n} = \mathbf{c}_k^{\mathsf{T}} \frac{\mathrm{d}}{\mathrm{d}t}\mathbf{B}_{\mathrm{RFI}}(\mathbf{p}_k, t_n) + \ell_k s_n + \xi_{k,n},$$

with pickup vector $\mathbf{c}_k$, coil position $\mathbf{p}_k$, optional NQR leakage ℓ_k, and reference noise $\xi_{k,n}$. Far-field RFI is represented by a nearly uniform vector field. Near-field RFI is represented by magnetic dipole sources with the usual $1/r^3$ spatial pattern, which makes multiple reference stations valuable because each station sees a different linear mixture of the interfering sources.

7.12.1 Masked and Windowed Reference Cancellation

The simplest stationary reference canceller predicts the RFI in the primary channel by a tapped linear model,

$$\widehat{r}_n = \sum_{k=1}^{K} \sum_{\ell=0}^{L-1} h_{k,\ell}\, x_{k,n-\ell}.$$

Let ϕ_n be the row vector of tapped reference samples at time n, and let B be the boolean fitting mask. Gated ridge fitting solves

$$\widehat{\mathbf{h}} = \arg\min_{\mathbf{h}} \sum_{n \in B} |y_n - \phi_n \mathbf{h}|^2 + \lambda \|\mathbf{h}\|_2^2.$$

For SLSE, B should exclude transmit, ringdown, and expected echo windows when the method does not explicitly model the NQR signal. The scalar canceller is the $L = 1$ case. The FIR form allows small group delays or front-end phase shifts between the reference and primary chains.

Time-varying interferers, such as randomly modulated AM-like carriers or moving near-field sources, are not well described by one global coefficient vector. Two alternatives are implemented. Mask-aware LMS/NLMS/RLS produces a continuous prediction but freezes coefficient updates outside trusted windows. The offline windowed ridge method fits one coefficient vector per window, $\mathbf{h}_w$, and regularizes adjacent windows:

$$\min_{\{\mathbf{h}_w\}} \sum_{w} \sum_{n \in B_w} |y_n - \phi_n \mathbf{h}_w|^2 + \lambda \sum_{w} \|\mathbf{h}_w\|_2^2 + \alpha \sum_{w>0} \|\mathbf{h}_w - \mathbf{h}_{w-1}\|_2^2.$$

This is the recommended first offline method when the front end has not saturated. It does not need to run in real time, and it can use future samples to estimate slowly varying coupling.

Impulsive pickup—Poisson-timed switching transients whose bursts are large and rare—violates the Gaussian-residual assumption of squared-error fitting: a handful of spiky samples dominate the normal equations and drag $\widehat{\mathbf{h}}$ away from the coherent coupling. The robust canceller `robust_fir_canceller` replaces the ℓ_2 loss with a Huber loss, minimized by iteratively reweighted least squares. Each iteration solves the weighted normal equations $\Phi^{\mathsf{H}} W \Phi\, \mathbf{h} = \Phi^{\mathsf{H}} W \mathbf{y}$ with $W = \mathrm{diag}(w_n)$, where $w_n = 1$ for $|r_n| \le \delta s$ and $w_n = \delta s / |r_n|$ otherwise, s is a robust (MAD-based) scale of the residual re-estimated each iteration, and δ (default 1.345) is the transition point. Samples in the impulsive tail are down-weighted so the fit tracks the stationary RFI path; the returned `fit_weights` expose which samples were treated as outliers.

The robust fit protects the coherent-transfer estimate but leaves the impulses in the cleaned output. `sparse_reference_canceller` instead models them, minimising $\|y - \Phi h - s\|^2 + \lambda\|h\|^2 + \mu\|s\|_1$ by alternating a ridge least-squares update of the FIR coefficients (fitted on the mask with s removed, so the transfer stays unbiased) with a soft-threshold update of the sparse impulse estimate s over the whole record. With μ set above the NQR echo amplitude and below the switching-transient amplitude, the bursts are captured in s while the signal passes through, and `cleaned` has both the coherent RFI Φh and the impulses s removed.

When the reference-to-primary coupling is strongly frequency dependent – a resonant probe or a long impulse response – a compact FIR would need many taps. `frequency_domain_canceller` instead estimates a complex transfer $W_k(f)$ in every DFT bin from averaged cross-spectra over Hann-windowed segments that lie inside the fit mask,

$$W(f) = \big(S_{xx}(f) + \lambda I\big)^{-1} s_{xy}(f), \qquad \widehat{Y}(f) = W(f)^{\mathsf{H}} X(f),$$

with $S_{xx}(f) = \sum_{\mathrm{seg}} X X^{\mathsf{H}}$ and $s_{xy}(f) = \sum_{\mathrm{seg}} X\,\overline{Y}$. The prediction is returned to the time domain by weighted overlap-add. Because the NQR signal is uncorrelated with the references it does not bias W, only

adds estimator variance, so gating to the baseline set is helpful but not required. The result also carries the multiple-coherence spectrum $\gamma^2(f) = s_{xy}^{\mathsf{H}} S_{xx}^{-1} s_{xy} / S_{yy} \in [0, 1]$, the fraction of primary power the references explain at each frequency – a direct readout of which parts of the band are cancellable. Application is exact for a pass-through and approximate when the coupling impulse response approaches the segment length (per-segment circular convolution); the first and last segment are edge-tapered.

All of the above need a correlated reference. When the interferer is narrowband and no clean reference exists, `kalman_harmonic_canceller` exploits its structure directly. Each carrier f_j is modelled as $I_j \cos(\omega_j n) - Q_j \sin(\omega_j n)$ with the quadratures $[I_j, Q_j]$ following a random walk, and a Kalman filter tracks them from the (scalar) primary. The subtracted estimate is the *a priori* prediction, so the current sample's NQR content never enters the carrier estimate, and measurement updates are frozen wherever `update_mask` is false, letting the tracker coast through the echo windows rather than absorb them. The knobs are the process standard deviation (how fast the amplitude may drift) and the measurement standard deviation (the observation-noise scale). Because it needs only the carrier frequencies, it removes a drifting amplitude-modulated broadcast line sitting on top of the NQR line with no reference channel at all.

7.12.2 Echo-Aware Joint Fitting

For SLSE, the echo train itself has strong temporal structure. Reference-only cancellation uses this structure only indirectly through masks and diagnostics; it does not prevent the RFI model from absorbing echo-like primary-channel structure. The echo-aware joint estimator adds an explicit signal subspace. Let $\Psi \in \mathbb{C}^{N \times M}$ be a basis for plausible NQR echo trains. The fixed joint model is

$$\mathbf{y} \approx \Phi \mathbf{h} + \Psi \mathbf{a},$$

where Φ is the tapped reference design matrix and $\mathbf{a}$ are signal-basis coefficients. The fitted reference contribution is still the RFI estimate, while $\Psi \widehat{\mathbf{a}}$ is the fitted NQR component.

The practical SLSE example uses the windowed form,

$$\begin{aligned} \min_{\{\mathbf{h}_w\}, \mathbf{a}} \quad & \sum_w \sum_{n \in B_w} |y_n - \phi_n \mathbf{h}_w - \psi_n \mathbf{a}|^2 \\ & + \lambda_{\text{ref}} \sum_w \|\mathbf{h}_w\|_2^2 + \alpha \sum_{w>0} \|\mathbf{h}_w - \mathbf{h}_{w-1}\|_2^2 + \lambda_s \|\mathbf{a}\|_2^2. \end{aligned}$$

Here ψ_n is the row of the SLSE basis at sample n. Unlike the reference-only methods, this fit may include the echo windows because the echo has its own model block. The implementation is `windowed_joint_signal_reference_canceller`. It returns both a cleaned waveform $y - \widehat{r}$ and `signal_estimate`, the fitted NQR component. For parameter estimation, the example scores `signal_estimate` because that is the quantity the joint estimator is designed to recover.

The default SLSE basis in `examples/plot_nqr_rfi_statistical_study.py` is a small grid of echo trains,

$$\psi_{\nu, T_{2e}}(t_n) = \sum_j g(t_n - \tau_j) \exp(-\tau_j / T_{2e}) \exp(\mathrm{i} 2\pi \nu \tau_j),$$

where g is the echo-window template and τ_j are the expected echo centers. The command-line options `-echo-aware-frequency-span-hz`, `-echo-aware-frequency-points`, and `-echo-aware-t2e-grid-ms` control this dictionary. This is still a linear ridge problem because the frequencies and decay constants are gridded. A future nonlinear refinement step can use the best grid component as an initial guess for continuous ν and T_{2e}.

7.12.3 Parameter Endpoints and Diagnostics

RFI suppression in dB is a useful engineering diagnostic, but it is not the experimental endpoint. For NQR detection, the important quantities are the resonant frequency, initial echo amplitude, and decay constant T_{2e}. The statistical example projects the recovered record onto the echo templates, producing complex responses c_j at echo times τ_j. It then estimates

$$\widehat{\nu} = \frac{1}{2\pi}\frac{\mathrm{d}}{\mathrm{d}\tau}\,\mathrm{unwrap}\arg c(\tau),$$

and fits

$$\log|c_j| \approx \log A_0 - \tau_j/T_{2e}.$$

The plotted endpoints are therefore $|\widehat{\nu}-\nu|$, relative A_0 error, and relative T_{2e} error, all averaged over Monte Carlo trials with approximate 95% confidence intervals of the mean.

The diagnostics layer also provides:

- RFI suppression using the clean signal when available;
- matched-filter SNR improvement;
- amplitude and phase bias of the recovered signal;
- prominent residual spectral lines;
- reference design rank and condition number;
- the reference-noise-injection estimate, i.e. the white noise a fitted canceller passes from noisy reference channels into the cleaned output ($\mathrm{var} = \sum_k \sigma_k^2 \sum_l |h_{k,l}|^2$), the "noise figure" that decides whether suppression is a net win;
- saturation flags for front-end clipping checks.

Saturation is a hard boundary. If the primary channel clips before digitization, linear reference cancellation cannot reconstruct the lost waveform. Offline methods are appropriate only when the RFI remains inside the linear dynamic range of the receiver.

7.12.4 Active Feedforward Cancellation

When the interferer would saturate the receiver, the cure is to cancel it in the analog domain, before the amplifier and ADC, so the digitizer only ever sees the residual. The `spin_dynamics.interference.active` module models this with a `CompensationActuator` and `feedforward_cancel`. A fitted linear model (the same `LinearCancellerModel` used for digital cancellation, typically calibrated on the reference chain) supplies the commanded compensation field $\widehat{r}_n$ from the continuous references. The actuator realizes it imperfectly, and the residual seen by the ADC is

$$y_n^{\mathrm{res}} = y_n - g\,\widetilde{r}_{n-L},$$

where g is the analog gain/phase mismatch, L the causal actuator latency, and $\widetilde{r}$ the commanded field after any drive-range clip and bandwidth limit. The ADC then clips y^{res} rather than y. This is the decisive difference from digital cancellation `model.apply(clip(y))`, which subtracts *after* the clip and cannot recover saturated peaks.

Two physical limits bound feedforward. The causal latency limits the cancellation bandwidth: a tone at angular frequency ω is only suppressed by the factor $|1 - g\,e^{-i\omega L}|$, so cancellation fails (and eventually *adds* interference) once ωL approaches π. The finite drive range `max_field` caps how much of a large interferer the coil can oppose, so deep nulls require both low latency and adequate headroom. The example `examples/plot_nqr_rfi_active_feedforward.py` drives the primary well past full scale and contrasts digital-too-late against active feedforward, then sweeps both limits. Because the digitized active residual is again linear, a digital canceller can be run on it afterward for the last few decibels.

7.12.5 Loading Measured Records

A measured acquisition is just windows of ADC samples: a primary channel and one or more reference channels on a shared clock. The gating the cancellers need is not recorded per sample – it is reconstructed from the pulse-sequence timing. `slse_mask_from_metadata` and `sorc_mask_from_metadata` rebuild an acquisition mask of exactly the recorded sample count from the echo spacing, detection duration, pulse count, ringdown, and baseline offsets; `nqr_recording_from_samples` pairs the sample windows with that mask and returns an `RFIRecording` that drops straight into the `(primary, references, mask)` contract. `save_rfi_recording` and `load_rfi_recording` round-trip a recording through a NumPy `.npz` archive, storing the reconstructed mask as its label array so no re-derivation is needed on reload.

7.12.6 Typical Monte Carlo Results

The example `examples/plot_nqr_rfi_statistical_study.py` synthesizes five independent near-field magnetic-dipole RFI sources, tri-axial reference stations, a complex SLSE echo train, receiver noise calibrated by initial echo SNR, and randomly modulated AM-like interference. It compares one-station scalar cancellation, all-station scalar cancellation, offline windowed ridge, NLMS adaptation, and the echo-aware joint fit.

The most important control is spectral overlap. With the command-line default `-rfi-spectral-mode in-band`, the RFI carrier is placed near the echo phase frequency and contaminates the NQR parameter estimator. With `-rfi-spectral-mode out-of-band`, the same machinery is a null/control case: the interference mostly averages out of the echo estimator, so parameter error is dominated by receiver noise.

For 48 in-band Monte Carlo trials, the default example produced:

Method	Frequency error (Hz)		Relative T_{2e} error	
	SNR 4	SNR 35	SNR 4	SNR 35
1 station scalar	25.09	19.76	4.659	3.901
all stations scalar	27.21	8.03	3.507	0.467
windowed ridge	26.05	3.19	3.584	0.195
NLMS adaptive	29.26	8.05	6.041	0.345
echo-aware joint	5.08	2.31	7.813	0.258

The echo-aware method gives the clearest improvement in resonant-frequency accuracy when the RFI overlaps the NQR estimator, especially at low initial echo SNR. Its low-SNR T_{2e} estimates remain heavy-tailed because decay fitting is sensitive to failed or over-regularized late echoes; this is a real limitation of the first linear-grid method, not a plotting artifact. At high SNR, windowed ridge and echo-aware joint fitting are both useful, with the joint method favoring frequency recovery and the windowed method sometimes giving a slightly cleaner decay estimate.

The same run also tests near-field reference-count scaling at initial echo SNR 12. For windowed ridge, going from one to five tri-axial stations improved mean frequency error from 15.44 to 8.43 Hz and relative T_{2e} error from 2.848 to 0.877. For the echo-aware joint fit, the corresponding frequency error was already much lower, 3.38 to 2.57 Hz, and relative T_{2e} error improved from 2.959 to 0.547. The suppression diagnostic is much larger for the joint fit because the reported endpoint is the fitted NQR component rather than only the waveform after subtracting predicted RFI.

In the out-of-band control, all methods become mostly noise-limited at high SNR: frequency errors collapse to roughly 0.2–0.4 Hz and relative T_{2e} errors to about 0.05. That null result is valuable. It shows why RFI rejection must be evaluated with spectral overlap and NQR parameter endpoints; residual power alone can make a benign out-of-band interferer look experimentally important, or make an in-band interferer look acceptable when it is biasing the frequency and decay estimates.

```
python examples/plot_nqr_rfi_cancellation.py \
    --output results/nqr_rfi_cancellation.png

python examples/plot_nqr_rfi_statistical_study.py \
    --rfi-spectral-mode in-band \
    --output results/nqr_rfi_statistical_study.png

python examples/plot_nqr_rfi_statistical_study.py \
    --rfi-spectral-mode out-of-band \
    --output results/nqr_rfi_statistical_outofband_control.png
```

7.13 Two-Frequency Population Transfer

The two-frequency NQR workflow follows the perturbation-plus-detection idea used in 2D NQR. A selective pulse on one transition changes populations in the shared three-level spin-1 manifold; a later SLSE block detects the changed amplitude on another transition. The normalized diagnostic is

$$\Delta(p, q) = \frac{S(p, q)}{S_0(q)} - 1,$$

where p is the perturbation transition and q is the detection transition.

```
from spin_dynamics.nqr import SelectivePulse, simulate_population_transfer

transfer = simulate_population_transfer(
    site,
    SelectivePulse("x", duration_seconds=50e-6, nutation_hz=10e3),
    sequence,
    orientations="powder",
)

print(transfer.normalized_difference)
```

The diagnostic plotting example constructs a compact 3×3 map over the spin-1 x, y, and z transitions. Off-diagonal features indicate that perturbation and detection frequencies are connected through the same quadrupolar site.

8 Unified Experiment Workflow

The `spin_dynamics.experiment` package is the recommended entry point for new code. It is a thin, declarative layer over the validated `run_*` and `simulate_*` workflows: you describe an experiment with frozen-dataclass specs, front-load validation with `plan()`, execute with `run()`, and save the result with provenance. The facade never re-implements any dynamics – a default `Experiment` reproduces the direct workflow call bit for bit – so it costs nothing in fidelity and adds compatibility checking, runtime estimation, automatic hardware wiring, and serialization on top. If you already know which `run_*` function you want, calling it directly (Chapter 9) remains fully supported; reach for the facade when you want that surrounding scaffolding or a single interface across the NMR, imaging, NQR, and ESR engines.

8.1 The Eight-Step Workflow

The facade organizes an experiment into the same stages every example follows: (1) define the sample; (2) define transmitters/receivers; (3) define the sequence; (4) define the acquisition; (5) `plan()` – resolve the workflow, check compatibility, estimate cost; (6) solve fields (done automatically inside `plan()`/`run()` when coils are given); (7) `run()`; and (8) view, analyze, and save.

```
from spin_dynamics.experiment import (
    Experiment, CPMGTrain, Sample, Hardware, Acquisition,
)

study = Experiment(
    sequence=CPMGTrain(num_echoes=8),
    sample=Sample(t1_seconds=2.0, t2_seconds=2.0),
    hardware=Hardware(probe="tuned"),
    acquisition=Acquisition(numpts=501, maxoffs=10.0),
)

plan = study.plan()
print(plan.report())
record = study.run()
record.save("run1.npz")
```

8.2 Planning Before Running

`plan()` is the front-loaded validation stage. It resolves which workflow will run and returns an `ExperimentPlan` whose `errors` and `warnings` are also available structurally as `plan.findings`. `run()` refuses to execute a plan with errors; warnings are surfaced but do not block. The current rules cover isochromat-grid rephasing (with the recommended `numpts`, respecting `Acquisition.rephase_action`), noise spec validity, coil-field wiring, and reduced-vs-full NQR model selection. Setting a spec field the resolved workflow does not consume produces an "ignored" warning, so mismatched specs never fail silently.

8.3 Automatic Hardware Wiring

For imaging sequences, describe the transmit coil as geometry in SI meters and the facade solves its Biot–Savart B_1 on the phantom grid at plan time, projects it transverse to B_0, normalizes it to a sample-weighted mean of one, and passes it to the workflow in place of the synthetic default. The solve is cached by a geometry hash and reused by `run()`; `plan()` reports the transverse fraction of the coil's B_1 over the sample and warns when most of it is parallel to B_0.

8.4 NQR and ESR

The same interface drives the NQR and ESR engines. For NQR, `plan()` picks the reduced two-level or full density-matrix engine via `select_nqr_model` and reports why; `NQRSLSE.nutation_hz` uses the reduced engine's effective two-level Rabi convention and the adapter converts to the bare $\gamma B_1/2\pi$ the full engine needs. For pulsed ESR, set `Sample.esr_system` and a `Hardware.b0 = UniformB0(field_tesla=...)` so the electron Larmor frequency is fixed, then use an `ESRFID` or `ESRHahnEcho` sequence.

8.5 Saving and Reloading

`record.save(path)` writes an NPZ archive holding every array field of the native result plus a JSON document with the full experiment spec, provenance, and scalar result fields. `load_run(path)` returns a `LoadedRun` exposing the raw arrays, the reconstructed native result, and – via `loaded.experiment` – the original spec, so a saved run can be re-run or tweaked. Specs are JSON-serializable directly (`Experiment.to_json()` / `Experiment.from_json(...)`). Runnable versions of every stage above live in `examples/experiment_facade_quicks` `examples/experiment_imaging_with_coil.py`, and `examples/experiment_nqr_auto_model.py`.

8.6 Config-Driven Runs (CLI)

For runs you would rather describe in a file than in Python, the facade reads a human-friendly TOML or JSON config in which each spec is a table tagged by its `type`. The `sample`, `hardware`, and `acquisition` sections may omit `type` (their class is implied) and may be dropped entirely to accept the defaults, so a minimal config is just the `[sequence]` table.

```
[sequence]
type = "CPMGTrain"
num_echoes = 8

[sample]
t1_seconds = 2.0
t2_seconds = 2.0

[hardware]
probe = "tuned"

[acquisition]
numpts = 501
maxoffs = 10.0
```

A command-line runner drives the same plan/run/save flow via `python -m spin_dynamics.experiment`:

Command	Effect
`plan CONFIG`	Resolve and validate the config; print the plan. Exits non-zero if the plan has errors.
`run CONFIG [-o NPZ]`	Plan, then run (refusing an erroring plan), and optionally save the result archive.
`show RUN.npz`	Print a saved run's spec and provenance.
`convert CONFIG OUT`	Rewrite a config between `.toml` and `.json`.

In Python, `save_config` / `load_config` and `experiment_to_config` / `experiment_from_config` expose the same round-trip. This friendly form covers spec fields with scalar or array values, including imaging phantoms and coil geometry; a fully general result archive still uses the NPZ/JSON form of the previous section. A ready-to-run config ships at `examples/experiment_config_cpmg.toml`.

9 Workflow Guides

This chapter is the practical half of the manual. The earlier chapters define the model layers; the sections below show how those layers are assembled into experimental workflows. The order is roughly from the most common NMR building blocks to more specialized field, motion, exchange, and analysis tools.

9.1 CPMG

Application code should normally start with the public workflow runners:

```
from spin_dynamics.workflows import run_tuned_cpmg

result = run_tuned_cpmg(numpts=101, maxoffs=10)
print(result.del_w.shape)
print(result.echo.shape)
print(result.snr)
```

The asymptotic CPMG runners return `CPMGResult` objects with the offset grid, asymptotic magnetization, received spectrum, time-domain echo, acquisition time vector, optional SNR, probe label, and the default phase-cycle metadata.

Finite and diagnostic CPMG examples use the same public timing vocabulary: `num_echoes` and `echo_spacing_seconds`. The ideal convention is a 90_y excitation that prepares $+I_x$, followed by 180_x refocusing pulses and the default two-step CPMG/PAP excitation phase cycle. The microscopic bulk-water example keeps that convention but replaces fitted scalar T_2 with a shared Redfield/dipolar Liouville propagator for one observed proton relaxed by an isotropically tumbling partner:

```
python examples/plot_redfield_water_cpmg.py \
    --echo-spacing-seconds 0.02 \
    --correlation-time-seconds 2.5e-12 \
    --output water_redfield_cpmg.png
```

The older unit-suffixed plotting conveniences, `-echo-spacing-ms` and `-tau-c-ps`, remain aliases, but new examples should prefer the seconds form used by the workflow API.

9.1.1 Uhrig Dynamical Decoupling

The sequence utility layer also provides ideal CPMG and Uhrig dynamical decoupling timing helpers. For n refocusing pulses in a total evolution window T, UDD places the jth pulse at

$$t_j = T \sin^2\left(\frac{j\pi}{2n+2}\right), \qquad j = 1, \ldots, n.$$

The pulses cluster near the beginning and end of the window. This does not usually improve simple unrestricted diffusion in a static gradient, but it can strongly suppress low-frequency correlated detuning noise.

```
from spin_dynamics.sequences import (
    cpmg_pulse_times,
    dephasing_filter_function,
```

```
    udd_pulse_times,
)

duration = 1.0
omega = 0.1 / duration
cpmg = cpmg_pulse_times(8, duration)
udd = udd_pulse_times(8, duration)

print(dephasing_filter_function(omega, cpmg, duration))
print(dephasing_filter_function(omega, udd, duration))
```

9.2 Finite Echo Trains

```
from spin_dynamics.workflows import run_matched_cpmg_train

train = run_matched_cpmg_train(
    numpts=51,
    num_echoes=8,
    auto_refine_grid=True,
    num_workers=None,
)
```

Finite train results contain one acquired spectrum and one echo per echo index. They also include trapezoidal echo integrals and physical echo-center times. The default CPMG/PAP phase-cycle table is available as `train.phase_cycle`. For long trains, use the rephasing checks or enable `auto_refine_grid=True` so the offset grid is refined before RF matrices are built.

9.2.1 CPMG-IR Trains

Finite inversion-recovery trains add an inversion delay axis before the CPMG readout:

```
from spin_dynamics.workflows import run_matched_cpmg_ir_train

ir = run_matched_cpmg_ir_train(
    numpts=31,
    num_echoes=4,
    tauvect=[0.5e-3, 2e-3, 6e-3, 12e-3],
    auto_refine_grid=True,
)
```

The returned arrays are indexed by inversion delay first and echo number second. This makes the workflow useful for compact T_1-encoded echo-train tests without manually assembling the inversion pulse, delay, excitation, and refocusing blocks.

9.2.2 Probe Parameter Sweeps

Probe-aware CPMG workflows can also be swept over resonator Q or mistuning:

```
from spin_dynamics.workflows import run_tuned_q_sweep

sweep = run_tuned_q_sweep(q_values=[20, 50, 100], numpts=51)
```

Use the finite-train sweep variants when echo-to-echo evolution matters. The finite sweep results keep one `CPMGTrainResult` per parameter value and also stack echo integrals for quick heat-map style summaries.

9.3 FID

```
from spin_dynamics.parameters import set_params_ideal_fid
from spin_dynamics.workflows.fid import sim_fid_ideal

sp, pp = set_params_ideal_fid(numpts=101)
macq, fid, tvect = sim_fid_ideal(sp, pp)
```

9.4 Imaging

```
import numpy as np
from spin_dynamics.workflows import (
    make_imaging_field_maps,
    run_tuned_phase_encoded_cpmg_imaging,
    form_imaging_image,
)

rho = np.eye(8)
fields = make_imaging_field_maps(
    rho,
    b0_map=np.zeros_like(rho),
    b1_tx_map=np.ones_like(rho),
    b1_rx_map=np.ones_like(rho),
)
result = run_tuned_phase_encoded_cpmg_imaging(
    fields,
    num_echoes=4,
    ny=9,
    receive_mode="weighted",
)
image = form_imaging_image(result, mode="echo_sum")
```

Imaging workflows return raw k-space, reconstructed per-echo images, field maps, relaxation maps, gradient metadata, and sequence timing. Image formation is intentionally separated from acquisition so selected-echo, echo-summed, fitted-ρ, and fitted-T_2 displays can be compared. `make_imaging_field_maps` accepts spatial B0, transmit-B1, and receive-B1 maps, and the ideal imaging path can include an inversion-recovery preparation through `run_t1_encoded_phase_encoded_cpmg_imaging`. When `noise` is supplied, clean k-space and image fields remain in the result while noisy counterparts are stored in `kspace_noisy`, `image_noisy`, and `magnitude_noisy`.

9.4.1 Frequency-Encoded Imaging: Spin-Warp and RARE

The phase-encoded workflows above fill k-space one point per phase-encode step. `run_spin_warp_imaging` and `run_rare_imaging` add a readout (frequency-encode) gradient during acquisition, so each spin echo samples a whole k-space line. They are built on the Lagrangian motion engine—one static isochromat

per voxel—which supports the two-axis gradient waveforms that the scalar-gradient arbitrary-pulse kernels cannot express; they are ideal-probe and reuse `reconstruct_image_from_kspace`.

```
import numpy as np
from spin_dynamics.workflows import run_spin_warp_imaging, run_rare_imaging

rho = np.zeros((32, 32)); rho[8:24, 10:18] = 1.0
t2 = np.full((32, 32), 60e-3)

# Spin-warp: one spin echo per phase-encode line (pz excitations, no blurring).
ref = run_spin_warp_imaging(rho, fov=(0.02, 0.02), t2_map=t2)

# RARE / fast spin echo: each echo reads a different line, so a train of length 8
# needs only ceil(pz / 8) excitations, at the cost of T2 blurring.
fse = run_rare_imaging(rho, fov=(0.02, 0.02), t2_map=t2, echo_train_length=8)
```

Readout is along x and the phase encode along z. Each echo is gradient-balanced (an x pre-dephase and rewind return k-space to the origin before each 180-degree pulse), so the train stays a clean Meiboom-Gill CPMG. The T2 decay across the echo train weights the phase-encode lines—`line_echo_time` records the echo time of each k_z line—which broadens the point-spread function (the RARE blurring, strongest for short T_2). `echo_train_length=1` recovers the spin-warp reference. The example `plot_rare_imaging.py` contrasts the two on a two-T_2 phantom.

Both workflows accept the same inputs as the phase-encoded path so the effects of inhomogeneity can be evaluated: pass `rho` with per-voxel `b0_map` (absolute angular off-resonance, rad/s), `b1_tx_map`, `b1_rx_map`, `t1_map`, and `t2_map`, or pass an `ImagingFieldMaps` container directly (with the map keywords omitted). B0 inhomogeneity distorts geometry along the readout axis and B1 maps shade the image. An unresolved sub-voxel B0 spread is modelled with `num_offsets` > 1 isochromats per voxel spread over $\pm$`offset_spread` (rad/s), the counterpart of the `ny`/`maxoffs` samples of the phase-encoded path; a spin echo refocuses the static spread at each echo, so it blurs the readout axis (the T_2^* point-spread function) without decaying the train. The example `plot_imaging_inhomogeneity.py` shows B0 distortion, sub-voxel T_2^* blur, and B1 shading.

A practical issue in inhomogeneous-field imaging is that the excited slice is neither flat nor uniform in real space: an RF pulse excites the spins whose frequency falls in its band, so the slice follows the curved iso-B0 contours and is shaded by B1. `imaging_slice_sensitivity` maps this sensitive slice from the same B0/B1 maps by computing, for every voxel at its own off-resonance (`b0_map - center_frequency`, rad/s) and transmit-B1, the excited transverse magnetization of a rectangular RF pulse (so the slice profile follows the pulse bandwidth $\sim 1/$`excitation_duration`), weighted by the receive B1; `refocusing=True` also applies the 180-degree refocusing efficiency for the spin-echo sensitive volume. The example `plot_sensitive_slice.py` visualizes the curved, non-uniform slice in a single-sided-like field.

9.4.2 Balanced SSFP: Steady-State Imaging and Off-Resonance Banding

Balanced SSFP (bSSFP; also TrueFISP, FIESTA, balanced FFE) is the workhorse steady-state gradient echo. Unlike the spin-echo imagers above, it never lets the magnetization recover to equilibrium between excitations: a train of small-flip pulses at a short, fixed T_R drives a *steady state*, and every gradient moment—readout and phase encode—is rewound to zero each T_R (the "balanced" condition). `run_bssfp_imaging` builds exactly this on the same moving-isochromat engine and the same (`B0, B1`) field maps as `run_spin_warp_imaging`, so the two are directly comparable: it is a continuous train of balanced T_Rs with a phase-cycled RF phase, one k-space line acquired per T_R, reconstructed with the same `reconstruct_image_from_kspace`.

```
import numpy as np
from spin_dynamics.workflows import run_bssfp_imaging, bssfp_optimal_flip_deg

rho = np.zeros((32, 32)); rho[8:24, 10:22] = 1.0
t1 = np.full((32, 32), 0.3); t2 = np.full((32, 32), 0.1)

img = run_bssfp_imaging(
    rho, t1_map=t1, t2_map=t2,
    b0_map=b0_rad_per_s,             # off-resonance -> banding
    b1_tx_map=b1_relative,           # flip shading
    flip_angle_deg=bssfp_optimal_flip_deg(0.3, 0.1),
    num_dummy_tr=200,                # dummy TRs to reach the steady state
    phase_increment_deg=180.0,       # phase cycling: passband centred on resonance
)                                    # img.magnitude has shape (px, pz, 1)
```

The appeal is SNR. On resonance the steady-state signal is high—at the optimal flip $\cos\alpha^\star = (T_1/T_2 - 1)/(T_1/T_2 + 1)$ it reaches $\frac{1}{2}M_0\sqrt{T_2/T_1}$ (`bssfp_optimal_flip_deg` returns $\alpha^\star$)—which is why bSSFP is a favourite at low field, where SNR is scarce. The catch is the flip side of "balanced": each T_R rewinds the *gradient* phase but not the *B0* phase, so the steady state depends on the per-T_R off-resonance precession. Its off-resonance response (`bssfp_steady_state_signal`, computed with the same engine primitives but no gradients) is flat over a passband and then drops to periodic nulls—the bSSFP "dark bands"—spaced $1/T_R$ in frequency. The RF phase increment sets where the passband sits: 180° centres it on resonance with nulls at $\pm 1/(2T_R)$; 0° shifts it half a band, nulls at $0, \pm 1/T_R$.

In an inhomogeneous B0 those bands cut across the object as signal voids, and B1 (flip-angle) inhomogeneity shades the image. This is the steady-state SNR-versus-robustness trade-off, the imaging cousin of the steady-state SORC result of Section 7.10.1—a steady-state sequence buys sensitivity but pays in off-resonance and field-inhomogeneity sensitivity. The example `plot_bssfp_field_inhomogeneity.py` makes this concrete for a phantom in mildly inhomogeneous fields: it plots the banding profile and the T2/T1 flip response, then images the phantom four ways—uniform (reference), a mild B0 residual ($\sim$ a band across the FOV) that carves a dark banding void, a transmit-B1 gradient that shades the image, and a *phase-cycled* pair (increments 180° and 0°, whose passbands are complementary) combined by maximum, the standard fix that fills the bands back in at twice the scan time.

Two practical points fall out of running bSSFP honestly. First, the steady state must be established: `num_dummy_tr` balanced dummy TRs (with an optional $\alpha/2$ "catalyzation" that lands the magnetization near the steady-state cone and suppresses the oscillatory transient) are played before acquisition, and too few of them leaves a transient that modulates the phase-encode lines and blurs or shifts the image—so a few $\times T_1/T_R$ dummies are needed. Second, because the RF phase is cycled, the receiver phase must follow it: each acquired line is demodulated by its transmit phase, without which the alternating pulse phase imprints a $(-1)^{\text{line}}$ modulation on k-space that shifts the image by half the field of view along the phase-encode axis.

9.4.3 Slice-Selective Excitation and 3D Multi-Slice Imaging

`imaging_slice_sensitivity` above is a passive analysis: it uses a hard (rectangular) pulse and relies on the existing B0 inhomogeneity to define the band, with no gradient applied during the RF. Real slice selection instead plays a shaped RF while a gradient is on, so a chosen plane is excited even in a uniform background. `make_slice_selective_excitation` builds this pulse as a windowed-sinc RF train carrying the slice gradient, followed by a refocusing (rephasing) gradient lobe; `simulate_slice_profile` excites a line of spins and returns the through-slice profile, which is a localized band (sharp, unlike the hard-pulse approximation) whose thickness scales as the excitation bandwidth over the gradient.

```
import numpy as np
from spin_dynamics.workflows import simulate_slice_profile, run_multislice_imaging

# Through-slice profile of a 1 ms windowed-sinc 90-degree pulse.
profile = simulate_slice_profile(duration=1e-3, slice_gradient=1.5e7)

# True-3D multi-slice imaging of a 3D volume in a spatially varying (B0, B1) field.
rho = np.zeros((16, 5, 16)); rho[6, 2, 9] = 1.0
volume = run_multislice_imaging(
    rho,
    slice_gradient=1.5e7,
    fov=(0.02, 0.02, 0.02),
    b0_map=b0_volume,        # rad/s, shape of rho
    b1_tx_map=b1_volume,     # relative
    b1_rx_map=b1_volume,
)            # volume.magnitude has shape (nx, n_slices, nz)
```

`run_multislice_imaging` is the true-3D path. A single 3D walker ensemble lives in a 3D `MotionFieldMaps` carrying the actual `(B0, B1)` field, and every slice is excited (its carrier offset to place the slice) and read out through the moving-isochromat engine. Because the slice is selected by *total* off-resonance—slice gradient plus local B0—a nonuniform B0 curves and displaces the excited slice and warps the read-out, exactly as in a real magnet; the example `plot_multislice_halbach_imaging.py` shows this for a structured phantom in a mild Halbach saddle, displaying acquired slices and the 3D reconstruction. `run_multislice_imaging_separable` is a fast approximation that reduces the slice pulse to a 1D through-plane weight and forms each slice with the validated 2D spin-warp workflow; it ignores in-plane field variation (the slice stays flat), trading that fidelity for speed on large volumes. Genuinely Fourier-encoded 3D (slab-select plus a second phase encode reconstructed with `ifftn`) is left for later.

The slice pulse, the moving-isochromat engine, and the phase-encoded imaging kernels all read the same per-voxel physics—off-resonance, transmit/receive B1, relaxation, and density—through the dimension-agnostic `spin_dynamics.fields` layer (`SpatialDomain` and `SpatialFieldMaps`), so the 1D, 2D, and 3D paths share one field representation and one gradient-coupling rule.

9.5 Field Solvers

`spin_dynamics.fields` computes device fields from first principles rather than assuming a synthetic map. Two families live here: closed-form *analytic* sources — permanent-magnet bars, finite Halbach arrays, and Biot-Savart coils — and structured-grid *numerical* solvers for the effects analytic superposition cannot represent: quasistatic induced-E and eddy currents, and nonlinear magnetostatics with flux-shaping ferrites, saturable iron, and permanent magnets in two and three dimensions. All are mesh-free (uniform grids) and depend only on NumPy; SciPy and (for the 3D solver) pyamg accelerate the larger linear solves when available. The single-sided NMR depth-profiling workflow that consumes these fields is described alongside the analytic sources it is built on.

9.5.1 Analytic magnet and coil sources

The imaging and motion workflows consume `(B0, B1)` maps; `spin_dynamics.fields.magnetostatics` *generates* them from first-principles magnet and coil models (pure NumPy, mesh-free), so a real device field can drive the spin dynamics instead of a synthetic field. The B0 of a uniformly magnetized rectangular bar

is the closed-form field of the magnetic charge-sheets on its faces—exact for the nearly linear rare-earth magnets used in single-sided NMR—and a soft-iron return yoke is added by the method of images across a $\mu \to \infty$ plane, which enforces $B_{\parallel} = 0$ at the iron surface. The coil B1 is the Biot-Savart field of straight current segments, and its transverse component is the part perpendicular to the local B0. Finite Halbach dipoles are represented by the lowest-order four-rod approximation: four diametrically magnetized cylinders or square rods arranged around a bore. Their finite length is included by a point-dipole cubature over the magnet volume, which is sufficient for bore-field and fringe-field studies but not a substitute for precision magnet design inside the magnet material.

```
import numpy as np
from spin_dynamics.fields.magnetostatics import (
    nmr_mouse_magnets, circular_loop, sample_magnet_field,
)

bars, yoke = nmr_mouse_magnets(gap=0.012, remanence=1.30)
coil = circular_loop((0.0, 0.030, 0.0), 0.008, axis="y")
x = np.linspace(-0.02, 0.02, 121); y = np.linspace(0.021, 0.045, 121)
fm = sample_magnet_field(x, y, bars, yoke_y=yoke, coil_segments=coil)
```

For the canonical NMR-MOUSE (two antiparallel bars on a yoke) this reproduces the device's hallmarks: $|B_0| \sim 0.25$–0.54 T over the gap, a proton Larmor frequency of ~ 5–23 MHz, and a strong static gradient $G \sim 7$–28 T/m. The example `plot_nmr_mouse_fields.py` shows the field, the gradient, the coil B1, and the depth-resolved sensitive slice.

```
from spin_dynamics.fields.magnetostatics import sample_halbach_dipole_field

fm3d = sample_halbach_dipole_field(
    x_axis, y_axis, z_axis,
    center_radius=30e-3,
    rod_shape="square",
    rod_width=16e-3,
    length=80e-3,
    remanence=1.30,
)
```

The 3D Halbach result exposes `b0_vector`, `b0_magnitude`, `b0_gradient`, and `larmor_hz`. The example `plot_halbach_dipole_field.py` plots the mid-plane field, vector direction, uniformity, finite-length axial falloff, and gradient map. These fields also feed the polarization-enhanced NQR transport model in Sec. 7.8.

Single-sided depth profiling. Single-sided NMR profiles a sample by depth: an excitation frequency selects the iso-B0 sensitive slice, and the strong static gradient encodes diffusion. The defining feature is that the spins *move through a spatially structured field*—as a molecule diffuses it samples a changing off-resonance set by the real gradient—which is irreducibly spatial and cannot be reduced to a fixed off-resonance distribution. The single-sided workflow module therefore drives the moving-isochromat engine directly with the magnet's own B0 map: `mouse_depth_profile` sweeps the carrier frequency so the excited signal traces the depth profile (a density gap shows up as a hole) and the echo decay gives the apparent T_2, diffusion-shortened where the gradient is strong. The companion helper `measure_diffusion_at_depth` runs the CPMG twice with identical initial walker positions (diffusion on and off) so the inhomogeneous-field echo envelope cancels in the ratio, and the attenuation rate $k = \frac{1}{12}\gamma^2 G^2 D\, t_E^2$ gives D using the slice's local gradient. This is a stochastic moving-walker Monte-Carlo whose diffusion sensitivity honestly falls off with depth as the gradient weakens; the engine's constant-gradient physics is validated against the exact Carr-Purcell law. The example `plot_nmr_mouse_depth_profile.py` profiles a layered phantom.

9.5.2 Quasistatic E-field and eddy currents

Biot-Savart gives a coil's B-field but not the accompanying E-field — and it is the E-field that drives eddy currents (resistive power loss, B1 perturbation, and gradient field decay) in a conductor. `spin_dynamics.fields.quasistati` adds the magnetoquasistatic induced field $\mathbf{E} = -\partial\mathbf{A}/\partial t - \nabla\phi$: the vector potential `vector_potential` is computed from the same filament segments (its curl reproduces `biot_savart`), the inductive field is `induced_efield`$= -\mathbf{A}_{\text{unit}}\, dI/dt$, and in a finite conductive sample a Neumann charge correction enforces $\mathbf{J}\cdot\hat{\mathbf{n}} = 0$ so `eddy_currents` returns the eddy current density and the deposited power. Coil generators live in `fields.coils` (`solenoid`, `planar_spiral`, `maxwell_pair`, `cylindrical_shield`).

First-order (Born) limitation. This uses the *primary* field only and ignores the eddy currents' reaction field, so it is valid for low conductivity, thin samples, or skin depth $\gg$ sample size; there is no skin-effect shielding and it over-estimates deposition at high conductivity.

The gradient eddy *dynamics* are a separate, self-consistent calculation. `fields.eddy_modes.EddyModes` models a conducting shield as coaxial rings, builds their resistance, inductance and drive coupling, and eigen-decomposes the $\mathbf{L}^{-1}\mathbf{R}$ dynamics into the residual-gradient decay $\sum_k \alpha_k e^{-t/\tau_k}$ — exactly the `eddy_terms` of the M5 `GradientDriverResponse`. So coil + shield geometry now *predicts* the pre-emphasis terms that were hand-specified in Sec. 9.21.3: `EddyModes.to_gradient_driver()` returns a driver the GRAPE PGSE optimizer consumes directly.

```
from spin_dynamics.fields import coils
from spin_dynamics.fields.eddy_modes import EddyModes

drive = coils.maxwell_pair(radius=0.04, axis="z")
radii, centers = coils.cylindrical_shield(radius=0.06, length=0.3, n_rings=16)
eddy = EddyModes(radii, centers, wire_radius=2e-3, resistivity=1.7e-8)
driver = eddy.to_gradient_driver(drive, tau_rl=1e-4)  # -> M5 GradientDriverResponse
```

Sample loading (reflected impedance). By reciprocity the eddy loss couples back into the coil as an added series resistance $R_{\text{refl}}(\omega) = \omega^2 \int \sigma\, |\mathbf{A}_{\text{unit}}|^2\, dV$ (`reflected_resistance`; frequency-independent geometry factor `geometric_loss_integral`). It grows as ω^2, degrades the loaded $Q = \omega L/R$ and raises the Johnson-noise floor by $\sqrt{R_{\text{total}}/R_{\text{coil}}}$; `coil_loading` sweeps it across frequency given `coil_inductance` and $R_{\text{coil}}(f)$.

The examples `plot_rf_coil_efield.py` (solenoid & planar spiral: B1, |E|, and eddy Joule density in a saline cylinder — note the solenoid's on-axis E null), `plot_gradient_eddy_preemphasis.py` (Maxwell pair + shield: eddy spectrum, gradient droop, and the geometry-derived pre-emphasis that flattens it), and `plot_logging_coil_loading.py` (an inside-out NMR well-logging tool — a solenoid coil in a saturated-brine borehole, 0.5–2 MHz — where the brine loading dominates the coil resistance and collapses Q) demonstrate the solvers. `plot_logging_cpmg_multinuclear.py` extends the logging tool to a full Jasper-Jackson magnet (two opposing SmCo cylinders whose low-gradient sweet spot sets the sensitive shell), runs the asymptotic CPMG echo on the estimated (B_0, B_1) maps to visualize the ^{1}H sensitive region, plots the tuned-probe echo and estimates the per-echo SNR against the loading noise, and adds ^{23}Na (spin-$3/2$, treated with the $(2I+1)$ machinery; resonant at the same frequency where B_0 is $\sim$3.8$\times$ higher, with non-optimal flip angles from the proton-calibrated pulses) to estimate its fractional signal contribution. Design note: `docs/field_solvers_quasistatic.md`.

9.5.3 Coil property extraction: inductance, resistance, Q, and self-resonance

The solvers above give a coil's *fields*; a probe designer needs the coil's *lumped RF properties*. `spin_dynamics.fields.coil` extracts them for a single-layer helical round-wire solenoid — `solenoid_properties` returns a `CoilProperties`

with the series inductance (the frequency-independent current-sheet value L_s and the RF value L_{eff}), the AC (skin + proximity) resistance R, the stray capacitance C_p, the unloaded quality factor $Q = \omega L_{\text{eff}}/R$, and the first (quarter-wave) self-resonant frequency f_{res}. These are exactly the numbers the tuned-probe noise model (Sec. 9.5, "Received-Signal Noise") and the matching-network designer consume, so a coil geometry now feeds the SNR budget directly instead of hand-typed L, R, C: `to_probe_params()` returns $\{L, R, C\}$ and `tuning_capacitance()` the tank capacitor $C = 1/(\omega^2 L)$.

```
from spin_dynamics.fields.coil_properties import solenoid_properties

cp = solenoid_properties(diameter=20e-3, length=30e-3, turns=10,
                         wire_diameter=1.0e-3, frequency=10e6)
cp.inductance_effective, cp.ac_resistance, cp.q_factor, cp.self_resonant_frequency
cp.to_probe_params()   # {"L", "R", "C"} -> tuned_probe_output_noise_density
```

The model is the $n = 0$ sheath-helix waveguide method of Corum, with the NIST round-wire self- and mutual-inductance corrections (Knight, Rosa), the Lundin field-non-uniformity correction, and the Medhurst proximity data. Because it is a transmission-line model it stays accurate for electrically long coils, where L_{eff} rises above the quasistatic L_s toward f_{res}. It is a faithful port of Serge Stroobandt's QOIL calculator (GPL-3.0, vendored in `References/`), re-expressed with `scipy.special` Bessel functions and `brentq` root finds; it reproduces QOIL to ~ 6 significant figures. `solenoid_field_inductance` provides an independent cross-check: it sums the Biot-Savart/Neumann partial inductances of the coaxial turns (via `quasistatic.coil_inductance`) and agrees with L_s to within $\sim 10\%$.

Conductor material and temperature. The model is not restricted to copper at room temperature. Presets `ANNEALED_COPPER`, `HARD_DRAWN_COPPER`, `SILVER`, and `ALUMINIUM`—or any custom `ConductorMaterial`—set the resistivity and permeability, and a `temperature` argument evaluates the resistivity (hence R and Q; L, C, and f_{res} are geometric) at the operating temperature. A linear temperature coefficient covers moderate excursions about room temperature; supplying a residual-resistivity ratio ($\text{RRR} = \rho_{293\,\text{K}}/\rho_0$) switches to a Matthiessen model with a residual floor, valid down to cryogenic temperatures. Cooling an OFHC-copper coil ($\text{RRR} = 100$) from 293 to 77 K cuts R roughly in half and nearly doubles the unloaded Q—the cryoprobe gain. The cryogenic estimate is conservative (the linear phonon term over-predicts the ideal resistivity below $\sim$Debye$/3$), so pass a measured resistivity for a precise cold value.

The example `plot_solenoid_coil_properties.py` sweeps L_{eff}, R, and Q versus frequency (overlaying the field-based inductance and marking f_{res}), shows the classic Q-vs-form-factor peak near a cubical coil ($\ell \approx D$), and closes the loop by feeding the extracted (L, R) into `coil_loading` for the loaded Q and noise penalty against a conductive sample. The field-based generalization to arbitrary geometry is the PEEC solver in the next subsection. Design note: `docs/coil_properties.md`.

9.5.4 Arbitrary-geometry coil solver (PEEC): multi-layer, saddle, and gradient coils

The analytic model above is limited to single-layer air-core solenoids. Real RF coils (multi-layer, saddle, birdcage, surface) and gradient coils have arbitrary wire paths, and `spin_dynamics.fields.coil_peec` extracts their lumped RF properties from first principles by the *partial-element equivalent-circuit* (PEEC) method — the mesh-light approach behind FastHenry/FastCap. A conductor is a polyline centreline plus a round cross-section; the cross-section is tiled into K parallel sub-filaments to resolve the current distribution. Because each sub-filament is a simple series chain, the branch system reduces *exactly* to a $K \times K$ chain-impedance system in which sub-filament k is the whole path swept to cross-cell k and $L_{kk'}$ is the Neumann mutual inductance between two such filaments (reusing `quasistatic.vector_potential`). Solving $Z(\omega) = 1/(\mathbf{1}^{\mathsf{T}}(R + j\omega L)^{-1}\mathbf{1})$ lets the current redistribute with frequency, which *is* the skin effect (own

cross-section) and proximity effect (neighbouring turns/legs); a dual thin-wire electrostatic solve on the same geometry gives the self-capacitance and hence the self-resonant frequency.

```
from spin_dynamics.fields.coil_peec import helical_solenoid, coil_properties_peec

coil = helical_solenoid(diameter=20e-3, length=30e-3, turns=10,
                        wire_radius=0.5e-3, n_radial=4, n_angular=8)
p = coil_properties_peec(coil, frequency=5e6)
p.inductance, p.ac_resistance, p.q_factor, p.self_resonant_frequency
p.to_probe_params()   # {"L","R","C"} -> same probe-noise / matching pipeline
```

Build any coil from its wire path with `Conductor` / `conductor_from_segments`; `extract_impedance` sweeps $L(\omega)/R(\omega)$ and `current_distribution` returns the cross-section current for the crowding picture. `PEECCoilProperties` mirrors the universal fields of `CoilProperties` (Sec. 9.5.3) so an arbitrary coil drops into the same probe-noise and matching workflow. The solver is validated against the exact Kelvin-function skin-effect ratio (isolated wire), the Neumann kernels, and the QOIL solenoid — its L, self-capacitance and self-resonance agree with QOIL to a few percent (the physical self-resonance, distinct from QOIL's lumped-equivalent C_p fitting capacitance).

Resolution ceiling. The cross-section must resolve the skin depth: for $a/\delta \lesssim 5$ a few hundred filaments give sub-percent AC-resistance accuracy (convergent to the Kelvin result), but deep skin (high-frequency RF, $a/\delta \gtrsim 15$) is under-resolved at modest cell counts, so R is under-predicted there — inductance, capacitance and self-resonance remain accurate regardless, being geometry-dominated. For a single-layer solenoid in that regime prefer the analytic `solenoid_properties`. A surface-impedance backend, magnetic-core coupling and FMM/H-matrix acceleration are planned follow-ons.

The example `plot_peec_coil_solver.py` shows a two-layer solenoid (L, R, Q, f_{res}, and the cross-section current-crowding map) and a gradient Maxwell pair (L, R, the $\mathrm{d}B_z/\mathrm{d}z$ efficiency from Biot-Savart, and shield eddy modes from `eddy_modes`) — the same solver spanning RF and gradient coils.

Cross-validation against FastHenry and FastCap. `fields.fasthenry_interop` exports a `Conductor` to a FastHenry `.inp` deck and, on a Windows FastHenry2 install (via its COM automation server), runs it and returns $L(f)/R(f)$ for a direct comparison (`run_fasthenry`, `compare_with_fasthenry`; use `cross_section="rect"` to match FastHenry's rectangular conductors exactly). For a 1×1 mm copper bar the resistance agrees with FastHenry to ∼1% at low frequency and the inductance to ∼6% (a GMD-discretization offset versus FastHenry's exact Hoer-Love formulas), with growing divergence in the deep-skin regime where either solver under-resolves. `capacitance_to_ground` matches the analytic isolated-wire capacitance (the single-conductor FastCap result) to ∼10%, and the coil self-capacitance is validated against the Medhurst formula. The example `validate_peec_vs_fasthenry.py` runs the comparison; a live test runs whenever the FastHenry COM server is present and is skipped otherwise. Design note: `docs/coil_peec.md`.

9.5.5 Nonlinear magnetostatics: ferrites and saturable iron

The analytic solvers above superpose permanent-magnet dipoles and Biot-Savart filaments in *free space*; they cannot represent flux-shaping magnetic materials. `spin_dynamics.fields.nonlinear_magnetostatics` adds a structured-grid nonlinear magnetostatic solver for exactly that: a high-permeability RF ferrite that focuses a B_1 field, or saturable iron that shapes and returns a B_0 field. Each nonlinear step's sparse linear system is solved by a SciPy LU factorization (FEMM-style; a NumPy matrix-free conjugate gradient is the fallback when SciPy is absent), with a material's saturation described by a smooth Brauer reluctivity $\nu(B^2) =$

$b_1 + b_2\, e^{b_3 B^2}$ clamped to a physical $\mu_r \geq 1$, and the nonlinear branch tracked by Anderson-accelerated Picard with residual-stall detection. No meshing dependency (structured grid). Two coordinate systems share the material model and solver: `PlanarMagnetostatics` solves the out-of-plane A_z for translationally-invariant (x, y) problems, and `AxisymmetricMagnetostatics` solves the azimuthal A_ϕ for rotationally-symmetric (r, z) problems (solenoid / loop coils with cores). The planar solver is validated against the uniformly-magnetized-cylinder interior field $B_r/2$ and the axisymmetric solver against the analytic solenoid on-axis field.

The example `plot_shim_a_ring_magnet.py` builds a "shim-a-ring": a diametrically-magnetized NdFeB ring inside a concentric soft-iron ring. A uniformly magnetized *tube* produces essentially zero bore field (the outer and inner $\sin\theta$ surface-current sheets cancel); the high-permeability iron ring provides a low-reluctance return path that breaks the cancellation and establishes the transverse bore field, and its *saturation* sets the achievable field — a nonlinear effect analytic superposition cannot capture. The example `plot_logging_ferrite_b1_focusing.py` uses the axisymmetric solver for the complementary RF case, coupled to the same asymptotic-CPMG echo model as `plot_logging_cpmg_multinuclear.py`. It optimizes an inside-out Jasper-Jackson logging coil over two design levers, each swept and traded off against the RF losses:

- **Ferrite core geometry** — the height and thickness of the high-permeability core behind the coil. The efficiency gain is demagnetizing-factor limited (set by the core's aspect ratio, not its permeability), so a short, fat core near the coil beats a coil-length core because it concentrates flux at the sweet-spot midplane.
- **Coil turn spacing** — the axial turn-density profile. Concentrating turns toward the sweet-spot midplane is a strong efficiency lever that the CPMG echo train tolerates despite the B_1 inhomogeneity it creates, so the region-integrated echo signal keeps rising and the optimum is set by practical limits (turn size, voltage), not an interior maximum.

The RF losses are the cost these levers are traded against, not a knob: both levers also boost the coil's coupling to the lossy borehole brine, so the reflected resistance — computed from the solved vector potential over the borehole annulus — rises with the field and offsets much of the naive efficiency gain; the ferrite's own loss-tangent RF loss is included and is negligible next to the brine. The reported figure of merit is therefore the *net* per-echo SNR and the RF power for a fixed sweet-spot B_1, not the naive reciprocity ratio. The material model and nonlinear scheme are adapted from the author's Motor_Design finite-element solver.

9.5.6 Three-dimensional magnetostatics: the reduced scalar potential

The vector-potential solvers above are two-dimensional — symmetry collapses $\mathbf{A}$ to a single scalar component. General 3D has no such symmetry, and the full curl-curl operator $\nabla \times (\nu \nabla \times \mathbf{A}) = \mathbf{J}$ is singular on a nodal grid (a large gradient null space), which forces edge elements or a gauged block system. `ReducedScalarPotential3D` takes the pragmatic route for magnet- and iron-dominated problems: the *reduced scalar potential*. Writing $\mathbf{H} = \mathbf{H}_s - \nabla\psi$ with $\mathbf{H}_s$ the free-space Biot-Savart field of the coils (so Ampere's law holds for any ψ), Gauss's law $\nabla \cdot \mathbf{B} = 0$ with $\mathbf{B} = \mu(|\mathbf{B}|)(\mathbf{H}_s - \nabla\psi) + \mathbf{B}_r$ reduces to a single scalar, symmetric-positive-definite, variable-coefficient Poisson problem,

$$\nabla \cdot (\mu\, \nabla\psi) = \nabla \cdot \big((\mu - \mu_0)\, \mathbf{H}_s + \mathbf{B}_r\big),$$

solved on the same structured grid with the same Anderson-accelerated Picard driver and the same Brauer material model as the 2D solvers. Permanent magnets enter through the remanence $\mathbf{B}_r$, soft or saturable iron through μ, and coils through $\mathbf{H}_s$; box, sphere, and cylinder region masks paint the geometry.

```
import numpy as np
from spin_dynamics.fields.scalar_potential_3d import ReducedScalarPotential3D
from spin_dynamics.fields.nonlinear_magnetostatics import ndfeb, linear_material

g = np.linspace(-0.09, 0.09, 121)
prob = ReducedScalarPotential3D(g, g, g)
# Two anti-parallel NdFeB bars on a soft-iron yoke (NMR-MOUSE).
prob.add_material(prob.box((-0.02, 0.02), (-0.032, 0.0), (-0.0325, -0.0065)),
                  ndfeb(1.2), remanence_direction=(0, 1, 0))
prob.add_material(prob.box((-0.02, 0.02), (-0.032, 0.0), (0.0065, 0.0325)),
                  ndfeb(1.2), remanence_direction=(0, -1, 0))
prob.add_material(prob.box((-0.02, 0.02), (-0.047, -0.032), (-0.0325, 0.0325)),
                  linear_material(1000.0))
sol = prob.solve(linear_solver="auto")         # AMG on the large grid
bx, by, bz = sol.sample([0.0], [0.0025], [0.0])  # field 2.5 mm above the gap
```

Two implementation points matter. The source term uses $(\mu-\mu_0)\mathbf{H}_s$, not $\mu\mathbf{H}_s$: because $\mathbf{H}_s$ is divergence-free the vacuum part cancels analytically, which makes free space exact ($\psi \equiv 0$, $\mathbf{B} = \mu_0\mathbf{H}_s$) and removes the grid-erratic discrete divergence of the near-singular sampled coil field at the wire. And μ is evaluated as a *stateless* function of ψ — inverting the B-H curve for $\mu(|\mathbf{H}|)$, since $\mathbf{H} = \mathbf{H}_s - \nabla\psi$ is available from the unknown alone — which keeps the $\mu \leftrightarrow \psi$ fixed point inside the Anderson mixing; carrying μ as hidden state fails to converge under partial saturation.

Accuracy and the linear solve. Inside high-permeability material the physical $\mathbf{H} = \mathbf{H}_s - \nabla\psi$ is a near-cancellation of two large quantities, so the field error scales as $\mu_r\,(h/L)^2$; the solver is trustworthy for $\mu_r \lesssim 100$–300, and excellent for permanent magnets ($\mu_r \approx 1.05$). The classical Simkin-Trowbridge total/reduced split does *not* help on this centered nodal grid: the two formulations are an exact linear variable shift apart and yield the identical discrete solution (verified to machine precision), so the residual error is discretization, not floating-point cancellation. The effective cure is grid refinement, made practical by the optional AMG solver. `solve(linear_solver="auto")` uses an exact sparse LU for small grids and switches to pyamg's AMG-preconditioned CG above $\sim 20^3$ unknowns; its grid-independent iteration count scales to $> 10^6$ unknowns (a 121^3 grid solves in seconds, where sparse-LU fill-in cannot reach 64^3). The design note `docs/reduced_scalar_potential_3d.md` gives the full quantification and the equivalence proof.

The NMR-MOUSE. The example `plot_nmr_mouse_3d.py` is the flagship application: the Blümich NMR-MOUSE, two anti-parallel NdFeB bars on a soft-iron yoke with a surface coil in the gap. Unlike the analytic treatment of Sec. 9.5.1, which adds the yoke by the method of images across a $\mu \to \infty$ plane, the solver treats the magnets and the *finite* iron yoke together, so the yoke's low-reluctance return path is resolved directly and is found to boost the sensor field by $\sim 10\%$ — the effect that motivates the yoke and that free-space dipole superposition cannot capture. With the paper's geometry (26 mm magnets, 13 mm gap, 15 mm yoke) the simulation reproduces the device's hallmarks: $B_0 \approx 0.43$ T at the 2.5 mm sensor (~ 18 MHz ^{1}H; paper 0.41 T, 17.5 MHz), a ~ 14 T/m penetration gradient (paper 10–14), and a near-quadratic lateral $B_{0z}(z)$ across the gap. A useful correctness check for any new geometry: the solver honours the problem's exact symmetries (here B_{0z} even in z, B_{0y} odd) to machine precision.

Driving the workflow with the solved field. The solved field is wired into the single-sided workflow (Sec. 9.5.1) exactly as the analytic magnet is. `ScalarPotentialSolution.to_magnet_field_maps(...)` samples the solved B_0 (and an optional coil B_1) onto a $(\text{lateral}, \text{depth})$ plane and returns the same

`MagnetFieldMaps` container the analytic `sample_magnet_field` produces, so a 3-D solve is a drop-in field source. `workflows.SolvedMouseField` wraps a solution, and `simulate_mouse_cpmg/mouse_depth_profile/resonant` now accept either bars (analytic, the unchanged default) or such a field source. The example `plot_nmr_mouse_depth_prof` drives the moving-walker depth-profiling measurement from the 3-D MOUSE solve — magnets and the finite iron yoke — and recovers a buried density gap as a hole in the profile, a higher-fidelity counterpart to the analytic image-yoke profile of Sec. 9.5.1. It also separates the two depth factors: the walker signal is the *intrinsic* slice response, which rises with depth (the slice thickness is pulse-bandwidth/gradient, and the weakening B_0 gradient thickens the slice – a larger excited sample volume), while the *measured* signal weights it by the geometric detection sensitivity $S(d) \sim B_0^2 B_1^2$ (Curie polarization, reciprocity reception, and transmit/receive B_1) computed from the solved fields; S falls $\sim 100\times$ over the first centimetre (surface-coil B_1 dominated), so the measured profile decays with depth as a real single-sided measurement does.

9.6 Received-Signal Noise

Noise is opt-in and output-referred. High-level workflows accept a `NoiseSpec`, a mapping, or a scalar white-noise standard deviation:

```
from spin_dynamics.noise import NoiseSpec
from spin_dynamics.workflows import run_tuned_cpmg

noisy = run_tuned_cpmg(
    numpts=101,
    noise=NoiseSpec(model="probe", target_snr=25.0, seed=3),
)
```

`model="white"` adds flat complex receiver noise. `model="probe"` uses the probe output noise density where the workflow can supply it, which is important when the received spectrum is filtered by a tuned, untuned, or matched resonator. Result objects preserve clean deterministic fields and add noisy fields plus `NoiseMetadata` so repeated SNR studies remain reproducible.

9.7 Non-Inductive Detection: SQUID and OPM Magnetometers

The probe noise model above describes a conventional inductive (Faraday) coil, whose observable is $\mathrm{d}\Phi/\mathrm{d}t$ and whose noise is the coil Johnson noise. At ultra-low field, and for NQR, the enabling detectors are instead *field* sensors—SQUIDs and optically pumped (atomic) magnetometers (OPMs)—which respond to B itself with a frequency-flat noise floor. The `spin_dynamics.detection` package models all three on one axis by referring signal and noise to *magnetic field at the sensor* (T, $\mathrm{T}/\sqrt{\mathrm{Hz}}$). A detector is then a transfer shape $H(f)$ plus a field-referred noise PSD $N^2(f)$, and the matched-filter detected SNR of a field-at-sensor spectrum $S(f)$ is detector-agnostic:

$$\mathrm{SNR}^2 = \int \frac{|S(f)|^2}{N^2(f)}\,\mathrm{d}f.$$

The field $S(f)$ that the precessing sample presents at the pickup is the same for every detector; all detector differences live in N^2. A Faraday coil's field-referred noise *rises* as $1/f$ toward low frequency (its responsivity $H \propto f$ shrinks), so $\int |S|^2/N^2$ reproduces the familiar B_0^2 signal collapse automatically, whereas an untuned SQUID flux transformer has a *flat* N^2. Referring the existing inductive probe density to field via $N^2_{\mathrm{field}} = N^2_{\mathrm{out}}/|H|^2$ leaves the matched-filter SNR unchanged, so the layer reproduces the tuned/ untuned/matched probe numbers exactly (`InductiveCoilDetector`).

```
import numpy as np
from spin_dynamics.detection import (
    SQUIDMagnetometer, IdealFaradayCoil, detected_field_snr,
)

squid   = SQUIDMagnetometer.berkeley_2007()          # flat 1 fT/rtHz floor
faraday = IdealFaradayCoil(field_noise_T_per_rtHz_ref=1e-15, f_ref_hz=1e6)

f = np.linspace(-500.0, 500.0, 2001)                 # a line at baseband
s = (5.0/np.pi) / (f**2 + 5.0**2)                    # unit-area Lorentzian
snr_squid   = detected_field_snr(s, f, squid).snr    # flat floor
snr_faraday = detected_field_snr(s, f, faraday).snr  # 1/f floor
```

Following Clarke, Hatridge & Mößle (*Annu. Rev. Biomed. Eng.* **9**, 389, 2007), the untuned SQUID floor sits near $1\ \mathrm{fT}/\sqrt{\mathrm{Hz}}$ and is frequency-independent, so it crosses the coil's rising $1/f$ floor at a few MHz: below the crossover the SQUID wins, and with a prepolarized sample (moment set by B_p, not B_0) the detected SNR *grows* as B_0 falls ($\mathrm{SNR} \sim B_0^{-1/2}$ once the line has narrowed to the homogeneous T_2 limit), while the Faraday SNR falls as $B_0^{+1/2}$. The example `examples/plot_squid_ulf_crossover.py` reproduces the noise-floor crossover, the prepolarized SNR-vs-B_0 scalings, and the B_0-proportional line narrowing.

Optically pumped (atomic) magnetometers (`OPMMagnetometer`) are field sensors like SQUIDs but reach sub-$\mathrm{fT}/\sqrt{\mathrm{Hz}}$ sensitivity without cryogenics, at the cost of a finite atomic-response bandwidth. The vapour-cell spins act as a Lorentzian filter of half-width Δf, so the field-referred noise is flat on resonance and rises as the response rolls off, $N^2(f) = S_0^2\,[1+((f-f_c)/\Delta f)^2]$. A `"serf"` (zero-field) OPM is a low-pass centred at DC—best for zero/ultra-low-field NMR but rolled off, and useless, at MHz; an `"rf"` (Mx) OPM tunes its bias field to place a band-pass on the line f_c, reaching $\sim 0.24\ \mathrm{fT}/\sqrt{\mathrm{Hz}}$ at $423\ \mathrm{kHz}$ for the $^{14}\mathrm{N}$ NQR of ammonium nitrate (Savukov et al.; US 7,521,928). Modelling the roll-off is essential: a SERF OPM's field noise at $423\ \mathrm{kHz}$ is $\sim f/\Delta f$ (thousands of times) above its DC floor, and the RF-OPM's bandwidth must cover the NQR linewidth or the detected SNR degrades. The example `examples/plot_opm_nqr_detection.py` compares an RF-OPM, a SERF OPM, a SQUID, and a coil on a $^{14}\mathrm{N}$ line and shows the bandwidth-matching penalty.

Where a detector sets *how quiet* the sensor is, a *pickup geometry* (`Gradiometer`) sets *where* it is sensitive. SQUID systems commonly wind the flux transformer as an axial gradiometer—coaxial loops with alternating turns—so a uniform ambient field (and, at second order, a uniform gradient) cancels while a nearby sample still couples. By reciprocity a pickup's receive sensitivity to a source at r equals the field it would transmit there per unit current, so the net sensitivity is the signed sum of loop fields $S(r) = \sum_i w_i B_{\mathrm{loop},i}(r)$, evaluated by reusing the `fields.magnetostatics` loop-field routines. A dipole field and its first and second axial derivatives fall off as $1/r^3$, $1/r^4$, $1/r^5$, so an order-n gradiometer's distant-source sensitivity falls as $1/r^{3+n}$ and it rejects uniform ($n \geq 1$) and first-gradient ($n \geq 2$) ambient noise—the basis of the $\sim 1000\times$ common-mode rejection in Clarke's microtesla MRI. A pickup attaches to a field sensor as `SQUIDMagnetometer(..., pickup=g)` (or `OPMMagnetometer(..., pickup=g)`), and `detected_field_snr_spatial` then routes a distributed sample through the pickup coupling for the signal while ambient sources (`AmbientFieldSource`) couple through the *same* pickup for the noise—so a uniform ambient field (coupling 1 for a magnetometer, 0 for a balanced gradiometer) is rejected exactly as a real gradiometer would. The example `examples/plot_gradiometer_sensitivity.py` maps the reciprocal (excited) region for a magnetometer and first/second-order gradiometers, shows how winding order localizes it, and prints the SQUID detected SNR with and without a uniform ambient field (the magnetometer loses $\sim 1000\times$, the gradiometers are unaffected). Detector-aware GRAPE objectives follow in a later increment (see `docs/non_inductive_detection.md`).

9.8 Diffusion

```
from spin_dynamics.workflows import run_matched_diffusion_q_sweep

sweep = run_matched_diffusion_q_sweep(
    q_values=[20, 50],
    num_echoes=3,
    numpts=21,
    dz=50e-6,
    auto_refine_grid=True,
)
```

The compact matched-probe diffusion workflow is solver-validated through $Q = 2000$. Higher-Q cases remain a stiffness target for the current NumPy matched-probe transient solver.

Matched diffusion CPMG also accepts `absolute_phase`. In this combined path, the diffusion-encoding π pulse and the CPMG refocusing pulses are solved at their laboratory-frame RF phase, while the free-precession intervals retain the diffusion attenuation:

```
from spin_dynamics.workflows import run_matched_diffusion_cpmg

combined = run_matched_diffusion_cpmg(
    num_echoes=16,
    echo_spacing_seconds=1.0e-3,
    diffusion_time=1.0e-3,
    q_value=50,
    absolute_phase={
        "rf_frequency_hz": 0.25 / 1.0e-3,
        "phase_bins": 16,
    },
)

metadata = combined.absolute_phase
encoding_phase = metadata.encoding_absolute_phase_rad
refocus_phases = metadata.refocus_absolute_phase_rad
```

`metadata.refocus_*` describes the echo-train refocusing pulses. `metadata.encoding_*` describes the diffusion-encoding π pulse, and `pulse_*` fields provide the full RF event list.

The comparison plotting example toggles diffusion and absolute phase independently:

```
python examples\plot_diffusion_absolute_phase_compare.py
python examples\plot_tuned_diffusion_absolute_phase_compare.py
```

The diffusion plotting examples use a narrow default `-dz-um` and enable automatic offset-grid refinement so the displayed decay is not an artifact of discrete-grid rephasing. They print the effective number of offsets after any refinement. Use `-no-auto-refine-grid` only when intentionally testing the rephasing guard. The matched diffusion comparison also prints the matched-probe absolute-phase residual and warns when it is below the plotting tolerance. The current matched pulse-shape solver is often nearly phase-invariant; in that case the plot is a diagnostic limitation rather than evidence for a measurable combined effect. The tuned-probe comparison uses the same diffusion kernel with tuned pulse-shape solves and usually shows a much larger residual, making it the better contrast example for phase-sensitive probe transients.

9.8.1 PGSE and PGSE-Prepared CPMG

For Stejskal-Tanner pulsed-gradient diffusion encoding, use the PGSE workflow helpers. The deterministic backend tracks the effective gradient moment and therefore evaluates

$$b = \int q(t)^2 \, \mathrm{d}t, \qquad q(t) = \gamma \int_0^t G_{\mathrm{eff}}(t') \, \mathrm{d}t'.$$

For a rectangular PGSE pair this reduces to

$$b = (\gamma G \delta)^2 \left(\Delta - \frac{\delta}{3} \right),$$

where δ is the lobe duration and Δ is the leading-edge separation between the two gradient lobes. The physical lobes are scheduled with the same polarity around the refocusing pulse; the 180-degree pulse flips the coherence-frame sign, so stationary spins refocus while diffusive motion attenuates the signal.

```
from spin_dynamics.workflows import run_pgse_moment

pgse = run_pgse_moment(
    num_echoes=8,
    gradient_amplitude=0.12,      # T/m
    gradient_duration=2.5e-3,     # delta
    diffusion_time=28.0e-3,       # Delta
    diffusion_coefficient=1.2e-9,
    first_echo_time_seconds=56e-3,
    echo_spacing_seconds=8e-3,
    t2_seconds=80e-3,
)
```

`num_echoes=1` is the ordinary spin-echo PGSE acquisition. Larger echo counts represent a PGSE preparation followed by a compact CPMG acquisition. For spatially explicit studies, `run_pgse_walkers` uses the moving-isochromat sequence driver and seeded Brownian walkers. It is slower and stochastic, but can be extended to inhomogeneous field maps, boundaries, and advection:

```
from spin_dynamics.workflows import run_pgse_walkers

walkers = run_pgse_walkers(
    gradient_amplitude=0.05,
    gradient_duration=2.0e-3,
    diffusion_time=16.0e-3,
    diffusion_coefficient=2.3e-9,
    walkers_per_cell=12000,
    seed=123,
)
```

The PGSE tests compare the moment backend to the analytical Stejskal-Tanner formula, and compare the walker backend against the same attenuation through a zero-diffusion walker reference.

9.8.2 Restricted Diffusion and Diffusive Diffraction

Because the walker backend integrates explicit displacements, it can model diffusion restricted by hard walls or exchanged through semi-permeable membranes—physics the closed-form moment backend cannot express. The `boundary` argument of `run_pgse_walkers` accepts the rectangular-box modes `"reflect"`,

`"periodic"`, and `"clip"`, or any callable mapping particle positions to confined positions. The helpers `make_circular_reflector(center, radius)` and `make_elliptical_reflector(center, semi_axes)` supply reflecting circular and elliptical pore walls.

For a slab pore of width L along the gradient axis, confining the walkers makes the echo attenuation bend above the free Stejskal-Tanner line $E = \exp(-bD)$: once the diffusion length $\sqrt{2D\Delta}$ approaches L the spins can no longer fully dephase, and the apparent diffusion coefficient $D_{\mathrm{app}} = -\ln(E)/b$ falls below the bulk value D as the diffusion time grows. The example `plot_pgse_restricted_diffusion.py` sweeps several pore widths and diffusion times to show both signatures.

9.8.3 Large 3D Porous-Rock Walker Challenge

The example `porous_rock_cpmg_walkers.py` is a heavier end-to-end stress test for the moving-walker machinery and the optional JAX/Numba walker backends. It builds a cylindrical rock-core analogue with a multimodal pore-body size distribution, connected throat voxels, constriction-dependent tortuosity, surface-relaxation T_2, and a susceptibility-like internal B_0 map. The sequence is a CPMG echo train with millions of walkers moving through the voxel pore space, rejecting proposed steps into solid voxels.

Before the expensive propagation begins, the script prints geometry-only estimates for the expected distributions:

$$\frac{1}{T_2(\mathbf{r})} = \frac{1}{T_{2,\mathrm{bulk}}} + \frac{3\rho_2}{a_{\mathrm{relax}}(\mathbf{r})}, \qquad D(\mathbf{r}) = \frac{D_0}{\tau(\mathbf{r})}.$$

Here a_{relax} is reduced near walls and in throats, while τ increases in constricted regions. The printed percentiles are a useful analytical check on the final D-T_2 plot: a healthy large run should not collapse to a single D, T_2 point.

The default challenge is intended for CUDA-enabled JAX and writes a four-panel verification figure containing a sampled 3D pore structure, the pore-weighted D-T_2 histogram, the CPMG decay, and the input pore-size distribution:

```
XLA_PYTHON_CLIENT_PREALLOCATE=false python examples/porous_rock_cpmg_walkers.py \
  --backend jax \
  --plot-output results/porous_rock_challenge.png \
  --output results/porous_rock_challenge.npz
```

Use `-estimate-only` to inspect the pore-space, D, T_2, and rough runtime estimates without launching the walker propagation. Use `-grid`, `-z-cells`, `-pores`, `-walkers-per-voxel`, `-num-echoes`, and `-substeps` to scale the calculation down for smoke tests or up for production benchmarks.

In a closed two-dimensional pore the restricted signal becomes non-monotonic. In the narrow-pulse, long-mixing limit ($\delta \ll a^2/D \ll \Delta$) the normalized echo approaches the squared magnitude of the pore form factor, the *diffusive diffraction* regime of q-space NMR. For a uniform disc of radius a,

$$E(q) \;\longrightarrow\; \left| \frac{2\,J_1(q_{\mathrm{ang}} a)}{q_{\mathrm{ang}} a} \right|^2,$$

with minima at the zeros of J_1, i.e. $q_{\mathrm{ang}} a = 3.83,\ 7.02,\ 10.17, \ldots$. Note the factor-of-$2\pi$ ambiguity in the definition of q: the *angular* wavenumber is $q_{\mathrm{ang}} = \gamma G \delta$ (units $\mathrm{rad/m}$, with $b = q_{\mathrm{ang}}^2(\Delta - \delta/3)$), while the *reciprocal-space* (Callaghan) convention uses $q = \gamma G \delta/(2\pi)$ (units $1/\mathrm{m}$, encoding kernel $\exp(i2\pi q x)$), for which the first disc feature sits near $qa \sim 1$.

```
import numpy as np
from spin_dynamics.motion import make_circular_reflector
```

```
from spin_dynamics.workflows import run_pgse_walkers

radius = 5.0e-6
axis = np.linspace(-radius, radius, 21)
xx, zz = np.meshgrid(axis, axis, indexing="ij")
rho = (xx**2 + zz**2 <= radius**2).astype(float)  # uniform disc

walkers = run_pgse_walkers(
    rho=rho, x_axis=axis, z_axis=axis,
    gradient_amplitude=2.0,        # large G reaches the q-space regime
    gradient_duration=0.4e-3,      # short delta (narrow-pulse limit)
    diffusion_time=80.0e-3,        # long Delta >> a^2 / D
    diffusion_coefficient=2.3e-9,
    boundary=make_circular_reflector((0.0, 0.0), radius),
    walkers_per_cell=28, substeps_per_interval=80, seed=2026,
)
```

The example `plot_pgse_circular_pore_diffraction.py` plots the resulting fringes against the Bessel form factor. It offers two methods. The default `walker-sweep` re-runs the confined PGSE simulation at every gradient and so captures finite-pulse blurring of the minima. The optional `-method sgp-propagator` exploits the fact that the walker trajectories are independent of q: it runs the confined diffusion once, records each walker's net displacement Δx over the diffusion time, and evaluates $E(q) = \left|\langle \exp(i q_{\mathrm{ang}} \Delta x) \rangle\right|$ for all q in a single vectorized sum. This is the average-propagator (q-space) method; it is tens of times faster and reproduces the same diffraction pattern in the $\delta \to 0$ limit. Keep the per-substep hop $\sqrt{2D\,\mathrm{d}t}$ well below the pore size so reflection stays accurate.

The inverse problem is exposed separately in `spin_dynamics.workflows.qspace`. Use `qspace_axes_from_real_space` and `pore_form_factor_from_density` to build ideal q-space responses for test phantoms. `reconstruct_qspace_image` directly inverts a complex form factor into pore density, but a conventional diffraction intensity $|F(q)|^2$ inverts to the Patterson/autocorrelation image, not a unique pore. When a loose support is known, `phase_retrieve_qspace_magnitude` performs support-constrained, non-negative HIO/error-reduction phase retrieval to estimate the pore shape from magnitude-only q-space data. The example `plot_pgse_qspace_pore_imaging.p` shows all three panels: the diffraction response, the autocorrelation, and the recovered pore estimate. It also adds a finite-SNR branch by perturbing the normalized q-space intensity with additive Gaussian noise (`-snr` is the zero-q peak divided by the noise standard deviation) and running the same constrained retrieval on the noisy data.

For slow exchange between two compartments, use `make_semipermeable_plane`. The membrane is an internal line $x = x_m$ or $z = z_m$ inside the rectangular bounds. A walker that crosses the line transmits with probability

$$P_{\mathrm{transmit}} = 1 - \exp(-k\,\Delta t),$$

where k is `exchange_rate`; otherwise the walker reflects from the membrane. This rate form stays consistent when `substeps_per_interval` is changed.

```
import numpy as np
from spin_dynamics.motion import make_semipermeable_plane
from spin_dynamics.workflows import run_pgse_walkers

x = np.linspace(-10e-6, 10e-6, 41)
z = np.array([-0.5e-6, 0.5e-6])
rho = np.ones((x.size, z.size))
membrane = make_semipermeable_plane(
    0.0,
```

```
    exchange_rate=25.0,  # s^-1 in this seconds-based workflow
    axis="x",
)

walkers = run_pgse_walkers(
    rho=rho, x_axis=x, z_axis=z,
    gradient_amplitude=0.2, gradient_duration=1.0e-3,
    diffusion_time=40.0e-3,
    diffusion_coefficient=2.0e-9,
    boundary=membrane,
    walkers_per_cell=64, substeps_per_interval=16, seed=2026, jitter=True,
)
```

Use `exchange_rate=0` for an impermeable internal wall and `np.inf` for a freely transmitting interface. The outer rectangular boundary still defaults to reflection, so the exchanging compartments remain confined by the sample box.

9.8.4 Stimulated-Echo PGSE (PGSTE)

`run_pgste_walkers` splits the diffusion encoding across three 90-degree pulses, $90-G(\delta)-90-[\text{storage}]-90-G(\delta)-\text{echo}$. The second pulse stores one quadrature of the encoded magnetization along the longitudinal axis, so during the (long) storage interval it decays with T_1 rather than T_2. This is why PGSTE is the method of choice for slow diffusion, large pores, and short-T_2 samples such as porous media, low field, and internal-gradient regimes. A spoiler gradient applied during storage crushes the residual transverse coherences. The workflow records an effective selected-pathway phase table and suppresses equilibrium recovery into a contaminating FID, so the surviving stimulated echo follows the Stejskal-Tanner attenuation at *half* the spin-echo amplitude:

$$E(b) \;\longrightarrow\; \tfrac{1}{2}\exp\!\left(-\frac{T_s}{T_1}\right)\exp(-bD),$$

where T_s is the storage interval. The argument `diffusion_time` is the leading-edge separation of the two gradient lobes, so $b=(\gamma G\delta)^2(\Delta-\delta/3)$ still holds, and $T_s=\Delta-\delta-2\,\texttt{encode\_delay}-2\,\texttt{excitation\_duration}$.

```
import numpy as np
from spin_dynamics.workflows import run_pgste_walkers

axis = np.linspace(-1.0e-3, 1.0e-3, 64)   # wide slab (see note below)
rho = np.ones((axis.size, 2))

ste = run_pgste_walkers(
    rho=rho, x_axis=axis, z_axis=np.array([-1e-6, 1e-6]),
    gradient_amplitude=0.1, gradient_duration=1.0e-3,  # delta
    diffusion_time=60.0e-3,            # Delta = lobe leading-edge separation
    diffusion_coefficient=2.3e-9,
    t1_seconds=0.4, t2_seconds=15.0e-3,  # storage decays with T1, not T2
    walkers_per_cell=64, seed=2026, jitter=True,
)
ste.phase_cycle.name   # "pgste_stimulated_echo"
```

Use a sample wide compared with the gradient phase wavelength $\pi/(\gamma G\delta)$ so the unwanted stimulated anti-echo dephases spatially; with diffusion present it is suppressed further. The example `plot_pgste_stimulated_echo.py` verifies the half-amplitude Stejskal-Tanner attenuation and contrasts the T_2-limited spin echo with the T_1-limited stimulated echo as the diffusion time grows.

9.8.5 Double Diffusion Encoding (DDE / Double-PGSE)

`run_dde_walkers` applies two refocused PGSE blocks separated by a mixing time, $90 - [G_1\,\text{block}] - \text{mixing} - [G_2\,\text{block}] - \text{echo}$, encoding displacement along `angle1` and `angle2`. Single-PGSE measures the diffusion tensor, so in an orientationally disordered sample of anisotropic compartments it sees only the isotropic orientation average. Sweeping the angle ψ between the two encoding directions instead produces a $\cos 2\psi$ modulation of the echo whose amplitude reports the *microscopic* anisotropy of the local geometry; because it depends only on the relative angle, it survives powder averaging and reveals shape anisotropy that single-PGSE cannot. Paired with `make_elliptical_reflector`, an elliptical pore shows the $\cos 2\psi$ term while an equal-area circular pore does not. The $\cos 2\psi$ term is a higher-order (q^4) effect, so strong diffusion weighting is needed to resolve it in the powder average.

```
import numpy as np
from spin_dynamics.motion import make_elliptical_reflector
from spin_dynamics.workflows import run_dde_walkers

semi_axes = (8.0e-6, 3.0e-6)
x = np.linspace(-semi_axes[0], semi_axes[0], 15)
z = np.linspace(-semi_axes[1], semi_axes[1], 15)
xx, zz = np.meshgrid(x, z, indexing="ij")
rho = ((xx / semi_axes[0]) ** 2 + (zz / semi_axes[1]) ** 2 <= 1.0).astype(float)

dde = run_dde_walkers(
    rho=rho, x_axis=x, z_axis=z,
    gradient_amplitude=1.0, gradient_duration=1.0e-3,
    diffusion_time=12.0e-3, mixing_time=1.0e-3,
    angle1=0.0, angle2=np.pi / 2,          # vary to trace E(psi)
    diffusion_coefficient=2.0e-9,
    boundary=make_elliptical_reflector((0.0, 0.0), semi_axes),
    walkers_per_cell=64, substeps_per_interval=10, seed=2026, jitter=True,
)
```

The example `plot_pgse_double_encoding_elliptical_pore.py` traces $E(\psi)$ for the ellipse and the equal-area circle, both at a single orientation and powder-averaged, and reports the fitted $\cos 2\psi$ amplitude.

9.8.6 Oscillating-Gradient Spin Echo (OGSE)

PGSE varies the diffusion time Δ; `run_ogse_walkers` instead varies the oscillation frequency of a cosine-modulated gradient. Each diffusion-encoding lobe is a waveform $G\cos(2\pi f t)$ of `num_periods` whole periods, applied symmetrically around the refocusing pulse, $90 - \cos$ lobe $- 180 - \cos$ lobe $-$ echo. Because each lobe spans an integer number of periods, its zeroth gradient moment vanishes and stationary spins refocus. The encoding power is concentrated at the angular frequency $\omega = 2\pi f$, so sweeping `oscillation_frequency` maps the diffusion spectrum $D(\omega)$ and reaches the short-diffusion-time regime that ordinary PGSE cannot. The waveform is discretized into `samples_per_period` constant-gradient steps, and the b-value follows the cosine spectrum $b = (\gamma G/\omega)^2\, N/f$, which falls steeply with frequency.

```
import numpy as np
from spin_dynamics.motion import make_motion_field_maps_2d
from spin_dynamics.workflows import run_ogse_walkers

x = np.linspace(-2.5e-6, 2.5e-6, 15)          # 5 um reflecting slab along x
z = np.array([-0.5e-6, 0.5e-6])
rho = np.ones((x.size, z.size))
```

```
ogse = run_ogse_walkers(
    rho=rho, x_axis=x, z_axis=z,
    fields=make_motion_field_maps_2d(x, z),  # walls = the slab
    gradient_amplitude=0.5, oscillation_frequency=200.0,  # sweep this
    num_periods=2, diffusion_coefficient=2.0e-9,
    walkers_per_cell=160, substeps_per_interval=6, seed=2026, jitter=True,
)
d_app = -np.log(abs(ogse.signal[0]) / rho.sum()) / ogse.b_value  # D(omega)
```

In restricted geometry D_{app} rises from the long-time (tortuosity) value toward the bulk value as the frequency increases, and a smaller pore stays restricted to higher frequency (transition $f_c \sim D/L^2$). The example `plot_ogse_frequency_diffusion.py` sweeps the frequency for free diffusion and two reflecting slab widths.

9.9 Moving Isochromats

The motion layer treats inhomogeneity in the same spirit as static isochromats, but the samples move through two-dimensional B0, transmit-B1, and receive-B1 maps:

```
from spin_dynamics.motion import make_motion_field_maps_2d,
    initialize_ensemble_from_density
from spin_dynamics.sequences.motion import (
    run_motion_cpmg_sequence,
    run_motion_udd_sequence,
)

fields = make_motion_field_maps_2d([-1, 0, 1], [-1, 0, 1])
ensemble = initialize_ensemble_from_density([[0, 1, 0], [1, 2, 1], [0, 1, 0]])
motion = run_motion_cpmg_sequence(
    ensemble,
    fields,
    num_echoes=4,
    echo_spacing=1e-3,
    excitation_duration=20e-6,
    refocusing_duration=40e-6,
    gradient=(0.0, 0.1),
)

udd = run_motion_udd_sequence(
    ensemble,
    fields,
    num_pulses=4,
    total_duration=4e-3,
    excitation_duration=20e-6,
    refocusing_duration=40e-6,
    gradient=(0.0, 0.1),
)
```

Lower-level helpers such as `free_precession_with_motion_step` are useful for custom trajectories. The sequence helpers split long intervals into substeps so field maps are sampled along the particle path rather than only at the beginning of an interval.

The same machinery handles deterministic flow as well as diffusion. The example `plot_cpmg_pipe_flow.py` seeds an axisymmetric cylindrical pipe, assigns a laminar Poiseuille velocity profile, computes upstream polarization from residence time in a static magnet, and then runs a downstream CPMG train through spatial transmit/receive coil maps. Increasing the mean flow velocity lowers the initial longitudinal magnetization and adds echo-train dephasing as spins move through the B0 and B1 maps during the sequence.

The engine is not limited to two spatial axes. `make_motion_field_maps` and `initialize_ensemble_from_domain` build one-, two-, or three-dimensional `MotionFieldMaps` and walker ensembles over a `SpatialDomain`, and a `MotionSequenceStep` accepts a gradient vector of matching length, so the same precession/advection/diffusion machinery runs in 3D. This is what the true-3D slice-selective imaging workflow uses; see *Slice-Selective Excitation and 3D Multi-Slice Imaging*.

The motion sequence runners also accept `detuning_waveform`. The callable may return a scalar B0 offset for the whole ensemble or one value per particle. This makes it possible to model slow fluctuating gradients or local bath fields:

```
import numpy as np

fluctuating_gradient = lambda time, positions: (
    1500.0 * positions[:, 0] * np.cos(2 * np.pi * 0.35 * time)
)

udd = run_motion_udd_sequence(
    ensemble,
    fields,
    num_pulses=8,
    total_duration=0.72,
    excitation_duration=0.25e-3,
    refocusing_duration=0.5e-3,
    t1=5.0,
    t2=1.0,
    detuning_waveform=fluctuating_gradient,
    substeps_per_interval=8,
)
```

This is the more realistic regime where UDD can outperform evenly spaced CPMG: the noise is low-frequency and correlated across time, but varies across the sample so residual phase spread reduces the received signal. In contrast, ordinary unrestricted diffusion in a static gradient usually gives very similar CPMG and UDD attenuation.

9.10 Time-Varying Fields

```
from spin_dynamics.workflows import (
    sinusoidal_field_waveform,
    run_ideal_time_varying_amplitude_sweep,
)

waveform = sinusoidal_field_waveform(num_echoes=16)
sweep = run_ideal_time_varying_amplitude_sweep(
    amplitudes=[0, 0.5, 1.0, 2.0],
    waveform=waveform,
    numpts=101,
    auto_refine_grid=True,
```

```
)
```

9.11 WURST

```
from spin_dynamics.workflows import (
    run_ideal_wurst_inversion,
    run_matched_wurst_cpmg,
)

ideal = run_ideal_wurst_inversion(numpts=101)
matched = run_matched_wurst_cpmg(num_echoes=2, numpts=51, num_steps=64)
```

9.12 Nonresonant Field-Reversal Echoes

The `spin_dynamics.nonresonant` package models *nonresonant* NMR, in which nuclear magnetization is manipulated without RF — by suddenly switching and adiabatically rotating applied magnetic fields (Brill *et al.*, *Science* **297**, 369, 2002). Because the manipulation is a field reversal rather than a resonant pulse, it excites spins with an efficiency independent of the Larmor frequency (useful for ex-situ sensors in grossly inhomogeneous fields), but a non-reversible background field — typically Earth's field — survives the reversals and dephases the echoes within a few milliseconds. A non-interacting spin-1/2 is a classical Bloch 3-vector, so the engine propagates an ensemble of isochromats by exact rotations about the instantaneous local field.

The efficiency trick is that a constant-field segment (a "hold", including free precession) is a *single* exact rotation by $\gamma|B|\tau$ about the field direction, with no Larmor-timescale time stepping; only the adiabatic rotations and finite-duration reversals are sub-stepped, and only finely enough to resolve the (slow) field trajectory.

```
from spin_dynamics.nonresonant import (
    NonresonantFieldModel, sample_isochromats,
    csar_sequence, simulate_field_reversal_echoes,
)

model = NonresonantFieldModel(
    coil_a_tesla=2.14e-3, coil_b_tesla=2.14e-3, background_tesla=50e-6)
ens = sample_isochromats(model, 500, b_inhomogeneity=0.25, direction_tilt_deg=18.0)
unit = csar_sequence(model, echo_spacing_seconds=2.5e-3, tau_rev_seconds=8e-6)
result = simulate_field_reversal_echoes(model, ens, unit, num_echoes=256,
                                        t2_seconds=510e-3)
```

The Combined Sudden switching And adiabatic Rotation (CSAR) sequences suppress the background-field dephasing. `csar_sequence` is the 90-degree unit whose echo-to-echo propagator is a π rotation, refocusing every *second* echo (the odd echoes stay near zero and the even echoes carry a $\sim 50\%$ component that decays with T_2 plus a $\sim 50\%$ component with a Bessel-like odd-even modulation set by the finite reversal time). `csar_double_reversal_sequence` gives a 2π rotation that refocuses *both* parities (with an enhanced decay), and `csar_supercycle_sequence` alternates the adiabatic-rotation sense over a four-unit supercycle to suppress the imperfection decay down to the intrinsic T_2 limit. `effective_rotation` extracts the net-rotation axis and angle (the paper's analysis tool), and `sequence_waveform` returns the coil-current drive waveform for plotting.

```
python examples/plot_brill2002_field_reversal_echoes.py --output brill2002.png
```

The example reproduces the paper's key results: the basic reversal sequence decaying within tens of milliseconds from Earth's field, the CSAR train sustaining hundreds of echoes out to $T_2 = 510\,\mathrm{ms}$ with the odd-even modulation, an accurate fitted T_2, and the sequence comparison in which the supercycle reaches the T_2-limited decay. (The 90-degree waveform follows the paper's Fig. 1D directly; the 2π and supercycle sequences here are built from the same primitives and reproduce the same effects, but their exact coil waveforms differ from the paper's specific Fig. 3B/3C implementations.)

9.13 Phase Cycling

The Python API represents phase cycling with `spin_dynamics.phase_cycling.PhaseCycle`. A phase cycle is a scan table: rows are cycle steps, and columns are named logical RF pulse roles or events. For example, the default CPMG/PAP table has one pulse column, `excitation`, and two rows with phases $\pi/2$ and $3\pi/2$. Each row also carries a receiver phase, a branch weight, and an optional label. Branches are combined as

$$\sum_k w_k \exp(-i\phi_{\mathrm{rx},k})\, S_k,$$

with normalization by the sum of absolute receiver-weighted branch weights unless the table disables normalization.

```
from spin_dynamics.phase_cycling import cpmg_two_step_phase_cycle
from spin_dynamics.workflows import run_ideal_cpmg_train

cycle = cpmg_two_step_phase_cycle()
cycle.pulse_names        # ("excitation",)
cycle.branch_weights     # array([0.5+0.j, -0.5+0.j])

train = run_ideal_cpmg_train(num_echoes=8, numpts=51)
train.phase_cycle.name   # "cpmg_two_step"
```

The phase-cycle columns are not the set of unique RF phase values. They are the named pulse roles whose phase can vary from one scan branch to the next. This keeps the table aligned with sequence events and leaves room for receiver phase and weighting information on each row.

PGSTE walker results expose the same metadata field, but the interpretation is more specific. The `pgste_` constructor returns a one-row table for the selected stimulated-echo pathway with columns `excitation_90`, `store_90`, and `read_90`. The PGSTE workflow does not explicitly simulate and receiver-combine several phase-cycle branches; pathway selection is implemented by the spoiler and by suppressing equilibrium regrowth during storage.

`plot_phase_cycled_stimulated_echo.py` shows the explicit three-pulse phase table on a deterministic inhomogeneous-field ensemble. It compares a single three-90° scan, a read-pulse-only two-step cycle, and a full 64-step table over `excitation_90`, `store_90`, and `read_90`. During storage the example applies physical relaxation: transverse magnetization decays with T_2, while M_z relaxes back toward equilibrium with T_1. The recovered M_z produces a prompt last-pulse FID, and the full receiver signature $-\phi_1 + \phi_2 + \phi_3$ rejects that pathway while retaining the stimulated echo at τ.

9.14 Absolute RF Phase

Finite CPMG train workflows accept an optional `absolute_phase` mapping. The ideal workflow records the laboratory-frame phase schedule while preserving its nominal rectangular pulses. Probe-aware finite trains use the same schedule to solve the tuned, un-tuned, or matched probe waveform for each pulse, discretize the rotating-frame waveform into short segments, and build a separate refocusing matrix for each echo:

```
from spin_dynamics.workflows import run_tuned_cpmg_train

train = run_tuned_cpmg_train(
    numpts=21,
    num_echoes=32,
    absolute_phase={
        "rf_frequency_hz": 0.25 / 200e-6,
        "rf_phase_at_zero_rad": 0.0,
    },
)

phase_step = train.absolute_phase.delta_refocus_phase_cycles
```

For long trains, add `phase_bins` to reuse solved refocusing pulses on a uniform absolute-phase grid:

```
train = run_tuned_cpmg_train(
    num_echoes=256,
    absolute_phase={
        "rf_frequency_hz": 0.03125 / 200e-6,
        "phase_bins": 32,
    },
)

library = train.absolute_phase.refocus_pulse_library
matrix_phases = train.absolute_phase.refocus_matrix_phase_rad
```

The scheduled phase for every echo remains available as `refocus_absolute_phase_rad`. The quantized matrix phase, bin index, unique phase list, and exported `PulseShapeLibrary` are recorded in result metadata so cache behavior can be audited or reused in downstream analysis. The matched diffusion CPMG workflow uses the same `absolute_phase` mapping and additionally records the diffusion-encoding π pulse in `encoding_*` metadata fields.

The same solved pulse shapes can be inspected without running a full echo train:

```
import numpy as np

from spin_dynamics.pulse_diagnostics import solve_probe_pulse_shape_sweep

sweep = solve_probe_pulse_shape_sweep(
    probe="tuned",
    absolute_phase_rad=2 * np.pi * np.array([0.0, 0.25, 0.5, 0.75]),
    numpts=17,
)

shape = sweep.shapes[1]
drive = shape.drive
quadrature_fraction = shape.quadrature_energy_fraction
library = sweep.pulse_shape_library
```

When sweeping the phase advance between refocusing pulses, keep the absolute phase of the first refocusing pulse fixed if the goal is to isolate echo-to-echo variation. A half-cycle phase advance produces the same transverse rotating-frame pulse shape for a linear tuned probe, while intermediate advances change the rise/fall and quadrature transients and can produce extra attenuation in the matched-filter echo amplitude.

The Mandal-2015 plotting examples expose the diagnostic workflow directly:

```
python examples\plot_mandal2015_phase_step_sweep.py --probe tuned
python examples\plot_mandal2015_echo_modulation.py --probe tuned
python examples\plot_mandal2015_pulse_shapes.py --probe tuned
```

The finite-train plotting scripts accept `-phase-bins N` to exercise the same quantized pulse-cache path used by the workflow API. They also enable automatic offset-grid refinement by default and use `-rephase-action raise`; this keeps demonstration plots from silently using too few isochromats. Pass `-no-auto-refine-grid` only for explicit diagnostic runs.

`plot_mandal2015_pulse_shapes.py` solves a single refocusing pulse at several absolute RF phases and plots $\mathrm{Re}(B_1)$, $\mathrm{Im}(B_1)$, $|B_1|$, and the phase of the nonzero-amplitude segments. It is intended as a debugging and teaching aid: the finite train examples use the same public diagnostics API, pulse-shape solvers, and discretized segments.

9.15 Radiation Damping

```
from spin_dynamics.workflows import run_radiation_damping_fid

fid = run_radiation_damping_fid(
    probe="matched",
    fill_factor=0.7,
    equilibrium_magnetization=0.8,
    flip_angle=1.0,
    model="circuit",
    detuning=2.0e4,
)
```

Finite tuned and matched CPMG trains accept an opt-in `radiation_damping` mapping:

```
from spin_dynamics.workflows import run_tuned_cpmg_train

train = run_tuned_cpmg_train(
    numpts=51,
    num_echoes=8,
    radiation_damping={
        "fill_factor": 0.7,
        "equilibrium_magnetization": 0.8,
        "model": "circuit",
        "detuning": 2.0e4,
        "apply_during_pulses": True,
    },
)
```

The CPMG train path is deterministic and keeps the radiation-damping feedback inside the finite acquisition model. For strongly inverted samples, `simulate_nmr_maser` uses the same feedback machinery with a pumped longitudinal magnetization; it is intended as an idealized threshold and saturation diagnostic rather than a full spectrometer-control model.

9.16 Inverse Laplace Analysis

```
import numpy as np
from spin_dynamics.analysis import invert_t2, t2_kernel

echo_times = np.linspace(0.5e-3, 90e-3, 40)
t2_axis = np.logspace(-4, -1, 60)
signal = t2_kernel(echo_times, t2_axis) @ np.exp(-((np.log(t2_axis) + 4.5) ** 2))

result = invert_t2(
    signal,
    echo_times,
    t2_axis,
    regularization=5e-4,
    regularization_order=2,
)
```

For two-dimensional distributions, the separable forward model is

$$D = K_1 F K_2^{\mathsf{T}},$$

where D is the measured data, F is the unknown distribution, and K_1, K_2 are relaxation or diffusion kernels. Use `invert_t1_t2` or `invert_d_t2` for common NMR cases.

The PGSE D-T2 example ties this analysis layer to a sequence model: it builds the b-axis with `run_pgse_moment`, simulates a PGSE-prepared CPMG echo matrix, and recovers the D-T2 distribution with the SciPy-backed non-negative `invert_d_t2` solver.

9.17 Chemical and Site Exchange

The `spin_dynamics.exchange` namespace adds Bloch-McConnell site exchange, the relaxation-domain counterpart to the diffusion-exchange (DEXSY) walker example. An `ExchangeSystem` holds a tuple of `ExchangeSite` objects and a rate matrix whose entry (i, j) for $i \neq j$ is the first-order rate constant $k_{i\to j}$. Site populations are normalized, and detailed balance $p_i k_{i\to j} = p_j k_{j\to i}$ is checked unless disabled.

```
import numpy as np
from spin_dynamics.exchange import (
    two_site_exchange, simulate_relaxation_exchange_2d, exchange_spectrum,
)
from spin_dynamics.analysis import invert_t2_t2

system = two_site_exchange(
    offset_a_hz=0.0, offset_b_hz=0.0,
    k_ab_hz=8.0, population_a=0.5,
    t2_a_seconds=0.010, t2_b_seconds=0.200,
    labels=("fast", "slow"),
)
```

The kinetic generator X ($\dot{m} = Xm$) is column-conserving, so exchange alone preserves total magnetization. Adding per-site offset precession and transverse relaxation gives the transverse generator used for lineshapes: `exchange_spectrum` integrates the transverse Bloch-McConnell equations and reproduces the slow-exchange (two resolved lines) to fast-exchange (a single population-averaged line) coalescence crossover.

For relaxation exchange spectroscopy (REXSY), the encode-mix-detect model places exchange in a longitudinal mixing interval described by a mixing propagator G:

$$S(t_1, t_2) = \sum_b e^{-t_2/T_{2,b}} \sum_a G_{ba}\, p_a\, e^{-t_1/T_{2,a}}.$$

Inverting $S(t_1, t_2)$ with `invert_t2_t2` yields a T_2-T_2 map: diagonal peaks for spins whose T_2 is unchanged across mixing, and off-diagonal cross peaks whose intensity grows with the exchange rate.

```
encode = np.linspace(0.0, 0.06, 28)
detect = np.linspace(0.0, 0.8, 28)
rexsy = simulate_relaxation_exchange_2d(system, encode, detect, mixing_time=0.06)

t2_axis = np.logspace(-3, 0, 48)
ilt = invert_t2_t2(rexsy.data, encode, detect, t2_axis, regularization=1e-3)
exchange_map = ilt.distribution  # off-diagonal mass == exchange
```

The `plot_t2_t2_exchange.py` example runs this end to end. The REXSY forward model assumes refocused encode/detect trains and exchange confined to the longitudinal mixing interval; use the transverse engine directly when offset evolution during encoding matters.

9.18 Internal and Susceptibility Gradients

The `spin_dynamics.susceptibility` namespace generates the static internal field produced by magnetic-susceptibility contrast between a solid matrix and the pore fluid, the inhomogeneity that usually dominates porous-media NMR. Each `CylindricalInclusion` is an infinitely long cylinder perpendicular to the two-dimensional map plane, magnetized by a uniform in-plane applied field. Outside the cylinder the parallel field perturbation is the 2D dipole

$$\Delta B_{\parallel}(\rho, \varphi) = B_0 \frac{\Delta\chi}{2} \left(\frac{a}{\rho}\right)^2 \cos(2\varphi),$$

with radius a, distance ρ, and angle φ measured from the applied-field direction. Cylinders superpose in the dilute ($|\Delta\chi| \ll 1$) limit.

```
import numpy as np
from spin_dynamics.susceptibility import (
    CylindricalInclusion, susceptibility_offresonance_map,
    internal_gradient_distribution, make_susceptibility_field_maps,
)

axis = np.linspace(-50e-6, 50e-6, 161)
grains = [CylindricalInclusion(cx, cz, 12e-6)
          for cx in (-30e-6, 30e-6) for cz in (-30e-6, 30e-6)]

field = susceptibility_offresonance_map(
    axis, axis, grains, b0_tesla=2.0, susceptibility_difference=1e-6,
)
dist = internal_gradient_distribution(field)  # pore-space |g| in T/m
maps = make_susceptibility_field_maps(field)  # drop into motion helpers
```

The off-resonance map is returned in the angular-frequency (rad/s) convention used by `spin_dynamics.motion`, so it feeds the moving-isochromat sequence helpers directly. With no applied gradient, a CPMG echo train of diffusing walkers then decays purely from diffusion in the internal gradient.

The acceleration of that decay with echo spacing (rate $\sim G^2 D\, T_E^2$) is the general diffusion-in-a-gradient effect and is *not* by itself a signature of an internal gradient: a uniform applied or static gradient produces it too. What distinguishes a susceptibility-induced gradient is its field dependence. Because $G_{\mathrm{internal}} \propto \Delta\chi\, B_0$, the diffusion-induced decay rate scales as B_0^2, whereas an applied gradient is set by hardware and is independent of B_0. The broad pore-space gradient *distribution* is a second distinguishing feature; a uniform gradient would be a single value. The `plot_internal_gradients.py` example shows the internal field, the internal-gradient distribution, and the recovered B_0^2 scaling of the diffusion-induced decay rate end to end. Background-gradient-suppressing sequences are implemented in the next section; the internal field produced here is their input.

9.19 Bipolar 13-Interval PGSTE

The `spin_dynamics.workflows.bipolar` module implements the Cotts 13-interval alternating pulsed-gradient stimulated-echo (APGSTE) sequence, the sequence in the Bruker `diff_stebp.gp` program, which measures diffusion while suppressing a constant background gradient g_0 such as the internal gradient of Section 9.18. It is a stimulated echo (three 90 degree pulses) with one 180 degree pulse in each of the two encoding periods, a gradient lobe on each side of every 180, and the applied polarity inverted between the two periods. The 180 pulses refocus the background gradient within each period, and the polarity alternation cancels the applied-times-background cross-term.

Working in the toggling (coherence) frame, the diffusion attenuation in a constant background gradient is

$$\ln E = -D\left(b_{\mathrm{applied}} + g_0\, c_{\mathrm{cross}} + g_0^2\, c_{\mathrm{bg}}\right),$$

and `toggling_frame_moments` returns the three coefficients. For the 13-interval sequence $c_{\mathrm{cross}} = 0$; for an ordinary monopolar stimulated echo it is non-zero, so its apparent diffusion coefficient drifts with g_0.

```
from spin_dynamics.workflows import (
    run_cotts_thirteen_interval_moment, run_monopolar_pgste_moment,
)

c = run_cotts_thirteen_interval_moment(gradient_amplitude=0.1, background_gradient=0.05)
m = run_monopolar_pgste_moment(gradient_amplitude=0.1, background_gradient=0.05)
print(c.cross_term_bias, m.cross_term_bias)  # ~0 vs non-zero
print(c.phase_cycle.name)                   # "diff_stebp"
```

The phase cycle is the 16-step Bruker `diff_stebp` table, built on the package's `PhaseCycle` machinery as `diff_stebp_phase_cycle()`. Its receiver satisfies $\mathrm{ph31} = -(\mathrm{ph1} + \mathrm{ph2} + \mathrm{ph3}) \bmod 4$ (with $\mathrm{ph4} = 0$ on both 180 pulses), selecting the stimulated-echo coherence-transfer pathway $\Delta p = (+1, -2, +1, +1, -2)$ with detection at $p = -1$; the coherence order $p(t)$ this cycle selects is exactly the sign carried by the toggling-frame intervals.

For restricted geometry and finite-pulse effects, `run_cotts_thirteen_interval_walkers` runs the full five-pulse sequence with moving walkers, applying a constant background gradient as a linear off-resonance map. For free diffusion the walker echo reproduces $\exp(-bD)$ with the moment-model b-value, and the apparent diffusion coefficient from a b-value sweep is unbiased by the background gradient. The `plot_bipolar_pgste.py` example traces the wavevectors, the moment-model apparent-diffusion suppression versus g_0, and the walker runner validated against $\exp(-bD)$. The moment model is the free-diffusion b-value for ideal rectangular lobes; trapezoidal ramps need a shaped-gradient extension.

9.20 Optimization Workflows

The optimization package provides compact, plotting-free drivers for phase-program searches and result handoff:

```
from spin_dynamics.optimization import run_tuned_refocusing_multistart

opt = run_tuned_refocusing_multistart(
    num_segments=3,
    num_starts=4,
    numpts=21,
    max_passes=4,
)
```

Use the optimization examples as diagnostics rather than turnkey production pulse design: they expose refocusing, target-excitation, and inverse-excitation objective behavior while the inverse-cancellation workflow remains under validation.

9.21 GRAPE Optimal Control

The `optimal_control` package generalizes the package's autodiff machinery from a single hand-built objective (the ideal-v0crit refocusing-phase search above) into a reusable gradient-ascent pulse engineering (GRAPE) toolkit: it optimizes the full piecewise-constant RF control waveform of an arbitrary pulse against a state-transfer or propagator/gate fidelity target, using reverse-mode autodiff through a differentiable Hilbert-space propagator chain (`jax.lax.scan` over segment-wise `expm(-i H_seg dt)`, mirroring the `core._jax_kernels` convention that keeps compile time independent of segment count).

Control is parameterized as amplitude and phase per segment (not Cartesian I/Q), because **phase-only control is the primary mode this package targets**: field-deployable NQR/NMR hardware frequently uses switching power amplifiers that cannot vary RF amplitude continuously, so only the phase is agile. Peak-B1 is then an exact box constraint on the amplitude channel in this parameterization (independently box-bounding Cartesian I/Q components would incorrectly permit an effective amplitude above the true hardware limit at the box corners), and fixing amplitude to a hardware-realistic constant and optimizing only phase requires no constrained-optimization machinery beyond the existing bounded L-BFGS-B driver.

```
from spin_dynamics.coupling.systems import coupled_spin_system
from spin_dynamics.optimal_control import (
    coupled_spin_control_model,
    grape_optimize_phase_only,
    rectangular_seed_phase,
)

model = coupled_spin_control_model(coupled_spin_system([300.0], [[0.0]]))
n_segments = 16
dt = [12.5e-6] * n_segments  # segment duration = hardware control bandwidth

result = grape_optimize_phase_only(
    model,
    rectangular_seed_phase(n_segments),
    fixed_amplitude=5000.0,          # Hz, held constant (phase-only GRAPE)
    dt=dt,
    target=[0.0, 1.0],               # invert |up> -> |down>
```

```
    psi0=[1.0, 0.0],
    hamiltonian_batch=training_offset_hamiltonians,  # robust/ensemble objective
)
```

`hamiltonian_batch` turns the objective into a robust/ensemble optimization: the same controls are propagated (via `jax.vmap`) across a batch of `H_drift` variants (e.g. a B0-offset or B1-scale-factor grid) and the per-case fidelities are reduced (mean by default, or worst-case) to a single score – directly reusing the vmap-batched primitive from the acceleration workstream. `examples\grape_broadband_pulse_optimization.py` demonstrates this end to end: a phase-only inversion pulse trained on a coarse B0-offset ensemble and evaluated on a finer held-out grid, where a plain rectangular pulse's worst-case fidelity collapses to zero across a wide offset band while the GRAPE-optimized pulse holds above 0.97.

Validation. The differentiable propagator agrees with two independent, pre-existing implementations to machine precision rather than merely within tolerance: the plain-NumPy reference (bit-identical to the JAX path, as expected for the same arithmetic traced two ways) and the pre-existing `core.rotations` functions `calc_rot_axis_arba4` and `sim_spin_dynamics_exc` (max absolute deviation $\sim 10^{-15}$–10^{-16}, checked for both a single on-resonance pulse and a five-segment off-resonance train). Reverse-mode gradients match central finite differences to a relative error of $\sim 5 \times 10^{-10}$ – well inside the 5×10^{-3} tolerance the existing v0crit autodiff port uses, since this engine differentiates through an explicit $\exp(-iHt)$ rather than the more numerically delicate windowed v0crit objective. In the broadband-pulse example above (16 segments, 5000 Hz nutation, 200 µs total duration, trained on 7 offsets across ± 3500 Hz, evaluated on a held-out 41-point grid), the rectangular baseline's worst-case fidelity is 0.000 – it cannot invert the band edges at all – while the GRAPE-optimized pulse holds 0.974 worst-case and 0.982 mean, at the identical peak B1.

Multistart is not optional. A rectangular (constant-phase) pulse seeded against a perfectly offset-symmetric ensemble sits at an exact zero-gradient stationary point: mirrored offsets' gradient contributions cancel in the ensemble mean, so an optimizer started there reports immediate convergence with no improvement. This is a genuine symmetry, not a bug or an unlucky initial guess – for a constant or antisymmetric-phase pulse the effective rotation axis is confined to the x-z plane with n_x even and n_z odd in offset, so a purely-x figure of merit cannot see any first-order improvement from a symmetric perturbation. Random-restart multistart (`run_grape_multistart`) reliably escapes it – in one tested case a random start reached 0.901 where the rectangular seed's own local search reported 0.780 with zero gradient – so every solver entry point seeds a rectangular baseline alongside genuinely random restarts rather than relying on the rectangular pulse alone.

Scope note: this engine targets a single (or J-coupled) spin-1/2 system with unitary (relaxation-free) propagation. Gradient-waveform control (joint RF+gradient optimization for spatially selective pulses) is now implemented and documented in the next subsection; higher-spin quadrupolar targets are designed as a straightforward extension – the existing `nqr` Hamiltonian builders in place of `coupled_spin_control_model`, same propagation chain – but are not implemented yet.

9.21.1 Gradient control channel and slice-selective pulses

A magnetic-field gradient couples to a spin as a *position-dependent* offset: the local off-resonance of a spin at position r is $g(t)\,r$, an I_z term whose weight is the applied gradient times the position (the `positions @ gradient` rule the motion/imaging engine's `workflows.slice_selective` uses). GRAPE promotes the gradient waveform $g(t)$ to a genuine optimized control channel alongside RF amplitude and phase:

$$H(t) = H_{\text{drift}} + w_1(t)\big(\cos\phi(t)\,H_x + \sin\phi(t)\,H_y\big) + g(t)\,H_{\text{grad}},$$

where $H_{\text{grad}} = 2\pi I_z$ (`gradient_control_operator`). The defining feature that distinguishes the gradient from the RF channel is that a *single* shared waveform $g(t)$ acts through a *position-scaled* operator per ensemble member: for a position ensemble $\{r_k\}$, member k is driven by $g(t)\, r_k\, H_{\text{grad}}$ (`position_gradient_batch` builds the $r_k\, H_{\text{grad}}$ operators). This is a vmap-axis extension of the robust-ensemble primitive, not a new engine.

Because different positions want different outcomes – inverted inside a slice, untouched outside – the objective accepts *per-case* targets: pass `target` as a $(\text{batch}, \text{dim})$ array (one target ket per ensemble member) instead of the shared $(\text{dim},)$ form. Activate the channel by passing `gradient_operator_batch`; set `optimize_gradient=True` (with `initial_gradient` and `gradient_max`) to optimize $g(t)$, or hold it at `fixed_gradient`. The control vector is `concat([amplitude?, phase, gradient?])`.

```
from spin_dynamics.optimal_control import (
    coupled_spin_control_model, gradient_control_operator,
    position_gradient_batch, constant_gradient_seed, grape_optimize,
)

system = coupled_spin_system([0.0], [[0.0]])
model = coupled_spin_control_model(system, include_gradient=True)
h_grad = gradient_control_operator(system)          # base 2*pi*Iz

positions = np.linspace(-1.0, 1.0, 15)
h_grad_batch = position_gradient_batch(h_grad, positions)  # r_k * h_grad
targets = np.stack([[0.0, 1.0] if abs(p) <= 0.3         # invert in-slice
                    else [1.0, 0.0] for p in positions]) # preserve out-slice

result = grape_optimize(
    model, initial_phase, dt=dt, target=targets, psi0=[1.0, 0.0],
    fixed_amplitude=2000.0,                     # phase-only RF
    optimize_gradient=True,
    initial_gradient=constant_gradient_seed(len(dt), gradient=1.0),
    gradient_max=5.0, gradient_operator_batch=h_grad_batch,
)
opt_gradient = result.best_gradient             # the optimized g(t)
```

`examples\grape_slice_selective_pulse.py` demonstrates this end to end: co-optimizing the RF phase and the gradient waveform for a slice-selective inversion, evaluated on a held-out position grid, sharpens the in-slice inversion from ~ 0.73 to ~ 0.90 over a fixed-gradient rectangular baseline while holding out-of-slice preservation at ~ 0.92, lifting the mean per-position fidelity from 0.62 to 0.92. A conditioning note applies whenever RF and gradient channels are optimized jointly: a raw gradient in hertz-per-position is $O(10^3)$ while phases are $O(1)$, a scale mismatch that stalls L-BFGS-B. Folding a characteristic offset scale into the gradient operator (so the optimized $g(t)$ is itself $O(1)$) restores a well-conditioned problem; the example does exactly this.

9.21.2 Hardware response: probe and gradient-driver dynamics

The pulses above assume the spins see exactly the commanded waveform. Real hardware does not deliver it: a tuned or matched RF probe has finite bandwidth (loaded Q) and a ring-down tail, and a gradient driver has a finite L/R slew plus eddy-current droop. Each is, to excellent approximation, a *linear time-invariant* (LTI) filter between the commanded waveform and the field the spins see. `optimal_control.control_response` models these as one differentiable stage the objective inserts in the loop, between the *command* and the

Hamiltonian:

$$u_{\text{delivered}}(t) = \big(\mathcal{L}\, u_{\text{command}}\big)(t), \qquad u = w_1\, e^{i\phi}.$$

Because the whole chain is JAX, autodiff supplies the filter's adjoint for free – no hand-derived circuit gradient. Two consequences make this the physically correct thing to optimize: fidelity is scored on the delivered pulse (*including* the ring-down/eddy tail, which really does rotate the spins), and the box constraints stay on the command, so the optimizer pre-emphasizes only up to the real hardware limits rather than demanding voltages the amplifier cannot produce.

Transmit is filtered *once*, in time, on the rotating-frame envelope: the coil emits one physical $B_1(t)$ that every isochromat sees, differing only by its own detuning (already in the drift), so there is no per-offset transmit filtering. Tuned and matched probes have genuinely different dynamics and are separate classes: a tuned probe is a single resonance, a one-pole envelope lowpass with ring-down time constant $\tau = 2Q/\omega_0 = Q/(\pi f_0)$ (`TunedProbeResponse`); a matched probe adds a matching-network pole and is a distinct two-pole response (`MatchedProbeResponse`); an untuned coil is near-flat (`UntunedProbeResponse`). RF probes filter the complex envelope by an FFT multiply; the gradient driver (`GradientDriverResponse`) is a causal state-space cascade of one L/R slew pole and a shelf per eddy term, whose delivered step is the classic $G(t) = G_\infty[1 - \sum_k \alpha_k e^{-t/\tau_k}]$ with DC gain one (the steady-state gradient equals the command). `eddy_terms_from_step_response` fits the (α_k, τ_k) from a measured step. The command is upsampled by the response's `oversample` factor before filtering, and a ring-down/eddy tail (`TailSpec`: a multiple of the dominant τ, or an explicit window) is appended.

```
from spin_dynamics.optimal_control import (
    coupled_spin_control_model, grape_optimize, TunedProbeResponse,
)

probe = TunedProbeResponse.from_quality_factor(   # ring-down tau = Q/(pi f0)
    f0_hz=1.0e6, quality_factor=120.0, oversample=4,
)
result = grape_optimize(
    model, initial_phase, dt=dt, target=[0.0, 1.0], psi0=[1.0, 0.0],
    fixed_amplitude=6000.0,        # command bound stays on the amplifier
    rf_response=probe,             # fidelity scored on the DELIVERED pulse
)
```

Both `rf_response` and `gradient_response` are optional and default to the ideal (unfiltered) behaviour bit-for-bit; they require a uniform scalar `dt`. `examples\grape_probe_response_pulse.py` shows a band-limited tuned probe cutting a rectangular inversion pulse's delivered fidelity to ~ 0.84, which GRAPE optimizing the command recovers to 1.0; it also illustrates the gradient-driver step response and the eddy fit. Probe-robust pulses follow by making the filter parameters (tuning, Q, eddy τ_k) the ensemble axis, reusing the robust-ensemble machinery. The design note `docs/optimal_control_hardware_response.md` records the full rationale, including the output-side `ReceiverResponse` for objectives defined on the detected signal.

9.21.3 PGSE diffusion: correct q-vector and detected SNR

The diffusion capstone (`optimal_control.diffusion`) co-optimizes the RF pulses *and* the gradient waveform of a pulsed-gradient spin echo on realistic hardware: a tuned RF probe on transmit *and* receive, a gradient driver with L/R slew plus eddy currents, and an inhomogeneous B_0/B_1 field. It exercises the entire stack – the M2 gradient channel, the M4 hardware-response layer, and a combined (position $\times$ B_0 $\times$ B_1) ensemble – against two objectives:

1. **Correct q-vector**: a functional of the *delivered* gradient. `gradient_moments` returns $q(t) = \int g(\tau)s(\tau)\,d\tau$ (with the coherence sign $s = \pm 1$ that a $180°$ pulse flips), its encode-lobe value and its residual at the echo, matching `workflows.pgse`'s convention. The encode moment must hit a target q^* and the residual must refocus to zero; eddy droop violates both, so the optimizer pre-emphasizes the commanded gradient (bounded) to correct them.
2. **Maximize detected SNR**: `detected_echo_signal` acquires the coherently-summed echo over a readout window, `detected_echo_snr` filters it through the Rx probe and returns the matched-filter amplitude over a fixed noise floor.

`make_pgse_objective` assembles the weighted `value_and_grad` (SNR $- \lambda_q(q_{\text{enc}} - q^*)^2 - \lambda_0 q_{\text{echo}}^2$), propagating with `propagator="expm"` by default – on-resonance spins have $H_{\text{seg}} = 0$ during RF/gradient gaps, whose `eigh` gradient is NaN (the same degeneracy the NQR models hit). `examples\plot_grape_pgse_diffusion.py` runs it in a *grossly* inhomogeneous field (B_0 spread several times the nutation rate). Two effects, both large: (i) a hard $180°$ refocuses only within $\sim \pm$nutation, so the naive echo collapses across the band, while phase-only GRAPE finds a broadband phase-modulated refocusing pulse – the RP2 / symmetric-phase-alternating family – that recovers the echo across the full band for a several-fold detected-SNR gain; and (ii) eddy droop pushes the naive encode moment below q^* with imperfect refocusing, which the gradient pre-emphasis corrects to $\sim 0\%$ error. **Multistart is essential**: a rectangular ($\phi = 0$) seed against a symmetric offset ensemble sits at an exact saddle (mirrored offsets' phase gradients cancel), so a single start never moves the phase – random phase restarts escape it. The design note is `docs/optimal_control_pgse_diffusion.md`.

9.21.4 Detector-aware objectives: optimizing for a flux readout

The PGSE objective scores the detected echo through a tuned-probe `ReceiverResponse` over a scalar noise floor – the inductive-coil picture. A SQUID or OPM (Section 9.7) is a field sensor with a frequency-dependent field-noise floor $N^2(f)$ (flat for a SQUID, a Lorentzian band-pass for an RF-OPM) and no $\mathrm{d}\Phi/\mathrm{d}t$ weighting. `optimal_control.detection_objective` scores the same acquired echo with the field-referred matched filter

$$\mathrm{SNR} = \sqrt{\int \frac{|S(f)|^2}{N^2(f)}\,\mathrm{d}f},$$

so the optimizer shapes the pulse to place signal where the *detector* is quiet. Because $N^2(f)$ is fixed over the acquisition frequency grid (it does not depend on the controls), it is precomputed once with NumPy (`detector_noise_grid`, which also re-centres a band-pass detector's response to baseband) and passed as a static weight into the differentiable objective; a flat $N^2 = \sigma^2$ reproduces `detected_echo_snr` with `noise` $= \sigma$ (Parseval), so it is a strict generalization. `make_detected_snr_objective` builds the "value_and_grad" over a (B_0, B_1) ensemble; with `per_rf_power` the score is SNR per unit RF amplitude, the efficiency figure of merit that makes the readout actually shape the pulse (raw SNR always rewards more excitation, since out-of-band signal never hurts). The example `examples/plot_grape_detector_aware_pulse.py` optimizes an excitation for a broadband SQUID versus a narrow SERF OPM and shows, by cross-evaluation, that each pulse is best under its own detector.

9.21.5 Axis-matched excitation (AMEX) and CPMG SNR per unit time

`su2_effective_axis` extracts a refocusing cycle's effective rotation axis directly from its GRAPE propagator (validated against `core.rotations.calc_rot_axis_arba4` to machine precision), and `bloch_vector_to_ket` converts that axis to the spin state it is the fixed point of. In an inhomogeneous field the steady state reached

after many refocusing cycles is the initial magnetization's projection onto this offset-dependent axis, so exciting directly onto it (axis-matched excitation, AMEX) reaches the asymptotic echo amplitude from the first cycle – provably a fixed point of the cycle – while a conventional excitation must settle through a transient that need not converge cleanly. `examples\grape_cpmg_snr_per_time.py` demonstrates this together with the time cost of long refocusing pulses: it reuses the `optimization.spa` figure of merit `fom_time = echo_spacing / SNR`^{2} (`echo_spacing` includes the pulse length directly, so longer pulses are penalized for producing fewer echoes per unit time), sweeps candidate refocusing-pulse durations, and follows the sequential/separate methodology (refocusing optimized first assuming an idealized excitation, then AMEX excitation designed onto its axis) alongside a joint optimization for comparison – both, and a rectangular pulse at the same peak B1, are reported so the trade-off is decided empirically rather than assumed.

Validation: an ideal axis-matched state is an exact fixed point. Simulating the refocusing cycle's unitary applied repeatedly (the echo-train primitive, `iterate_unitary`) confirms the theory directly: starting from the axis-aligned ket, the ensemble-averaged transverse signal is identical at every one of 25 simulated cycles (agreement to $\sim 10^{-9}$, i.e. no numerically detectable transient), whereas starting from the conventional $+x$ state produces a genuine, unsettled oscillation that has not converged even after 25 cycles for the offset ensembles tested here – not a clean exponential decay to a plateau, since different offsets precess around their own tilted axes at different rates and beat against each other in the ensemble average.

Segments	t_p (µs)	SNR (rect+rect)	SNR (GRAPE+AMEX)	SNR (joint)	fom_time gain
4	32	0.327	0.759	0.805	5.4×
8	64	0.121	0.641	0.703	27.9×
16	128	0.0001	0.762	0.567	4.3×10^4

Table 9.1: CPMG SNR-per-unit-time sweep at fixed peak nutation rate 5000 Hz, 25 refocusing cycles, ± 3000 Hz training ensemble (7 offsets), 41-point held-out evaluation grid, 25 µs free-precession delay on each side of the refocusing pulse. "fom_time gain" is the rectangular-baseline to GRAPE+AMEX ratio of `fom_time` (SNR-per-unit-time improvement) at that duration; the 16-segment entry is dominated by the rectangular pulse's total nutation angle ($\sim 230°$ at this peak amplitude and duration) being badly off a $180°$ refocusing condition – GRAPE's phase modulation recovers a working pulse from what is, at fixed constant phase, an essentially failed one, rather than showing a moderate, generic optimization gain.

The joint optimization (refocusing and excitation phases optimized together against the same asymptotic-signal objective) is not consistently better or worse than the sequential/AMEX approach across this sweep (ahead at 4 and 8 segments, behind at 16) – read as an honest, instance-specific answer rather than a general preference for one methodology, since with a fixed multistart budget the larger joint parameter space is not uniformly easier to search.

Comparison with prior work. Mandal et al. (2013, *Axis-matching excitation pulses for CPMG-like sequences in inhomogeneous fields*) and Mandal et al. (2014, *Direct optimization of signal-to-noise ratio of CPMG-like sequences in inhomogeneous fields*) are the original sources for both AMEX and the SNR-per-unit-time figure of merit; both use GRAPE or `fmincon` for this same class of refocusing-axis/excitation optimization, and the 2014 paper hand-derives (Appendix A) the analytic gradient that this engine obtains automatically via reverse-mode autodiff. Their $\mathrm{SNR}_{\mathrm{time}} \propto \mathrm{SNR}_{\mathrm{power}}/t_{E,\min}$ is the same physics as `fom_time` (their "SNR" is explicitly a power, i.e. amplitude squared, so $\mathrm{SNR}_{\mathrm{time}} = 1/\mathtt{fom\_time}$) – `optimization.spa`'s formula traces to this same result. The symmetry that makes a rectangular (constant-phase) refocusing pulse a zero-gradient stationary point for a symmetric offset ensemble (Section 9.21 above) is their Eq. (5)–(7) (n_x even, $n_y \equiv 0$, n_z odd in offset for constant or antisymmetric-phase pulses), which

they exploit deliberately to halve their own search space rather than treat as an obstacle – a natural refinement for a future increment here. Quantitatively, however, this example's offset range (at most $\sim 0.7\times$ the nutation rate) and duration sweep (up to $1.3\times$ the nominal $180°$ pulse length) are both far milder than either paper's benchmark regime: the 2013 paper uses offsets up to $\sim 10\times$ the nutation rate, the 2014 paper up to $\sim 50\times$, and the 2014 paper's central finding is that SNR per unit time peaks for refocusing pulses 2–$5\times$ the standard $180°$ duration – a regime this sweep does not reach. That mismatch, not a discrepancy in the physics, is why this example's refocusing-only gain is modest (a few percent) where the papers report gains of several-fold, and why its AMEX-to-ideal-excitation ratio is not close to their reported $\sim 2\times$: mild inhomogeneity leaves a conventional excitation and a simple rectangular refocusing pulse much closer to adequate already. Reproducing their specific benchmark numbers directly would require re-running this example at their offset range and duration sweep, which has not yet been done.

9.21.6 NQR / quadrupolar targets

The GRAPE engine is Hilbert-space-general – it needs only a `ControlHamiltonianModel` (`h_drift`, `h_x`, `h_y`) – so extending it to quadrupolar (NQR) nuclei is a matter of supplying that model, not a new engine. `nqr_site_control_model(site)` builds it for a spin $I = 1, 3/2, 5/2, 7/2, 9/2$ site.

Frame. A piecewise-constant lab/PAS field cannot drive an MHz quadrupolar transition (it is a DC field), so the model is built in the *rotating frame at the RF carrier*, in the energy eigenbasis — exactly the frame the full NQR density-matrix engine uses (Section 7.4). `h_drift` is the rotating-frame static Hamiltonian (`static_hamiltonian_rotating`: diagonal detunings) and `h_x`/`h_y` are the co-rotating RWA drive quadratures extracted from `pulse_hamiltonian`, so `pulse_hamiltonian` $\equiv$ `h_drift` $+ w_1(\cos\phi$ `h_x` $+\sin\phi$ `h_y`$)$ and a pulse optimized on this model round-trips through the real simulator to machine precision. The default carrier is the fundamental (strongest) transition; the model operates in the eigenbasis, so the fundamental transition's two eigenstates are the natural inversion endpoints (`nqr_fundamental_states` returns their indices). The single-carrier RWA is faithful only near the fundamental line, so driving a satellite warns (exact satellite pulsing needs non-RWA time-domain propagation, not yet implemented).

The eigh-vs-expm gradient. Quadrupolar spectra are Kramers-degenerate. The default segment propagator diagonalizes H_{seg} with `eigh`, whose reverse-mode gradient carries $1/(\lambda_i - \lambda_j)$ terms that become *NaN* for degenerate eigenvalues. NQR models must therefore pass `propagator="expm"`, which propagates via `jax.scipy.linalg.expm` – its gradient is well-defined under exact degeneracy (agreeing with the `eigh` forward map to $\sim 10^{-9}$ for non-degenerate systems and with finite differences to $\sim 10^{-3}$).

Two robustness axes. Field NQR detection faces two dominant broadenings, each a different ensemble over the shared control waveform:

- **EFG / detuning inhomogeneity.** A distribution of ν_Q (EFG line broadening) detunes spin packets from a fixed carrier; this is a per-case `h_drift` ensemble (`hamiltonian_batch`), identical in structure to the spin-1/2 B0-broadband case. In zero-field NQR the transition frequencies depend only on ν_Q and η, so this is a frequency spread, *not* a powder of orientations (which vary coupling, next item). `examples\grape_nqr_broadband_pulse.py` inverts a ^{63}Cu fundamental line across a ± 25 kHz ν_Q band where a fixed-carrier rectangular pulse's held-out worst-case inversion collapses to ~ 0.01 while the optimized phase-only pulse holds ~ 1.0. `examples\plot_grape_nqr_broadband_spectrum.py` plays the same idea for *excitation*: it optimizes a broadband $90°$ pulse, then Fourier- transforms the excited coherence over a Gaussian ν_Q distribution into a detected spectrum – the rectangular

pulse's sinc-like excitation nulls narrow and distort the line, while the GRAPE pulse recovers the true broadened lineshape.

- **Powder orientation.** A crystallite's RF-to-EFG coupling depends on the RF direction in its principal-axis system, so `h_x`/`h_y` vary per orientation while `h_drift` is shared — a per-case *drive-operator* ensemble. Pass `control_operator_batch` (from `nqr_powder_control_batch`); `examples\grape_nqr_powder_pul` inverts a ^{35}Cl line across a powder (held-out worst-case $\sim 0.03 \rightarrow 0.87$).

The two axes compose: passing `hamiltonian_batch` and `control_operator_batch` of equal length makes each ensemble member carry its own $(\texttt{h\_drift}, \texttt{h\_x}, \texttt{h\_y})$, so a flattened *orientation* $\times$ ν_Q product grid optimizes a single pulse robust to detuning *and* orientation at once (no extra machinery – the per-case drift and per-case drive paths already coexist).

Offset-robust excitation for a single-crystal SLSE train. `examples\plot_grape_nqr_slse_excitation.py` optimizes the excitation of a spin-lock spin-echo (SLSE) detection train for a *single crystallite* at a known carrier offset. An off-resonance spin loses signal under a conventional rectangular 90: the refocusing cycle spin-locks only part of the magnetization, so the echo train decays/oscillates to a small steady value. Numerically optimizing the excitation waveform against the *actual* steady echo – the thermal `equilibrium_density` propagated through the full $(2I+1)$ density matrix and read out by `detection_operator` – recovers the on-resonance signal level and a flat, transient-free train. The signal-to-noise gain grows with offset: none on resonance (a rectangular pulse already covers it), rising to ~ 1.3–$1.4\times$ once the offset exceeds the pulse bandwidth ($\gtrsim 2\times$ the nutation rate), where it also beats a flip-angle-optimized rectangular excitation.

This is a single-crystal (known-offset) result and does not extend to a powder. Over a powder the refocusing-cycle response varies strongly and oppositely with orientation (RF coupling and effective axis), one pulse cannot serve a spread of offsets at once, and the conventional rectangular SLSE already navigates the powder with its powder-optimum ("magic-angle", $\sim 119^\circ$ for spin-1 with $\eta \neq 0$) flip – against that flip-angle-optimized baseline an optimized pulse does not improve the powder echo, a genuine negative result reported rather than hidden. The per-orientation refocused-signal ratio is $\min / \max \approx 0.02$ for SLSE versus ≈ 0.26 for the small-tip SORC steady state, so SORC is far more powder-robust to begin with. The clean GRAPE wins for NQR are therefore broadband *excitation* and *inversion* (above), not beating well-tuned powder detection trains.

Higher-spin transition-selective pulses (drive one line, leave another) and exact lab-frame satellite control are natural follow-ons on the same engine.

10 Examples

The example scripts can be run from `PythonSpinDynamics`; each script also works from inside `examples/` by adding the local source tree to `sys.path` when needed. Treat the chapter as a runnable index rather than a tutorial: the explanatory chapters above describe the model choices, while these commands provide quick checks and reproducible figures. Start with the plain scripts when checking installation and with the plotting scripts when comparing physical effects.

```
python examples\ideal_cpmg.py --numpts 101
python examples\ideal_fid.py --numpts 101
python examples\ideal_cpmg_train.py --numpts 101 --num-echoes 8
python examples\ideal_time_varying_cpmg.py --numpts 101 --num-echoes 16
python examples\compare_cpmg_fid.py --numpts 101
python examples\export_validation_arrays.py results\validation_arrays.npz --numpts 101
python examples\tuned_probe_cpmg.py --numpts 101
python examples\probe_cpmg_compare.py --numpts 101
python examples\probe_parameter_sweeps.py --numpts 101
python examples\matched_cpmg_ir_train.py --numpts 21 --num-echoes 4 --num-tau 4
python examples\finite_probe_train_sweeps.py --numpts 21 --num-echoes 3
python examples\matched_diffusion_cpmg.py --numpts 21 --num-echoes 3
python examples\porous_rock_cpmg_walkers.py --estimate-only
python examples\received_signal_noise.py --numpts 51
python examples\radiation_damping_fid.py --probe matched --points 401
python examples\radiation_damping_cpmg_train.py --probe tuned --numpts 21 --num-echoes 4
python examples\nmr_maser.py
python examples\heteronuclear_j_editing.py --points 33
python examples\coupled_isochromat_fields.py --points 21
python examples\diagnose_optimization_backends.py --backend all --numpts 21 --segments 3
python examples\refocusing_gradient_optimization.py --segments 10 --starts 8
python examples\grape_broadband_pulse_optimization.py --segments 16 --starts 24
```

Plotting examples require Matplotlib:

```
python examples\plot_ideal_workflows.py --numpts 201 --output results\ideal_workflows.png
python examples\plot_ideal_imaging.py --pixels 6 --ny 7 --output results\ideal_imaging.png
python examples\plot_custom_imaging_fields.py --pixels 8 --ny 7 --output results\
    custom_imaging_fields.png
python examples\plot_inverse_laplace.py --output results\inverse_laplace.png
python examples\plot_pgse_d_t2.py --output results\pgse_d_t2.png
python examples\plot_phase_cycled_stimulated_echo.py --output results\phase_cycled_ste.png
python examples\plot_probe_parameter_sweep.py --probe tuned --sweep q --output results\
    tuned_q_sweep.png
python examples\plot_probe_cpmg.py --numpts 101 --output results\probe_cpmg.png
python examples\plot_finite_train_workflows.py --numpts 65 --num-echoes 4 --output results
    \finite_trains.png
python examples\plot_diffusion_sweep.py --numpts 65 --num-echoes 3 --output results\
    diffusion_sweep.png
python examples\plot_diffusion_absolute_phase_compare.py --output results\
    diffusion_absolute_phase_compare.png
python examples\plot_tuned_diffusion_absolute_phase_compare.py --output results\
    tuned_diffusion_absolute_phase_compare.png
python examples\plot_time_varying_sweep.py --numpts 51 --num-echoes 12 --output results\
    time_varying_sweep.png
```

```
python examples\plot_udd_cpmg_filter.py --pulses 8 --output results\udd_cpmg_filter.png
python examples\plot_sensitive_slice.py --output results\sensitive_slice.png
python examples\plot_multislice_halbach_imaging.py --pixels 16 --slices 5 --output results
    \multislice_halbach.png
python examples\plot_halbach_dipole_field.py --rod-shape square --output results\
    halbach_dipole.png
python examples\plot_nmr_mouse_fields.py --pixels 121 --output results\nmr_mouse_fields.
    png
python examples\plot_nmr_mouse_depth_profile.py --num-depths 13 --num-d-depths 5 --seeds 4
     --output results\nmr_mouse_depth.png
python examples\plot_motion_linear.py --output results\motion_linear.png
python examples\plot_cpmg_pipe_flow.py --output results\cpmg_pipe_flow.png
python examples\plot_motion_diffusion_cpmg.py --output results\motion_diffusion_cpmg.png
python examples\plot_motion_diffusion_udd.py --output results\motion_diffusion_udd.png
python examples\porous_rock_cpmg_walkers.py --backend jax --plot-output results\
    porous_rock_challenge.png --output results\porous_rock_challenge.npz
python examples\plot_radiation_damping.py --output results\radiation_damping.png
python examples\plot_radiation_damping_detuning.py --output results\rd_detuning.png
python examples\plot_radiation_damping_cpmg_train.py --output results\rd_cpmg.png
python examples\plot_mandal2015_phase_step_sweep.py --probe tuned --output results\
    mandal_phase_sweep.png
python examples\plot_mandal2015_echo_modulation.py --probe tuned --output results\
    mandal_echo_modulation.png
python examples\plot_mandal2015_pulse_shapes.py --probe tuned --output results\
    mandal_pulse_shapes.png
python examples\plot_nmr_maser.py --output results\nmr_maser.png
python examples\plot_wurst_flow.py --numpts 41 --num-steps 64 --output results\wurst_flow.
    png
python examples\plot_j_editing_spectrum.py --output results\j_editing_spectrum.png
python examples\plot_j_editing_field_spread.py --output results\j_editing_field_spread.png
python examples\plot_tango_filter.py --target 160 --output results\tango_filter.png
python examples\plot_slic_two_spin.py --j-hz 7 --delta-hz 0.7 --output results\
    slic_two_spin.png
python examples\plot_bpp_relaxation_temperature.py --output results\
    bpp_relaxation_temperature.png
python examples\plot_wall_relaxation_xe.py --output results\wall_relaxation_xe.png
python examples\plot_redfield_water_cpmg.py --output results\redfield_water_cpmg.png
python examples\plot_redfield_nano2_slse.py --output results\redfield_nano2_slse.png
python examples\plot_t1rho_prepolarized_dispersion.py --output results\
    t1rho_prepolarized_dispersion.png
python examples\plot_earth_field_prepolarized_nmr.py --output results\
    earth_field_prepolarized_nmr.png
python examples\plot_optimization_workflows.py --numpts 11 --segments 2 --starts 2 --
    inverse-starts 4 --output results\optimization_workflows.png
python examples\plot_optimization_pipeline.py --numpts 11 --refocusing-segments 2 --
    refocusing-starts 2 --excitation-segments 2 --excitation-starts 2 --inverse-starts 3 --
    output results\optimization_pipeline.png
python examples\plot_grape_cpmg_snr_per_time.py --output results\grape_cpmg_snr_per_time.
    png
python examples\plot_nqr_powder_nutation.py --output results\nqr_powder_nutation.png
python examples\plot_nqr_full_powder_nutation.py --output results\nqr_full_powder_nutation.
    png
python examples\plot_nqr_auto_model_selection.py --output results\nqr_auto_model_selection.
    png
python examples\plot_nqr_population_transfer.py --output results\nqr_population_transfer.
```

```
    png
python examples\plot_nqr_slse_offset.py --output results\nqr_slse_offset.png
python examples\plot_nqr_slse_spacing.py --output results\nqr_slse_spacing.png
python examples\plot_nqr_efg_broadening.py --output results\nqr_efg_broadening.png
python examples\plot_nqr_temperature_broadening.py --output results\
    nqr_temperature_broadening.png
python examples\plot_nqr_slse_efg_broadening.py --output results\nqr_slse_efg_broadening.
    png
python examples\plot_nqr_weak_b0_spectrum.py --output results\nqr_weak_b0_spectrum.png
python examples\plot_nqr_polarization_enhancement.py --output results\
    nqr_polarization_enhancement.png
```

11 Validation

Validation fixtures live in `validation/fixtures`. The tests compare the Python implementation against compact MATLAB or Octave arrays for core rotations, echo conversion, FID, the `arb1D` kernel, finite acquisition, probe response, imaging, pulse-shape helpers, optimization result handling, and public workflow result shapes.

Regenerate dependency-light fixtures with Octave or MATLAB from the `PythonSpinDynamics` directory:

```
matlab -batch "run('validation/octave/generate_basic_fixtures.m')"
matlab -batch "run('validation/octave/generate_imaging_fixtures.m')"
matlab -batch "run('validation/octave/generate_optimization_fixtures.m')"
matlab -batch "run('validation/octave/generate_pulse_fixtures.m')"
```

Matched-probe fixtures require MATLAB toolboxes used by the reference implementation. Octave fixture generation skips matched-probe files when `fmincon` is unavailable.

12 API Reference

This chapter documents the intended public surface. Lower-level module imports remain available for validation and porting. For application code, start with:

```
spin_dynamics.workflows
spin_dynamics.parameters
spin_dynamics.analysis
spin_dynamics.coupling
spin_dynamics.nqr
spin_dynamics.noise
spin_dynamics.motion
spin_dynamics.sequences.motion
spin_dynamics.prepolarization
spin_dynamics.relaxation
spin_dynamics.optimization
```

Drop into lower-level modules only when a building block is needed directly. Use `spin_dynamics.coupling` for the scoped dense scalar-coupled spin-1/2 tools described in Chapter 5. Use `spin_dynamics.nqr` for the selective pulsed NQR tools described in Chapter 7.

12.1 Workflow Results

Class	Purpose
`CPMGResult`	Asymptotic CPMG spectrum, echo, time vector, SNR, probe label, and phase-cycle metadata.
`CPMGTrainResult`	Finite echo-train spectra, echoes, echo integrals, sequence timing, and phase-cycle metadata.
`CPMGIRTrainResult`	Inversion-recovery finite trains over a `tauvect`.
`MatchedCPMGIRTrainResult`	Matched-probe specialization of the CPMG-IR train result.
`CPMGParameterSweepResult`	Asymptotic Q or mistuning sweeps.
`CPMGFiniteParameterSweepResult`	Finite-train Q or mistuning sweeps.
`ZMagnetizationSweepResult`	Matched-probe z-magnetization Q sweeps.
`IdealTimeVaryingCPMGResult`	Final echo for ideal time-varying-field CPMG.
`ProbeTimeVaryingCPMGResult`	Probe-aware time-varying-field CPMG result.
`IdealCPMGImagingResult`	Ideal phase-encoded CPMG imaging result.
`ProbeCPMGImagingResult`	Tuned or matched phase-encoded CPMG imaging result.
`FrequencyEncodedImagingResult`	Spin-warp / RARE frequency-encoded image, k-space, and per-line echo times.
`SliceSensitivityResult`	Real-space sensitive slice (excited/refocused, B1-weighted) of an excitation in a non-uniform field.
`ImagingEchoFitResult`	Voxel-wise mono-exponential echo-stack fit.
`ImagingNoiseStatistics`	Repeated-trial image-domain noise summary.

`MotionSequenceResult`	Moving-isochromat sequence samples and final particle ensemble.
`HalbachDipoleFieldMaps`	Sampled 3-D finite Halbach dipole field maps, including rods and Cartesian field components.
`NoiseMetadata`	Generated received-noise parameters and realized SNR.
`MatchedDiffusionCPMGResult`	Matched diffusion-aware CPMG train.
`MatchedDiffusionQSweepResult`	Matched diffusion Q sweep.
`PGSEMomentResult`	Deterministic PGSE or PGSE-prepared CPMG attenuation from gradient moments.
`PGSEWalkerResult`	Random-walker PGSE signal, sequence samples, and initial particle ensemble.
`PGSTEWalkerResult`	Random-walker stimulated-echo (PGSTE) signal, storage interval, selected-pathway phase-cycle metadata, and initial particle ensemble.
`QSpaceReconstructionResult`	Direct q-space inverse image or autocorrelation with matched real/q axes.
`QSpacePhaseRetrievalResult`	Magnitude-only q-space pore-shape estimate, support mask, and residual history.
`DDEWalkerResult`	Random-walker double diffusion encoding (DDE) signal, encoding angles, and mixing time.
`OGSEWalkerResult`	Random-walker oscillating-gradient spin-echo (OGSE) signal, oscillation frequency, and b-value.
`BipolarPGSTEResult`	Cotts 13-interval bipolar PGSTE moment model: b-value, background cross-term bias, and the `diff_stebp` phase cycle.
`BipolarPGSTEWalkerResult`	Explicit moving-walker bipolar 13-interval PGSTE: acquired echo, moment-model b-value, sequence, and ensemble.
`WURSTInversionResult`	WURST inversion profile and pulse metadata.
`MatchedWURSTCPMGResult`	Matched WURST-CPMG echoes and spectra.
`RadiationDampingFIDResult`	Radiation-damping FID simulation and analytic comparison fields.
`MultiStartOptimizationResult`	Repeated random-start pulse-optimization result.
`RefocusingOptimizationResult`	Bounded fixed-amplitude refocusing phase optimization.
`ExcitationOptimizationResult`	Bounded fixed-amplitude target-excitation or inverse-excitation optimization.
`TunedExcitationInversePipelineResult`	Selected-refocusing to excitation/inverse-excitation pipeline result.
`SLSEResult`	NQR SLSE echo train, local orientation signals, and transition metadata.
`SLSESweepResult`	NQR SLSE selected-echo amplitudes and effective decay estimates across a sweep.
`NQRFIDDistributionResult`	EFG-distribution FID, FFT spectrum, and isochromat metadata.
`SLSEAcquisitionSpectrumResult`	Finite-window SLSE echo acquisition and spectrum, including optional noise and deconvolution metadata.

`WeakB0SpectrumResult`	Static weak-B0 transition spectrum and perturbation-ratio metadata.
`PopulationTransferResult`	NQR perturbation-plus-SLSE signal, reference signal, and normalized difference.
`PolarizationTransferResult`	Polarization-enhanced NQR transport result with sampled trajectory fields, adiabaticity metrics, and ideal enhancement factors.
`ProtonCouplingEstimate`	CIF-based dipole-dipole proton coupling estimate for one quadrupolar nucleus.
`RelaxationExchange2DResult`	Encode-mix-detect T2-T2 relaxation exchange (REXSY) data and longitudinal mixing propagator.

12.2 Prepolarization and Relaxation API

```
from spin_dynamics.prepolarization import (
    PrepolarizedMagnetization,
    longitudinal_recovery,
    field_ratio_equilibrium,
    prepolarized_magnetization,
    prepolarized_state,
    residence_time_seconds,
    prepolarized_flow_state,
    apply_prepolarization_to_parameters,
)

from spin_dynamics.relaxation import (
    BPPRelaxationModel,
    BPPRelaxationRates,
    DipolarRelaxationSource,
    IsotropicLiquidMotionalAveraging,
    PhenomenologicalRelaxationModel,
    RedfieldDipolarRelaxationModel,
    RigidSolidMotionalAveraging,
    spectral_density_lorentzian,
    arrhenius_correlation_time,
    bpp_relaxation_rates,
    apply_relaxation_to_parameters,
)
```

Symbol	Purpose
`PrepolarizedMagnetization`	Prepared `m0`, sequence `mth`, and enhancement arrays for handoff to Bloch kernels or moving isochromats.
`prepolarized_state`	Field-ratio-normalized finite-time prepolarization for static samples.
`prepolarized_flow_state`	Same buildup model with residence time computed from path length and speed.
`BPPRelaxationModel`	Arrhenius $\tau_c(T)$, Lorentzian spectral-density, and configurable BPP R_1/R_2 model.

`BPPRelaxationRates`	Temperature, τ_c, $J(0)$, $J(\omega_0)$, $J(2\omega_0)$, R_1/R_2, and T_1/T_2 arrays.
`PhenomenologicalRelaxationModel`	Shared Liouville-space population/coherence damping from supplied T_1/T_2 values.
`DipolarRelaxationSource`	Target–bath dipolar source geometry, gyromagnetic ratios, bath spin, and optional explicit coupling.
`RigidSolidMotionalAveraging`	Fixed-frame dipolar covariance for rigid solids and NQR-style orientation-dependent relaxation.
`IsotropicLiquidMotionalAveraging`	Rotationally averaged dipolar covariance for liquid-state NMR.
`RedfieldDipolarRelaxationModel`	Secular Markovian Liouville-space dissipator built from stochastic dipolar bath sources.

12.3 High-Level Workflows

```
from spin_dynamics.workflows import (
    run_ideal_cpmg, run_tuned_cpmg, run_untuned_cpmg, run_matched_cpmg,
    run_ideal_cpmg_train, run_tuned_cpmg_train,
    run_untuned_cpmg_train, run_matched_cpmg_train,
    run_ideal_cpmg_ir_train, run_tuned_cpmg_ir_train,
    run_untuned_cpmg_ir_train, run_matched_cpmg_ir_train,
    run_tuned_q_sweep, run_matched_q_sweep,
    run_tuned_mistuning_sweep, run_matched_mistuning_sweep,
    run_tuned_finite_q_sweep, run_untuned_finite_q_sweep,
    run_matched_finite_q_sweep,
    run_tuned_finite_mistuning_sweep,
    run_untuned_finite_mistuning_sweep,
    run_matched_finite_mistuning_sweep,
    run_matched_z_magnetization_q_sweep,
    run_matched_diffusion_cpmg, run_matched_diffusion_q_sweep,
    run_pgse, run_pgse_moment, run_pgse_walkers, run_pgste_walkers,
    run_dde_walkers, run_ogse_walkers,
    pgse_b_value, gradient_moment_b_value,
    run_ideal_time_varying_cpmg_final,
    run_tuned_time_varying_cpmg_final,
    run_untuned_time_varying_cpmg_final,
    run_matched_time_varying_cpmg_final,
    run_ideal_time_varying_amplitude_sweep,
    run_tuned_time_varying_amplitude_sweep,
    run_untuned_time_varying_amplitude_sweep,
    run_matched_time_varying_amplitude_sweep,
    sinusoidal_field_waveform,
    run_ideal_wurst_inversion,
    run_matched_wurst_inversion,
    run_matched_wurst_cpmg,
    run_radiation_damping_fid,
)
```

The main CPMG runners share `numpts` and `maxoffs`. Finite train runners add `num_echoes`, relaxation constants, rephasing controls, optional `auto_refine_grid`, and optional worker controls. Probe-aware train runners also accept `q_value` and `mistuning_offset` where the underlying probe model supports

them.

12.4 Imaging API

```
from spin_dynamics.workflows import (
    ImagingFieldMaps,
    make_imaging_field_maps,
    load_imaging_field_maps_npz,
    run_ideal_phase_encoded_cpmg_imaging,
    run_tuned_phase_encoded_cpmg_imaging,
    run_matched_phase_encoded_cpmg_imaging,
    run_t1_encoded_phase_encoded_cpmg_imaging,
    run_spin_warp_imaging, run_rare_imaging, imaging_slice_sensitivity,
    make_slice_selective_excitation, simulate_slice_profile,
    slice_excitation_weights, slice_profile_table,
    run_multislice_imaging, run_multislice_imaging_separable,
    MultiSliceImagingResult,
    reconstruct_image_from_kspace,
    form_imaging_image,
    fit_imaging_echo_decay,
    summarize_imaging_noise_trials,
)
```

Prefer the `phase_encoded` names for new code. The shorter compatibility aliases remain available:

```
run_ideal_cpmg_imaging
run_tuned_cpmg_imaging
run_matched_cpmg_imaging
```

12.5 Parameter Constructors

```
from spin_dynamics.parameters import (
    set_params_ideal,
    set_params_ideal_fid,
    set_params_tuned_orig,
    set_params_untuned_orig,
    set_params_matched_orig,
    set_params_tuned_spa,
    set_params_untuned_spa,
    set_params_matched_spa,
    set_params_tuned_jmr,
    set_params_untuned_jmr,
    set_params_matched_jmr,
)
```

These constructors return dataclass instances mirroring MATLAB `sp`, `pp`, and `params` structures. They accept optional `numpts` arguments for lightweight tests and examples while preserving MATLAB defaults when omitted.

12.6 Analysis API

```
from spin_dynamics.analysis import (
    t2_kernel, t1_kernel, diffusion_kernel, laplace_kernel,
    invert_laplace_1d, invert_laplace_2d,
    invert_t2, invert_t1, invert_t1_t2, invert_d_t2, invert_t2_t2,
    default_regularization_strengths,
    estimate_noise_rms_from_snr,
    expected_residual_norm_from_snr,
    select_regularization_1d,
    select_regularization_2d,
)
```

For one-dimensional relaxation distributions, use:

```
invert_t2
invert_t1
```

For separable two-dimensional maps, use:

```
invert_t1_t2
invert_d_t2
invert_t2_t2
```

The `select_regularization_*` helpers choose Tikhonov strengths from an RMS SNR estimate.

12.6.1 Chemical / Site Exchange

The `spin_dynamics.exchange` namespace provides the Bloch-McConnell site exchange model described in Section 9.17:

```
from spin_dynamics.exchange import (
    ExchangeSite, ExchangeSystem, RelaxationExchange2DResult,
    two_site_exchange, exchange_generator, transverse_generator,
    transverse_propagator, mixing_propagator,
    simulate_exchange_fid, exchange_spectrum,
    simulate_relaxation_exchange_2d,
)
```

12.6.2 Internal / Susceptibility Gradients

The `spin_dynamics.susceptibility` namespace provides the internal-field generator described in Section 9.18:

```
from spin_dynamics.susceptibility import (
    CylindricalInclusion, SusceptibilityField, InternalGradientDistribution,
    susceptibility_offresonance_map, make_susceptibility_field_maps,
    internal_gradient_maps, internal_gradient_distribution,
)
```

12.7 Optimization Diagnostics

Optimization helpers expose compact NumPy/SciPy-compatible diagnostics for phase-program searches:

```
from spin_dynamics.optimization import run_tuned_refocusing_multistart

result = run_tuned_refocusing_multistart(num_segments=3, num_starts=4)
```

The plotting-free analysis helpers convert MATLAB-style result cells into score summaries, and the tuned excitation/inverse pipeline carries a selected refocusing candidate into bounded target-excitation and inverse-cancellation tests. The inverse stage is still best treated as a parity and diagnostics surface while strong inverse-pulse cancellation remains under validation.

12.8 Core Numerical API

```
from spin_dynamics.core.echo import (
    calc_time_domain_echo,
    calc_time_domain_echo_arb,
    calc_fid_time_domain,
)
from spin_dynamics.core.rotations import (
    calc_rotation_matrix,
    calc_rot_axis_arba3,
    calc_rot_axis_arba4,
    calc_v0crit,
    sim_spin_dynamics_asymp_mag3,
    sim_spin_dynamics_exc,
)
from spin_dynamics.core.kernels import (
    sim_spin_dynamics_arb7,
    sim_spin_dynamics_arb10,
    sim_spin_dynamics_arb10_chunked,
    sim_spin_dynamics_arb10_diffusion,
    sim_spin_dynamics_arb10_radiation_damping,
)
from spin_dynamics.core.isochromats import (
    analyze_rephasing,
    check_rephasing,
    estimate_rephase_time,
    recommended_numpts_for_rephasing,
)
```

Core functions are intended for validation, debugging, and continued porting. They are closer to MATLAB's array contracts than the workflow layer.

12.9 Probe and Pulse API

```
from spin_dynamics.probes.tuned import (
    tuned_probe_lp,
    tuned_probe_rx,
```

```
    tuned_probe_rx_tf,
    calc_rot_axis_tuned_probe_lp_orig2,
    calc_masy_tuned_probe_lp_orig,
)
from spin_dynamics.probes.untuned import (
    untuned_probe_lp,
    untuned_probe_rx,
    untuned_probe_rx_tf,
    calc_rot_axis_untuned_probe_lp,
    calc_masy_untuned_probe_lp,
)
from spin_dynamics.probes.matched import (
    matching_network_design2,
    find_coil_current,
    find_coil_current_wurst,
    matched_probe_rx,
    calc_rot_axis_matched_probe,
    calc_masy_matched_probe_orig,
    calc_masy_matched_nut,
)
from spin_dynamics.pulses import (
    quantize_phase,
    create_wurst_pulse,
    adjust_untuned_segment_lengths,
    tuned_rectangular_pulse_response,
    untuned_rectangular_pulse_response,
    matched_rectangular_pulse_response,
    matched_wurst_pulse_response,
)
```

12.10 Noise, Motion, and Radiation Damping

```
from spin_dynamics.noise import (
    NoiseSpec,
    NoiseMetadata,
    MatchedFilterSNRResult,
    add_received_noise,
    estimate_matched_filter_snr,
    ideal_noise_density,
    tuned_probe_output_noise_density,
    untuned_probe_output_noise_density,
    matched_probe_output_noise_density,
)
from spin_dynamics.motion import (
    ParticleEnsemble,
    MotionFieldMaps2D,
    MotionFieldMaps,
    make_motion_field_maps_2d,
    make_motion_field_maps,
    initialize_ensemble_from_density,
    initialize_ensemble_from_domain,
    make_semipermeable_plane,
    advect_diffuse_positions,
```

```
    move_ensemble,
    apply_free_precession,
    apply_rf_rotation,
    free_precession_with_motion_step,
    receive_signal,
    transverse_b1_magnitude,
)
from spin_dynamics.sequences.motion import (
    MotionSequenceStep,
    MotionSequenceResult,
    make_motion_cpmg_sequence,
    make_motion_udd_sequence,
    run_motion_sequence,
    run_motion_cpmg_sequence,
    run_motion_udd_sequence,
)
from spin_dynamics.fields import (
    SpatialDomain,
    SpatialFieldMaps,
    FiniteMagnetRod,
    HalbachDipoleFieldMaps,
    finite_magnet_array_b0,
    halbach_dipole_magnets,
    sample_halbach_dipole_field,
)
from spin_dynamics.fields.magnetostatics import (
    BarMagnet,
    FiniteMagnetRod,
    HalbachDipoleFieldMaps,
    bar_array_b0,
    biot_savart,
    circular_loop,
    finite_magnet_array_b0,
    halbach_dipole_magnets,
    nmr_mouse_magnets,
    sample_halbach_dipole_field,
    sample_magnet_field,
)
from spin_dynamics.workflows import (
    make_slice_selective_excitation,
    simulate_slice_profile,
    run_multislice_imaging,
    run_multislice_imaging_separable,
)
from spin_dynamics.workflows.single_sided import (
    SampleLayer,
    LayeredSample,
    mouse_depth_profile,
    measure_diffusion_at_depth,
    simulate_mouse_cpmg,
    resonant_depth,
)
from spin_dynamics.sequences import (
    cpmg_pulse_times,
    udd_pulse_times,
```

```
    interval_durations,
    toggling_frame_integral,
    dephasing_filter_function,
)
from spin_dynamics.radiation_damping import (
    radiation_damping_time,
    proton_thermal_magnetization_density,
    water_proton_sample,
    hyperpolarized_proton_sample,
    normalized_radiation_damping_weights,
    radiation_damping_probe_from_tuned,
    radiation_damping_probe_from_matched,
    initial_state_from_flip_angle,
    analytic_radiation_damping_envelope,
    simulate_radiation_damping,
    simulate_radiation_damping_fid,
    simulate_nmr_maser,
)
```

12.11 Scalar Coupling API

```
from spin_dynamics.coupling import (
    CoupledSpinSystem,
    CoupledIsochromatEnsemble,
    CoupledIsochromatStep,
    CoupledIsochromatSequenceResult,
    coupled_spin_system,
    coupled_isochromat_ensemble,
    free_precession_step,
    rf_step,
    simulate_coupled_isochromat_sequence,
    spin_operator,
    total_operator,
    product_operator,
    zeeman_hamiltonian,
    secular_j_hamiltonian,
    isotropic_j_hamiltonian,
    rf_hamiltonian,
    equilibrium_density,
    evolve_density,
    expectation,
    j_modulation_curve,
    proton_detected_j_modulation,
    carbon_detected_j_modulation,
    tango_b_filter,
    fit_known_j_spectrum,
    JEditingFitResult,
    simulate_slic_spectrum,
    two_spin_slic_transfer_time,
    SLICSpectrumResult,
)
```

Object	Purpose
`CoupledSpinSystem`	Validated small spin-1/2 system with offsets and scalar couplings in hertz.
`CoupledIsochromatEnsemble`	Ensemble of local B0/B1 field samples for one coupled-spin system.
`free_precession_step`, `rf_step`	Sequence-step builders with optional time-varying B0/B1 overrides.
`simulate_coupled_isochromat_sequence`	Dense coupled-spin sequence propagation over an isochromat ensemble.
`spin_operator`, `total_operator`, `product_operator`	Dense spin-1/2 operator builders.
`zeeman_hamiltonian`	Rotating-frame offset Hamiltonian in radians per second.
`secular_j_hamiltonian`	Weak-coupling $I_z I_z$ scalar Hamiltonian.
`isotropic_j_hamiltonian`	Isotropic scalar Hamiltonian for strongly coupled spin-1/2 systems.
`rf_hamiltonian`	RF nutation Hamiltonian for selected spins.
`equilibrium_density`, `evolve_density`, `expectation`	Minimal dense density-matrix utilities.
`j_modulation_curve`	Sum of cosine J-modulation components.
`proton_detected_j_modulation`	Proton-detected low-field J-editing response.
`carbon_detected_j_modulation`	Carbon-detected $\cos^n$ J-editing response for CH_n groups.
`tango_b_filter`	Ideal TANGO-B scalar-coupling filter envelope.
`fit_known_j_spectrum`	Least-squares amplitudes for supplied J-coupling positions.
`simulate_slic_spectrum`	Spin-lock response scan for dense coupled-spin systems.

12.12 ESR API

```
from spin_dynamics.esr import (
    ESRSpinSystem,
    ESRRelaxationModel,
    ESRDistributionSample,
    NuclearSite,
    ElectronNuclearSystem,
    resonance_frequency_hz,
    resonance_field_tesla,
    effective_g_value,
    simulate_field_sweep,
    simulate_frequency_spectrum,
    static_disorder_grid,
    simulate_field_sweep_distribution,
    simulate_frequency_spectrum_distribution,
    gaussian_lineshape,
```

```
    lorentzian_lineshape,
    flip_angle_duration,
    simulate_fid,
    simulate_hahn_echo,
    electron_nuclear_system,
    diagonalize_hyperfine_system,
    simulate_hyperfine_field_sweep,
)
```

Object	Purpose
`ESRSpinSystem`	Single-electron spin-1/2 ESR system with scalar, diagonal, or full g tensor.
`resonance_frequency_hz`, `resonance_field_tesla`	Convert between field and microwave frequency for one ESR orientation.
`effective_g_value`	Direction-dependent $g_{\mathrm{eff}} = \lvert\mathbf{g}^{\mathsf{T}}\hat{\mathbf{n}}\rvert$.
`simulate_field_sweep`, `simulate_frequency_spectrum`	Single-crystal or powder CW ESR spectra with Gaussian/Lorentzian absorption or derivative display.
`gaussian_lineshape`, `lorentzian_lineshape`	Unit-height absorption profiles and first derivatives.
`static_disorder_grid`	Weighted diagonal g-strain and applied-field-offset samples.
`simulate_field_sweep_distribution`	Field-swept ESR spectrum averaged over static disorder samples.
`flip_angle_duration`	Rectangular-pulse duration for a spin-1/2 flip angle and nutation rate.
`simulate_fid`, `simulate_hahn_echo`	Rotating-frame pulsed ESR FID and Hahn-echo density-matrix simulations.
`ESRRelaxationModel`	Phenomenological Liouville-space T_1/T_2 relaxation model for pulsed ESR.
`NuclearSite`, `ElectronNuclearSystem`	Dense electron-nuclear product-space hyperfine model containers.
`electron_nuclear_system`	Convenience builder for isotropic electron-nuclear hyperfine systems.
`diagonalize_hyperfine_system`	Exact dense energy levels and ESR-active transitions for a hyperfine system.
`simulate_hyperfine_field_sweep`	Field-swept ESR spectrum including isotropic electron-nuclear hyperfine coupling.

12.13 Interference and RFI API

```
from spin_dynamics.interference import (
    AcquisitionMask,
    SampleLabel,
    mask_from_intervals,
    RFIWaveform,
    tone_waveform,
    am_carrier_waveform,
    chirp_waveform,
```

```
    colored_noise_waveform,
    impulsive_waveform,
    UniformPlaneWaveSource,
    MagneticDipoleSource,
    ReferenceCoil,
    coil_voltage,
    reference_matrix,
    scalar_canceller,
    gated_ridge_fir_canceller,
    robust_fir_canceller,
    sparse_reference_canceller,
    frequency_domain_canceller,
    kalman_harmonic_canceller,
    windowed_ridge_fir_canceller,
    adaptive_lms_canceller,
    adaptive_rls_canceller,
    joint_signal_reference_canceller,
    windowed_joint_signal_reference_canceller,
    rfi_suppression_db,
    matched_filter_snr_improvement,
    signal_bias,
    residual_spectral_lines,
    reference_design_diagnostics,
    reference_noise_injection,
    saturation_diagnostics,
    CompensationActuator,
    feedforward_cancel,
    RFIRecording,
    save_rfi_recording,
    load_rfi_recording,
)

from spin_dynamics.nqr import slse_acquisition_mask, sorc_acquisition_mask
```

Object	Purpose
`AcquisitionMask`, `SampleLabel`	Per-sample transmit, ringdown, signal, and baseline labels on one shot clock.
`mask_from_intervals`	Build a generic acquisition mask from second-valued intervals.
`slse_acquisition_mask`, `sorc_acquisition_mask`	Convert NQR SLSE or SORC sequence timing into an acquisition mask with optional baseline windows.
`RFIWaveform`	Real sampled interference waveform with a sample rate.
`tone_waveform`, `am_carrier_waveform`	Narrowband and amplitude-modulated RFI waveform helpers.
`chirp_waveform`, `colored_noise_waveform`, `impulsive_waveform`	Broadband, drifting, and transient RFI waveform helpers.
`UniformPlaneWaveSource`	Far-field RFI source with spatially uniform vector polarization.

`MagneticDipoleSource`	Near-field magnetic-dipole source with $1/r^3$ spatial dependence.
`ReferenceCoil`	Pickup coil with vector sensitivity, position, optional noise, saturation, and NQR leakage.
`coil_voltage`, `reference_matrix`	Simulate one pickup channel or stacked reference matrix for a set of sources.
`scalar_canceller`, `gated_ridge_fir_canceller`	Stationary masked least-squares reference cancellers.
`robust_fir_canceller`	Outlier-robust Huber IRLS reference canceller for impulsive switching-transient pickup.
`sparse_reference_canceller`	Splits the primary into reference-explained RFI and an L1-sparse impulse term, modelling and removing the bursts.
`frequency_domain_canceller`	Per-frequency multi-reference Wiener canceller (WOLA) for frequency-dependent coupling; returns the transfer and coherence spectra.
`kalman_harmonic_canceller`	Reference-free Kalman tracker of drifting narrowband carriers, with mask-aware update freezing.
`windowed_ridge_fir_canceller`	Offline RFI canceller with one regularized reference transfer function per time window.
`adaptive_lms_canceller`, `adaptive_rls_canceller`	Mask-aware adaptive cancellers whose coefficient updates can freeze during signal windows.
`joint_signal_reference_canceller`	Fixed joint fit of reference-derived RFI and a supplied signal basis.
`windowed_joint_signal_reference_canceller`	Offline echo-aware joint fit with windowed reference coefficients and global signal-basis coefficients.
`rfi_suppression_db`	RMS RFI suppression in dB, optionally relative to a known clean signal.
`matched_filter_snr_improvement`	Matched-filter SNR before and after cleanup.
`signal_bias`	Complex gain, amplitude bias, and phase bias of the recovered signal.
`residual_spectral_lines`	Largest residual FFT lines after cancellation.
`reference_design_diagnostics`	Rank and condition number of a tapped reference design matrix.
`reference_noise_injection`	White noise a canceller injects into the cleaned output from noisy references (the "noise figure" of cancellation).
`saturation_diagnostics`	Samples whose magnitude reaches or exceeds a specified front-end threshold.
`CompensationActuator`, `feedforward_cancel`	Active feedforward: a compensation coil subtracts RFI before the ADC, with latency, drive-range, gain/phase, and noise limits.
`RFIRecording`, `save_rfi_recording`, `load_rfi_recording`	Container and ".npz" I/O pairing measured ADC windows with a reconstructed acquisition mask.
`nqr_recording_from_samples`, `slse_/sorc_mask_from_metadata`	Rebuild the gating for a window of ADC samples from echo-spacing metadata (in `spin_dynamics.nqr`).

12.14 NQR API

```
from spin_dynamics.nqr import (
    QuadrupolarSite,
    NQRTransition,
    NQREigensystem,
    spin_matrices,
    quadrupole_frequency_scale_hz,
    quadrupole_hamiltonian,
    zeeman_hamiltonian,
    nqr_hamiltonian,
    diagonalize_site,
    OrientationSample,
    single_crystal_orientation,
    powder_average_grid,
    b0_b1_powder_average_grid,
    b0_powder_average_grid,
    SelectivePulse,
    apply_selective_pulse,
    transition_drive_scale,
    NQRRelaxationModel,
    slse_sequence,
    sorc_sequence,
    simulate_slse,
    simulate_sorc,
    simulate_slse_offset_sweep,
    simulate_slse_spacing_sweep,
    sorc_powder_theory_signal,
    sorc_powder_pathway_signal,
    fid_powder_theory_signal,
    gaussian_efg_distribution,
    temperature_efg_distribution,
    simulate_fid_efg_distribution,
    simulate_slse_acquisition_spectrum,
    simulate_weak_b0_spectrum,
    weak_field_ratio,
    zeeman_frequency_hz,
    simulate_population_transfer,
    PolarizationEnhancedNQRSample,
    CylindricalSampleGeometry,
    LinearTransportMotion,
    HalbachPrepolarizationMagnet,
    ideal_spin1_enhancement_factors,
    simulate_adiabatic_polarization_transfer,
    load_cif_structure,
    dipolar_coupling_hz,
    estimate_proton_dipolar_couplings,
    estimate_proton_dipolar_couplings_from_cif,
    SLSEResult,
    PopulationTransferResult,
    PolarizationTransferResult,
    ProtonCouplingEstimate,
    WeakB0SpectrumResult,
)
```

Object	Purpose
`QuadrupolarSite`	One quadrupolar nucleus in the EFG principal-axis system.
`spin_matrices`	Dense angular-momentum operators for arbitrary integer or half-integer spin.
`quadrupole_frequency_scale_hz`	Spin-dependent Hamiltonian scale for spin-1 and spin-3/2 frequency conventions.
`quadrupole_hamiltonian`	Zero-field quadrupole Hamiltonian in radians per second.
`zeeman_hamiltonian`	Optional weak-B0 perturbation in radians per second.
`diagonalize_site`	Energy levels, transition frequencies, and transition dipole vectors.
`OrientationSample`	One EFG orientation relative to lab RF and optional B0 fields.
`single_crystal_orientation`	One fixed orientation sample.
`powder_average_grid`	Normalized spherical powder quadrature grid.
`b0_b1_powder_average_grid`	Powder grid with correlated static-field and RF-field directions.
`SelectivePulse`	Rectangular selective pulse on one transition.
`apply_selective_pulse`	Embedded two-level RF propagation in the energy basis.
`NQRRelaxationModel`	Phenomenological Liouville-space population/coherence relaxation rates.
`slse_sequence`	Convenience builder for SLSE detection.
`sorc_sequence`	Convenience builder for the strong off-resonance comb sequence.
`simulate_slse`	Selective-pulse SLSE echo train for a fixed orientation or powder.
`simulate_sorc`	Finite-pulse SORC pathway train by default, with optional coherent transient propagation.
`simulate_slse_offset_sweep`, `simulate_slse_spacing_sweep`	SLSE diagnostics versus RF offset or pulse period.
`sorc_powder_theory_signal`, `sorc_powder_pathway_signal`	Infinite-N and finite-N powder SORC responses for Konnai-style comparisons.
`fid_powder_theory_signal`	Spin-1 powder FID pulse-width response used beside SORC pulse-width sweeps.
`gaussian_efg_distribution`, `temperature_efg_distribution`	Static EFG isochromat distributions.
`simulate_fid_efg_distribution`	Time-domain FID and FFT spectrum from an EFG distribution.
`simulate_slse_acquisition_spectrum`	Finite-window acquired SLSE echo and spectrum, with noise and optional deconvolution.
`simulate_weak_b0_spectrum`	Static weak-B0 transition spectrum for spin-1 or spin-3/2.
`weak_field_ratio`, `zeeman_frequency_hz`	Weak-field regime checks based on $\lvert\gamma B_0\rvert/\nu_{\mathrm{ref}}$.

`simulate_population_transfer`	Perturbation pulse plus SLSE detection for 2D NQR diagnostics.
`PolarizationEnhancedNQRSample`	Sample and spin parameters for polarization-enhanced NQR transport.
`CylindricalSampleGeometry`, `LinearTransportMotion`	Detector-volume and conveyor/translation parameters.
`HalbachPrepolarizationMagnet`	Links a sampled Halbach dipole map or constant field to the polarization-transfer model.
`ideal_spin1_enhancement_factors`	Glickstein–Mandal spin-1 enhancement factors from B_{p}, T, and NQR line frequencies.
`simulate_adiabatic_polarization_transfer`	Compute trajectory fields, adiabaticity, residence loss, and ideal transferred NQR signal gains.
`load_cif_structure`	Read fractional coordinates, lattice constants, and symmetry operations from a CIF file.
`dipolar_coupling_hz`	Point-dipole proton–quadrupolar-nucleus coupling estimate for a specified separation and angle.
`estimate_proton_dipolar_couplings`	Find nearby protons around a quadrupolar nucleus in a loaded CIF structure.
`estimate_proton_dipolar_couplings_from_cif`	Load a CIF file and summarize the nearby-proton dipolar coupling scale.

12.15 Optimization API

```
from spin_dynamics.optimization import (
    spa_pulse_list,
    rectangular_refocusing_lengths,
    evaluate_tuned_refocusing_pulse,
    evaluate_untuned_refocusing_pulse,
    evaluate_matched_refocusing_pulse,
    evaluate_ideal_v0crit_refocusing_pulse,
    evaluate_ideal_v0crit_excited_refocusing_pulse,
    evaluate_ideal_time_varying_refocusing_pulse,
    optimize_tuned_refocusing_phases,
    optimize_untuned_refocusing_phases,
    optimize_matched_refocusing_phases,
    optimize_ideal_v0crit_refocusing_phases,
    optimize_ideal_v0crit_excited_refocusing_phases,
    optimize_ideal_time_varying_refocusing_phases,
    run_tuned_refocusing_multistart,
    run_untuned_refocusing_multistart,
    run_matched_refocusing_multistart,
    run_ideal_v0crit_refocusing_multistart,
    run_ideal_v0crit_excited_refocusing_multistart,
    run_ideal_time_varying_refocusing_multistart,
    evaluate_tuned_excitation_pulse,
    evaluate_tuned_inverse_excitation_pulse,
    optimize_tuned_excitation_phases,
    optimize_tuned_inverse_excitation_phases,
    run_tuned_excitation_multistart,
    run_tuned_inverse_excitation_multistart,
```

```
    run_tuned_excitation_inverse_pipeline,
    multistart_to_matlab_results,
    save_multistart_results_npz,
    load_optimization_results,
    summarize_matlab_results,
    select_matlab_result_program,
    analyze_matlab_optimization_results,
    analyze_tuned_inverse_result_pair,
)
```

The optimization layer is useful for compact pulse-design studies and MATLAB-style result inspection. Broad historical `fmincon` parity and strong inverse-excitation cancellation parity remain validation targets.

12.16 Optimal Control API

```
from spin_dynamics.optimal_control import (
    ControlBounds,
    ControlHamiltonianModel,
    GrapeMultiStartResult,
    GrapeOptimizationResult,
    amplitude_phase_bounds,
    assemble_control_bounds,
    average_gate_fidelity,
    bloch_vector_to_ket,
    constant_gradient_seed,
    coupled_spin_control_model,
    eddy_terms_from_step_response,
    export_to_pulse_shape,
    grape_optimize,
    grape_optimize_phase_only,
    gradient_bounds,
    gradient_control_operator,
    make_grape_objective,
    nqr_fundamental_states,
    nqr_powder_control_batch,
    nqr_site_control_model,
    phase_only_bounds,
    position_gradient_batch,
    random_phase_starts,
    rectangular_seed_phase,
    robust_ensemble_fidelity,
    run_grape_multistart,
    state_transfer_fidelity,
    su2_effective_axis,
    # hardware response (probe / gradient driver)
    TunedProbeResponse,
    MatchedProbeResponse,
    UntunedProbeResponse,
    GradientDriverResponse,
    ReceiverResponse,
    TailSpec,
    # PGSE diffusion
    gradient_moments,
```

```
    detected_echo_signal,
    detected_echo_snr,
    make_pgse_objective,
)
```

`coupled_spin_control_model` builds the fixed `ControlHamiltonianModel` (drift plus transverse RF-drive operators) from an existing `coupling.systems.CoupledSpinSystem`; pass `include_gradient=True` to also populate the gradient control operator. Everything beyond Hamiltonian construction requires the optional `jax` extra – reverse-mode autodiff is the entire point of GRAPE, so no finite-difference fallback is provided. `grape_optimize_phase_only` is the primary entry point (fixed amplitude, phase-only control); `grape_optimize` generalizes to joint amplitude+phase control given a peak-B1 bound (`amplitude_max_hz`) and, via `gradient_operator_batch`/`optimize_gradient`, to the gradient control channel (`gradient_control_operat` `position_gradient_batch`, `constant_gradient_seed`, `gradient_bounds`). `nqr_site_control_model` builds the model for a quadrupolar (NQR) site in the rotating frame (Section 9.21.6), with `nqr_fundamental_states` and `nqr_powder_control_batch` for the inversion endpoints and the per-orientation drive operators; NQR optimizations must pass `propagator="expm"` because the Kramers-degenerate spectrum makes the default `eigh` gradient undefined. `run_grape_multistart` is not optional in practice: GRAPE landscapes are multimodal, so a single-start result is not a trustworthy optimum on its own. The hardware-response classes (`TunedProbeResponse`, `MatchedProbeResponse`, `UntunedProbeResponse`, `GradientDriverResponse`) fold finite probe bandwidth and gradient slew/eddy dynamics into the optimization as LTI actuators (Section 9.21.2); pass them as `rf_response` / `gradient_response` to `grape_optimize`. `ReceiverResponse` is the output-side filter for detected-signal objectives, and `eddy_terms_from_step_response` fits eddy terms from a measured step. `make_pgse_objective` (with `gradient_moments`, `detected_echo_signal`, `detected_echo_snr`) is the PGSE diffusion objective co-optimizing RF phase and gradient waveform for correct q-vector and detected SNR on a tuned probe with eddy currents (Section 9.21.3).

13 Known Gaps

The Python port is intentionally conservative where MATLAB parity still matters. The largest remaining gaps fall into three groups. First are parity and validation gaps: broader high-Q matched-probe validation, full historical MATLAB `.mat` structure parity, broad `fmincon` parity, and paper-level scalar-coupling fixture comparisons. Second are physics extensions: relaxation-aware dense coupled-spin pulse-sequence runners, full nonselective quadrupolar pulse propagation, experimental NQR fixture comparisons, ESR anisotropic hyperfine and multi-electron interactions, and coherent many-spin dipolar networks beyond the stochastic Redfield treatment. Third are implementation extensions, especially acceleration backends beyond the current NumPy implementation.

New work should add small reference fixtures first, then a NumPy implementation, then optional acceleration or plotting once numerical behavior is pinned down.